

University of Prince Edward Island

CS-4820: Software System Development Project

Dr. David LeBlanc

14 April 2026

Project:

Developing a Health-Data Bank for Precariously Housed Individuals

Final Project Plan

Submitted by Team 1:

Sam Alan

Alex O'Brien

Poorvi Bagga

Joshua MacKinnon

Sonal Seewooruttun

Table Of Contents

Table Of Contents.....	1
Vision Statement.....	7
Key Technical & Operational Requirements.....	8
1. Technical Requirements.....	8
1.1 Application Architecture.....	8
1.2 Frontend Technologies.....	8
1.3 Backend Technologies.....	8
1.4 Database and Data Storage.....	9
1.5 Security and Privacy.....	9
1.6 Reporting and Analytics.....	9
1.7 Notifications and Communication.....	10
1.8 Development and Deployment Tools.....	10
2. Operational Requirements.....	10
2.1 Accessibility.....	10
2.2 Availability and Reliability.....	10
2.3 Performance.....	10
2.4 Data Governance and Compliance.....	11
2.5 Operational Constraints.....	11
2.6 Maintainability.....	11
System Acceptance Tests.....	12
Use Cases.....	13
Use Case: Create User Account.....	13
Use Case: User View Personal Dashboard.....	16
Use Case: User Updates Personal Health Goals.....	18
Use Case: User Completes and Submits Form.....	19
Use Case: User Generates Personal Report (Graphical or Numerical).....	21
Use Case: User Reviews Personal Data Against Group Aggregate Values.....	22
Use Case: Register Researcher Account.....	24
Use Case: Researcher Appends Report.....	26
Use Case: Researcher Generates Anonymous Aggregated Report.....	28
Use Case: Researcher Exports Data to CSV.....	30
Use Case: Researcher Adds New Form.....	32
Use Case: Researcher Updates Existing Form.....	34
Use Case: Researcher Updates Database.....	36
Use Case: Researcher Creates Cohort (Group).....	39
Use Case: Healthcare Provider Registers Account.....	40
Use Case: Healthcare Provider Accesses Patients Data.....	41
Use Case: Healthcare Provider Views Graphical Report for Selected Group(s).....	44

Use Case: Healthcare Provider Views Graphical Report for Selected Participants.....	45
Use Case: Healthcare Provider Provides Personalized Feedback.....	47
Use Case: Register Admin Account.....	48
Use Case: Admin Views Audit Log.....	50
Use Case: Admin Deletes a Problematic User Report.....	52
Use Case: Admin Backs Up Database.....	54
Use Case: Admin Restores Database.....	55
Use Case: Admin Approve/Reject Forms.....	56
Use Case: Log In.....	58
Use Case: Update Account Information (All Roles).....	60
Stretch Goal Use Case: Reminders for Data Entry.....	61
Stretch Goal Use Case: System-Generated Suggestions (Automated Insight).....	63
Feature List.....	65
Database.....	65
Account Management.....	65
Login/Registration Page User Interface (All Roles).....	65
User Registration System (All Roles).....	65
Password Reset System.....	65
Design Account Settings Page User Interface (All Roles).....	66
Implement Account Settings Page User Interface (All Roles).....	66
Test Account Settings Page User Interface (All Roles).....	66
Account Settings System (All Roles).....	66
Authentication.....	66
User Authentication System (All Roles).....	66
User Permissions System.....	67
Form Submission.....	67
Design Form Fillout Page User Interface.....	67
Implement Form Fillout User Interface.....	67
Test Form Fillout User Interface.....	67
Form Submission System.....	67
Form Input Validation System.....	68
Form Creation/Modification.....	68
Design Form Creation/Modification User Interface.....	68
Implement Form Creation/Modification User Interface.....	68
Test Form Creation/Modification User Interface.....	68
Form Creation/Modification System.....	68
Design Form Validation System.....	69
Implement Form Validation System.....	69
Test Form Validation System.....	69
User Features.....	69
Design Personal Dashboard User Interface.....	69

Implement Personal Dashboard User Interface.....	69
Test Personal Dashboard User Interface.....	69
Design Health Goals User Interface.....	70
Implement Health Goals User Interface.....	70
Test Health Goals User Interface.....	70
Design Health Goals System.....	70
Design Personal Health Summary User Interface.....	70
Implement Personal Health Summary User Interface.....	71
Test Personal Health Summary User Interface.....	71
Personal Health Summary System.....	71
Design Personal Health Comparison User Interface.....	71
Implement Personal Health Comparison User Interface.....	71
Test Personal Health Comparison User Interface.....	72
Personal Health Comparison System.....	72
Researchers Features.....	72
Design Researcher Home Page User Interface.....	72
Implement Researcher Home Page User Interface.....	72
Test Researcher Home Page User Interface.....	72
Aggregated Report Generation System.....	73
Design Cohort Definition User Interface.....	73
Implement Cohort Definition User Interface.....	73
Test Cohort Definition User Interface.....	73
Cohort Definition System.....	73
Design Researcher Database Update Workflow.....	74
Design Researcher Database Update User Interface.....	74
Implement Researcher Database Update User Interface.....	74
Researcher Database Update System.....	74
Implement Aggregated Report Generation User Interface.....	74
Test Aggregated Report Generation User Interface.....	75
Data Filtering System.....	75
Aggregated Report Generation System.....	75
Report Export System.....	75
Implement Report Appending User Interface.....	75
Test Report Appending User Interface.....	76
Report Appending System.....	76
Healthcare Provider Features.....	76
Design Healthcare Provider Home Page User Interface.....	76
Implement Healthcare Provider Home Page User Interface.....	76
Test Healthcare Provider Home Page User Interface.....	76
Design Healthcare Provider Patient Data User Interface.....	76
Implement Healthcare Provider Patient Data User Interface.....	77

Test Healthcare Provider Patient Data User Interface.....	77
Healthcare Provider Patient Data Retrieval System.....	77
Design Healthcare Provider Feedback User Interface.....	77
Implement Healthcare Provider Feedback User Interface.....	77
Test Healthcare Provider Feedback User Interface.....	77
Design Healthcare Provider Feedback System.....	78
Admin Features.....	78
Design Admin Page User Interface.....	78
Implement Admin Page User Interface.....	78
Test Admin Page User Interface.....	78
Audit Log System.....	78
Design Audit Log User Interface.....	79
Implement Audit Log User Interface.....	79
Test Audit Log User Interface.....	79
Design Admin Report Review User Interface.....	79
Implement Admin Report Review User Interface.....	79
Test Admin Report Review User Interface.....	79
Admin Report Review System.....	80
Design Database Backup/Restoration User Interface.....	80
Implement Database Backup/Restoration User Interface.....	80
Test Database Backup/Restoration User Interface.....	80
Database Backup/Restoration System.....	80
Database Schema Modification System.....	81
Design Admin Form Review User Interface.....	81
Implement Admin Form Review User Interface.....	81
Implement Form Approval Workflow.....	81
Implement Form Rejection Workflow.....	81
Implement Audit Logging for Form Review Actions.....	82
Validate Admin Permissions for Form Review.....	82
Validate Rejection Reason Input.....	82
Prevent Review of Already Processed Forms.....	82
Test Admin Form Review User Interface.....	82
Test Admin Form Approval and Rejection Workflow.....	82
Notifications/Reminders.....	83
Reminder System.....	83
Design Notification Center User Interface.....	83
Implement Notification Center User Interface.....	83
Test Notification Center User Interface.....	83
Notification System.....	83
System Documentation.....	84
Stretch Goal Features.....	84

Data Entry Reminder System.....	84
Automated Insight System.....	84
Class Definitions.....	85
Activity Diagrams.....	97
User Registers Account.....	97
User Views Personal Dashboard.....	98
User Updates Personal Health Goals.....	99
User Completes/Submits Form.....	100
User Generates Personal Report.....	101
User Reviews Personal Data v/s Aggregate.....	102
Researcher Registers Account.....	103
Researcher Appends Report.....	104
Researcher Generates Anonymous Report.....	105
Researcher Exports Data.....	106
Researcher Adds New Form.....	107
Researcher Updates Existing Form.....	108
Researcher Updates DB.....	109
Researcher Creates Cohort.....	110
Provider Registers Account.....	111
Provider Accesses Patient Data.....	112
Provider View Report For Group(s).....	113
Provider Views Report for Participants.....	114
Provider Provides Personalized Feedback.....	115
Register Admin Account.....	116
Admin Views Audit Log.....	117
Admin Deletes Problematic Report.....	118
Admin Backs up DataBase.....	119
Admin Restores Database.....	120
Admin Approve/Reject From.....	121
Log In.....	122
Update Account Information.....	123
Stretch Goal: Reminders for Data Entry.....	124
Stretch Goal: Automated Insight.....	125
Sequence Diagrams.....	126
User Registers Account.....	126
User Views Personal Dashboard.....	127
User Updates Personal Health Goals.....	128
User Completes/Submits Form.....	129
User Generates Personal Report.....	130
User Reviews Personal Data v/s Aggregate.....	131
Researcher Registers Account.....	132

Researcher Appends Report.....	133
Researcher Generates Anonymous Report.....	134
Researcher Exports Data.....	135
Researcher Adds New Form.....	136
Researcher Updates Existing Form.....	137
Researcher Creates Cohort.....	138
Provider Registers Account.....	139
Provider Accesses Patient Data.....	140
Provider Views Report for Selected Group(s).....	141
Provider Views Report for Selected Participants.....	142
Provider Provides Personalized Feedback.....	143
Admin Registers Account.....	144
Admin Views Audit Log.....	145
Admin Deletes Problematic User Report.....	146
Admin Backs Up Database.....	147
Admin Restores Database.....	148
Admin Approve/Reject Forms.....	149
Log In.....	150
Update Account Information (All Roles).....	151
Stretch Goal: Reminders for Data Entry.....	152
Stretch Goal: System-Generated Suggestions (Automated Insight).....	153

Vision Statement

The goal of this project is to develop a secure, web based Health-Data Bank system that enables precariously housed individuals to record, manage and review their personal health information in a simple and accessible manner. The system will allow users to submit health data through online forms, set and track personal health goals, and view summaries and graphs that help them understand changes in their health over time.

The system will also support healthcare providers and researchers by allowing controlled access to health data. Researchers will be able to generate anonymized, aggregated reports to identify community-level health trends. Healthcare providers will be able to view user data and provide personalized feedback. Administrative features will ensure that access is properly managed and that sensitive data is protected through validation, auditing, and backup processes. Overall, the Health-Data Bank system is intended to make health data more accessible and useful for individuals, while also supporting research and care efforts in a secure and responsible way.

Key Technical & Operational Requirements

1. Technical Requirements

1.1 Application Architecture

- The system shall be implemented as a web-based application accessible through modern desktop and mobile web browsers.
- The architecture shall follow a client–server model, separating frontend presentation logic from backend business logic and data management.
- The backend shall expose functionality via secure RESTful APIs.

1.2 Frontend Technologies

- The user interface shall be implemented using a modern web framework (e.g., React, Vue, or similar).
- The interface shall support responsive design to accommodate mobile, tablet, and desktop devices.
- Data visualizations (graphs, charts, summaries) shall be rendered using a client-side visualization library.
- The frontend shall enforce basic input validation prior to submission.

1.3 Backend Technologies

- The backend shall be implemented using a server-side application framework (e.g., Node.js, Django, Spring Boot).
- Business logic shall enforce role-based permissions for Users, Researchers, Healthcare Providers, and Administrators.
- The system shall support asynchronous operations for long-running tasks such as report generation and CSV exports.

1.4 Database and Data Storage

- The system shall use a relational database management system (RDBMS) to store structured health data, user profiles, forms, and audit logs.
- Database schema shall enforce referential integrity and support historical data preservation.
- User-submitted health entries shall be append-only; deletion shall be restricted to administrative workflows.
- Backup and restore functionality shall be supported.

1.5 Security and Privacy

- All data in transit shall be protected using HTTPS (TLS encryption).
- Sensitive data shall be encrypted at rest where supported by the database.
- Authentication shall require email verification and two-factor authentication (2FA).
- Authorization shall be enforced via a role-based access control (RBAC) model.
- All sensitive actions (data access, exports, modifications, approvals) shall be recorded in an immutable audit log.
- Aggregated reports shall enforce minimum cohort size rules to prevent re-identification.

1.6 Reporting and Analytics

- The system shall support generation of:
 - Personal health summaries and trend graphs
 - Anonymous, aggregated community reports
- Researchers shall be able to export aggregated report data to CSV format.
- Exported data shall exclude personally identifiable information (PII).

1.7 Notifications and Communication

- The system shall support email-based notifications for account verification, approvals, and reminders.
- Optional in-system notifications shall be supported for reminders and feedback.

1.8 Development and Deployment Tools

- Source code shall be managed using a version control system (e.g., Git).
- The system shall support deployment to a cloud-based hosting environment.
- Environment configuration shall separate development, testing, and production settings.

2. Operational Requirements

2.1 Accessibility

- The system shall be usable by individuals with limited technical experience.
- Interfaces shall follow basic web accessibility guidelines (e.g., readable text, clear navigation, keyboard accessibility).
- Forms shall minimize required user input and avoid unnecessary complexity.

2.2 Availability and Reliability

- The system shall be available 24/7, excluding scheduled maintenance windows.
- System failures shall not result in loss of previously submitted health data.
- Form submissions shall be resilient to temporary connectivity issues where possible.

2.3 Performance

- Common user actions (login, dashboard view, form submission) shall complete within an acceptable response time under normal load.
- Report generation for large cohorts may execute asynchronously with user notification upon completion.

2.4 Data Governance and Compliance

- The system shall follow ethical principles for handling sensitive health data.
- Administrative approval shall be required for:
 - Researcher and healthcare provider accounts
 - Form creation and modification
- Audit logs shall be retained for the lifetime of the system.

2.5 Operational Constraints

- The system shall rely on third-party services for email delivery and 2FA.
- The project must be completed within academic term time constraints.
- Development resources are limited to a six-person student team.
- The system must be deployable and maintainable using commonly available tools (no proprietary infrastructure required).

2.6 Maintainability

- Code shall be modular and documented to support future extension.
- Database schema changes shall be managed via controlled administrative workflows.
- The system shall allow new forms, reports, and cohorts to be added without requiring redeployment.

System Acceptance Tests

- The project has a data registry system that includes a series of forms that can collect specific data designated by the client.
- The project has a relational database that is searchable and stores data from the various fields of the specific forms for later viewing or management.
- The project has a reporting facility that enables the client to access their own data, as well as access by the client to generate anonymous aggregated reports.
- The project provides the ability for the client to produce a graphical summary of the data.
- The project provides the ability to generate a report summarizing the data.
- The project provides the ability to modify the data set to reduce the effect of outliers.
- The project includes a dashboard for the individual from the designated cohort to review their personal data - against the group aggregate values.
- The project enforces access such that users only have access to data appropriate to their role (user, admin, researcher, healthcare provider)
- The project enforces a two-factor authentication system for all roles during both registration and login.
- The project has the ability to log all actions in an audit log, which includes timestamps, user IDs, a description of the action etc.
- The project is both accessible and responsive across all platforms (desktop and mobile) and follows basic accessibility guidelines in its layout
- Admins are able to back up, restore, and view audit logs.
- The project provides a suite of admin tools (manage users, manage permissions, manage system etc.)
- The project provides notifications to the user for verification, reminders etc. through a variety of channels (email, text, dashboard etc.)
- The project enforces strict admin approval for all higher-level accounts (healthcare providers, researchers)
- The project directs all user roles to distinct dashboards / landing pages
- The project enforces secure session management in order to protect user data

Use Cases

Use Case: Create User Account

Description:

A new prospective user will provide the system with the requested information (email, name, password, basic information about the user), which the system will then verify and check for duplicate emails within the database. If successful, the user will then be prompted to verify and set up a 2FA (two-factor authentication) method for their account, finalizing the account creation process. The system will then store the user's credentials under a new profile, allowing them to log in and access their health data bank profile to view / submit further health information.

Actors:

- User (Primary)
- System (Application)
- Email/2FA (Secondary)

Trigger:

- The user attempts to open a new account.

Preconditions:

- The user has access to an internet connection, email address, and a method of 2 factor authentication
- The system's servers are online and functional.

Post Conditions:

- The user will have a new account created and verified, which will then be stored within the database.
- The user will have set up two factor authentication.
- The user can now log in to their account to view or modify health data.

Normal Flow:

1. The user arrives at the account log in and creation page
2. The user selects the option to create a new account
3. The system returns the new account creation form, requesting the user's email, full name, and password, among other basic information
4. The user fills out the form and submits

5. The system validates that the form has been filled successfully, then sends a verification email to the provided email address
6. The user is prompted to finalize the validation process via the email they were just sent
7. The user logs in to their email account and completes the validation process
8. The account is validated and activated, and the user is prompted to set up 2FA on their new account
9. The user follows the instructions to set up their 2FA method of choice
10. The registration process is completed. The system notifies the user of their success and redirects to the log in page.

Alternate Flows:

5A1: The form is filled out incorrectly

- The system detects that some part of the form was filled out incorrectly (information does not match expectations, required fields left empty, password too weak etc.)
- The system informs the user of what went wrong
- The user returns to step 3

5A2: The System experiences an error during form validation

- After form submission, the system experiences some kind of validation error (connection, server stress etc.)
- The relevant error is logged
- The system informs the user that an error occurred and prompts them to re-try submission
- The user returns to step 3

6A1: The provided email is already found within the system database

- The system informs the user that the email they provided is already in use
- The user is prompted to either attempt to log in with this email, or to reset the password associated with it if they believe it to be theirs
- The use case ends

7A1: The user fails to complete the email verification process

- The verification link sent to the user is valid only for a set period of time (X hours)
- If not clicked in time, either via user or system error, the account will not be activated and the user must request a new link
- The user returns to step 7

8A1: There is a failure in the 2fa setup process

- The user fails to set up 2fa via some means
- The system informs the user that they must finish 2fa setup to continue
- The system presents the user with the option to retry 2fa setup
- The user returns to step 8

Use Case: User View Personal Dashboard

Description:

After logging into the system, the user can access their personal dashboard to view stored health information and recent activity. The dashboard provides a summary of past entries, trends, and health statistics through graphs or tables. This allows the user to review their progress over time and identify patterns related to their wellbeing.

Actors:

- User (Primary)
- System

Trigger:

- The user indicates that they want to view their personal data.

Preconditions:

- The user is authenticated.
- The system has access to the user's stored entries.

Post-conditions:

- The user will see their latest entries, trends and quick actions.
- A view event will be recorded in the audit log.

Normal Flow:

1. The user will indicate that they want to see their personal dashboard.
2. The system will retrieve the user's most recent entries and summaries.
3. The system will display the dashboard, including trends and shortcuts (eg. "Add Entry").
4. The user will view their information and navigate through sections.
5. The user exits the dashboard.

Alternate Flows:

2A1: The user has no entries yet.

- The system will display a message such as "No Data Yet" and a link to submit a new form.
- The use case ends.

2A2: Data Retrieval Failure (for example, a temporary network or database error):

- The system displays an error message and a “Retry” option.
- The relevant error is logged.
- If the retry succeeds, the use case returns to Step 2; otherwise, it ends.

3A1: Graph/Visualization Error

- The system displays available textual data and a message such as “Graph temporarily unavailable.”
- The use case continues at Step 5.

Use Case: User Updates Personal Health Goals

Description:

An authenticated user updates personal health goals in the system. The system tracks progress against these goals over time.

Actors:

- User (Primary)
- System

Trigger:

- The user chooses the option to update a personal health goal.

Preconditions:

- The user is authenticated.
- The system has access to the user's stored health data for tracking.

Post-conditions:

- The user will see that their latest entries and goals are stored.
- Progress is tracked automatically

Normal Flow:

1. The user will select to open the Goals page.
2. The system will retrieve the user's most recent entries and summaries.
3. The system will display the options that the user can edit.
4. The user will add/edit a goal and select "Save".
5. The system will validate values.
6. The system will store goals, and record audit log entry.
7. The system will update dashboard widgets.
8. The user exits the dashboard.

Alternate Flow:

5A1: The user entries are not realistic.

- The system will display an error.
- The user returns to step 2.

Use Case: User Completes and Submits Form

Description:

After the user logs into their account, they can fill out a health form to record their daily or periodic health information, such as substance use, physical activity or symptoms. The system will validate and store this information as part of the user's health record, ensuring that previous entries are not deleted but rather appended. The user will receive confirmation that their data was successfully saved, and their dashboard will update to reflect the new entry.

Actors:

- User (Primary)
- System

Trigger:

- The user indicates that they want to add a new health entry.

Preconditions:

- The user is authenticated
- At least one active form template exists in the system

Post-conditions:

- A new health entry will be appended to the user's record.
- An audit log will record the submission.

Normal Flow:

1. The user will indicate that they want to fill out a form.
2. The system will present the active form and required fields.
3. The user will complete required fields and indicate that the form is ready to submit.
4. The system will validate inputs.
5. The system will save this new entry to the user's record.
6. The system will confirm successful submission.
7. The system will update the dashboard.

Alternate Flows:

2A1: No active form available

- The system will display a message that no active health forms are available at this time.

- The use case will end.

4A1: The system detects missing or invalid inputs.

- The system will highlight invalid fields and explain the issue.
- The user will correct the inputs and resubmit form.
- The use case continues at step 5.

4A2: The system experiences an error during form validation

- After form submission, the system experiences some kind of validation error (connection, server stress etc.)
- The relevant error is logged.
- The system informs the user that an error occurred and prompts them to re-try submission.
- The user returns to step 3.

6A1: There is a submission timeout

- The system will show a progress message and provide a cancel option.
- If cancelled, the system stores the data as a draft for later submission.
- The use case resumes at Step 5 once the connection stabilizes.

Use Case: User Generates Personal Report (Graphical or Numerical)

Description:

The User wishes to view a graphical summary of their personal data, causing the System to retrieve the relevant personal data from the database, summarise the data retrieved, and generate a graphical or numerical representation of the summarized data

Actors:

- User (Primary)
- System

Trigger:

- The user wishes to view a graphical/numerical summary of their data.

Preconditions:

- The user is authenticated
- The user is on the dashboard page

Post-conditions:

- The user is shown a graphical summary/numerical of their data

Normal Flow:

1. The user navigates personal data section of dashboard
2. The user selects graphical or numerical summary option
3. The system retrieves relevant personal data from the database
4. The system summarizes data retrieved
5. The system generates graphical or numerical representation of summary
6. The system displays graphical/numerical summary to user
7. The user closes graphical/numerical summary and is returned to dashboard

Alternate Flows:

2A1: System does not find any data belonging to User.

- System will inform User they do not have any data to display, suggest entering some forms before trying again
- System aborts summary creation
- User is returned to step 1

Use Case: User Reviews Personal Data Against Group Aggregate Values

Description:

The user wishes to compare their personal data with that of the aggregate values of their cohort, causing the system to find other users of a similar demographic, aggregate the relevant data the user wishes to compare with, and display the users data against the aggregate data in a tabular format

Actors:

- User (Primary)
- System

Trigger:

- The user wishes to view their personal data in comparison to anonymized aggregate values of members of their cohort

Preconditions:

- The user is authenticated
- The user is on the dashboard page

Post-conditions:

- The system displays relevant personal/aggregate comparisons to the user

Normal Flow:

1. The user navigates to personal data section of dashboard
2. The user selects compare to others option
3. The user is prompted to select which metrics they would like to see
4. The system retrieves relevant fields from the database
5. The system searches for users of a similar demographic as the user
6. The system retrieves and aggregates relevant fields from the database
7. The system transforms data into a table format for easier reading
8. The system displays table to the user
9. The user closes table and is returned to the dashboard

Alternate Flows:4A1: System is unable to retrieve data necessary for selected metrics

- System informs user that comparison cannot be performed using currently selected metrics, suggests user to select different metrics to compare
- User is given option to either return to metric selection, or cancel and return to dashboard
- If User opts to return to metric selection, use case continues from step 3
- Otherwise, User is returned to dashboard

5A1: System is unable to find sufficient amount of other users of the same demographic

- The user is prompted to choose if they would like to continue with comparison against a broader range of other users, and is warned that these results may be less informative.
- If User choose to continue on, the use case continues from step 5, but with broadened search parameters
- Otherwise, System aborts comparison, and User is returned to dashboard, step 1

Use Case: Register Researcher Account

Description:

After a researcher decides to use the Health Data Bank for academic or healthcare studies, they can opt to register a researcher account. The researcher provides institutional details, such as name, organization, and email. Once the account is created and verified, the researcher can access features to generate anonymized reports, manage forms, and export aggregated data.

Actors:

- Researcher (Primary)
- System

Trigger:

- The researcher indicates that they want to create a new researcher account to access reporting and data analysis tools.

Preconditions:

- The researcher has access to an internet connection, email address, and a method of 2 factor authentication
- The system is online and registration is available.

Post-conditions:

- A researcher account will be created in the system with research-level permission.
- The researcher will be able to log in and access research-related functions such as generating reports and exporting data.

Normal Flow:

1. The researcher will open the "Register" page.
2. The system will display the registration form and required fields (e.g name, email)
3. The researcher will enter the required information and submit the form.
4. The system will validate that the form has been filled successfully, then send a verification email to the provided email address.
5. The researcher will be prompted to finalize the validation process via the email.
6. The researcher will log in to their email account and complete the validation process.
7. The account will be validated and activated, and the researcher will be prompted to set up 2FA on their new account.
8. The researcher will follow the instructions to set up their 2FA method of choice.

9. The system will create a Researcher account and will assign appropriate permissions.
10. The system will confirm successful registration.
11. The researcher will log in to access reporting tools to generate reports or manage forms.

Alternate Flows:**3A2: The email is already registered**

- The submitted email already exists in the system.
- The system informs the researcher and provides options to Log In or Reset Password.
- If the researcher chooses to reset the password, a reset email is sent, and this use case ends.

4A1: The system detects invalid or missing information.

- The system will highlight the missing or invalid fields.
- The researcher will correct the input and resubmit form.
- The use case continues to step 3.

5A1: The verification email fails or expires

- The verification link sent to the researcher is valid only for a set period of time (X hours).
- If not clicked in time, either via researcher or system error, the account will not be activated and the researcher must request a new link.
- The user returns to step 6.

Use Case: Researcher Appends Report

Description:

A verified researcher wants to append new findings or observations to already existing community health reports within the Health Data Bank System. The researcher first chooses a specific report related to a health trend or dataset and then provides any additional data, notes etc. as an update to the record. After validating the researcher's credentials, the system goes on to check that the report actually exists, and logs all appended content along with timestamps and the researcher's ID. These updates are then presented as the new "main" version of the report in question, while previous versions are stored for future accountability and review.

Actors:

- Researcher
- System

Trigger:

- The researcher selects the option to append new information to an existing report.

Preconditions:

- The researcher is logged in with valid credentials.
- The report to be updated already exists in the system.
- The system is online and accessible.

Post-conditions:

- The researcher's appended data is stored and linked to the corresponding report.
- The report's version history is updated to reflect the changes.
- Other researchers can now view the updated report with appended notes.

Normal Flow:

1. The researcher navigates to the Reports section of the system.
2. The researcher searches for or selects an existing report.
3. The system retrieves and displays the selected report.
4. The researcher chooses the option to append new data or observations.
5. The researcher enters new information into the append form.
6. The system validates the researcher's input and permissions.
7. The system stores the appended content, linking it to the report.
8. The system logs the researcher's ID and timestamp for the modification.
9. The system confirms that the report has been successfully updated.

Alternate Flows:5A1: Invalid or incomplete input

- The system detects that required fields in the append form are missing or formatted incorrectly.
- The system prompts the researcher to correct the issues.
- The researcher returns to step 5.

6A1: Researcher lacks editing permissions

- The system detects that the researcher does not have permission to edit this report.
- The system displays an error message and logs the failed attempt.
- The researcher returns to step 2 or exits the process.

7A1: System error during data storage

- The system encounters an issue saving the appended content.
- The error is logged automatically.
- The system notifies the researcher and allows them to retry submission later. The use case ends.

Use Case: Researcher Generates Anonymous Aggregated Report

Description:

A validated researcher account accesses the system in order to generate an aggregated report on community health trends. The researcher chooses from a variety of specific health metrics, date ranges, and demographic filters, which the system will then use as filters in order to compile a set of anonymized data from multiple users (calculating statistical aggregates averages, percentages, trends), and generate a comprehensive report that can either be viewed online or downloaded for further analysis.

Actors:

- Researcher (Primary)
- System

Trigger:

- The researcher selects the option to generate a new aggregated health report.

Preconditions:

- The researcher is authenticated with valid researcher credentials.
- The database contains sufficient user health entries for meaningful aggregation.
- The system's reporting functions are operational.

Post-conditions:

- An anonymized aggregated report is generated and stored in the system.
- The report generation event is logged in the audit log.
- The researcher can view, download, or share the report with authorized users.

Normal Flow:

1. The researcher logs in to their validated researcher account
2. The researcher navigates to the "Generate Report" section
3. The system presents report configuration options (metrics, date ranges, demographic filters)
4. The researcher selects desired parameters and submits the request
5. The system validates the researcher's permissions
6. The system validates that the parameters are appropriate and sufficient data exists
7. The system retrieves relevant anonymized data from the database
8. The system performs statistical aggregation and analysis
9. The system generates the report with visualizations and summary statistics

10. The system displays the report to the researcher
11. The event is logged in the audit log
12. The researcher views the report and optionally downloads it

Alternate Flows:

6A1: Insufficient data for meaningful aggregation

- The system detects that the selected filters result in too few data points (e.g., less than 10 users)
- The system informs the researcher that aggregation cannot proceed to protect user anonymity
- The researcher is prompted to broaden their filter criteria
- The researcher returns to step 4

6A2: Invalid parameter combination

- The system detects that the selected parameters are incompatible or invalid
- The system highlights the problematic selections and explains the issue
- The researcher corrects the parameters and resubmits
- The researcher returns to step 4

8A1: System error during data aggregation

- The system encounters an error while processing the data (database timeout, calculation error, etc).
- The error is logged automatically
- The system notifies the researcher and suggests trying again later
- The researcher may return to step 4 or exit the process

Use Case: Researcher Exports Data to CSV

Description:

The researcher accesses the database and navigates to the option to export data. The researcher is presented with a selection of filtering options (by date, by certain demographics, by certain health criteria etc.), and submits. The system uses these filters to compile a set of anonymous data, and produces a CSV download.

Actors:

- Researcher (Primary)
- System

Trigger:

- The researcher navigates to the option to generate a CSV report.

Preconditions:

- The database is online, accessible, and contains entries to export
- The researcher has the relevant credentials and has been validated by the system administrators

Post Conditions:

- A CSV file of the requested anonymous information will be generated for the researcher to download
- The event is logged in the audit log

Normal Flow:

1. The researcher logs in to their validated researcher account
2. The researcher navigates to the data export page
3. The researcher applies any relevant filters to the data and submits a request
4. The system validates the account requesting the data
5. The system validates that the parameters are valid
6. The system attempts to compile the relevant data
7. The system generates the CSV file and presents it to the researcher for download
8. The export is recorded in the audit log
9. The researcher downloads the CSV

Alternate Flows:1A1: The researcher's account has not been validated for data export / System fails to verify

- The system fails to verify that the researcher has been validated.
- The researcher receives an error message informing them that they lack the proper credentials and to contact a system administrator if they believe there has been a mistake.
- The researcher returns to step 1.

6A1: The system fails to export the requested data

- The system returns an error message requesting the user to try again shortly.
- The event is logged in the audit log.
- The researcher returns to step 3.

Use Case: Researcher Adds New Form

Description:

A validated researcher account creates a new health data collection form to gather specific information from users. The researcher designs the form by specifying field types, labels, validation rules, and whether fields are required or optional. Once submitted, the form is reviewed by system administrators for approval before being made available to users. This allows the research team to adapt data collection to emerging health trends or research needs.

Actors:

- Researcher (Primary)
- System
- Administrator (validates new form before deployment)

Trigger:

- The researcher selects the option to create a new health data form.

Preconditions:

- The researcher is authenticated with valid credentials
- The researcher has permission to create new forms
- The form builder interface is functional

Post-conditions:

- A new form template is created and stored in the system
- The form is marked as "pending approval" until reviewed by an administrator
- Once approved, the form becomes available for users to fill out
- The form creation event is logged in the audit log

Normal Flow:

1. The researcher logs in to their validated researcher account
2. The researcher navigates to the "Form Management" section
3. The researcher selects the option to "Create New Form"
4. The system presents the form builder interface
5. The researcher enters form details (title, description, purpose)
6. The researcher adds fields to the form (text inputs, dropdowns, checkboxes, date pickers, etc.)
7. For each field, the researcher specifies:
 - Field label and help text
 - Field type and format

- Whether the field is required or optional
- Validation rules (e.g., numeric ranges, date constraints)
- 8. The researcher previews the form to ensure it displays correctly
- 9. The researcher submits the form for approval
- 10. The system validates the form structure and field configurations
- 11. The system saves the form with "pending approval" status
- 12. The system notifies administrators that a new form requires review
- 13. The system confirms successful submission to the researcher
- 14. The event is logged in the audit log

Alternate Flows:

10A1: Invalid form configuration

- The system detects errors in the form structure (missing required metadata, invalid field types, conflicting validation rules)
- The system highlights the problematic elements and explains the issues
- The researcher corrects the errors and resubmits
- The researcher returns to step 9

10A2: Duplicate form detected

- The system detects that a very similar form already exists in the system
- The system alerts the researcher and displays the existing form
- The researcher decides whether to proceed with creating the new form or modify the existing one
- If proceeding, the use case continues from step 11
- If modifying existing, the researcher is redirected to Use Case: researcher updates existing forms

11A1: System error during form save

- The system encounters an error while saving the form (database connection issue, storage problem, etc).
- The error is logged automatically
- The system notifies the researcher and preserves their work if possible
- The researcher may retry submission from step 9

14A1: Form approval denied by administrator

- Administrator reviews the form and denies approval (inappropriate questions, privacy concerns, technical issues)
- The system notifies the researcher with an explanation for denial
- The researcher may revise the form and resubmit
- The researcher returns to step 6

Use Case: Researcher Updates Existing Form

Description:

A validated researcher modifies an existing health data collection form in order to improve clarity, add new fields, remove outdated fields, or just adjust validation rules. The researcher selects which form they'd like to edit, makes any necessary changes, and submits the updated version. Minor changes (like correcting typos) may be implemented immediately, while major changes (adding/removing fields) require administrator approval to ensure data consistency and that existing user data remains valid.

Actors:

- Researcher (Primary)
- System
- Administrator (validates significant changes before deployment)

Trigger:

- The researcher selects the option to edit an existing form.

Preconditions:

- The researcher is authenticated with valid credentials
- The researcher has permissions to modify forms
- The form to be modified exists in the system
- The form builder interface is functional

Post-conditions:

- The form template is updated with the changes
- If changes are major, the form is marked as "pending approval" until reviewed
- The system maintains version history of the form for audit purposes
- Existing user data remains associated with the appropriate form version
- The form modification event is logged in the audit log

Normal Flow:

1. The researcher logs in to their validated researcher account
2. The researcher navigates to the "Form Management" section
3. The system displays a list of existing forms
4. The researcher selects a form to edit
5. The system retrieves and displays the form in the form builder interface
6. The system shows the form's current structure and settings

7. The researcher makes desired changes:
 - Edits field labels or help text
 - Modifies validation rules
 - Adds new fields
 - Removes or disables obsolete fields
 - Updates form description or instructions
8. The researcher previews the updated form
9. The researcher submits the changes
10. The system analyzes the scope of changes to determine if approval is needed
11. If minor changes: System validates and immediately updates the form
12. If major changes: System saves the form with "pending approval" status and notifies administrators
13. The system creates a new version entry in the form's history
14. The system confirms the update to the researcher
15. The event is logged in the audit log

Alternate Flows:

7A1: Researcher attempts to delete field with existing data

- The researcher tries to remove a field that already has user data associated with it
- The system warns that deleting this field could affect data integrity and analytics
- The researcher is given options:
 - Mark the field as "deprecated" (hidden from new submissions but preserved in existing data)
 - Proceed with deletion (with confirmation that historical data will be preserved separately)
 - Cancel the deletion
- Based on the choice, the use case continues from step 9 or returns to step 7

10A1: Conflicting changes detected

- The system detects that another researcher has modified the same form since it was loaded
- The system displays both sets of changes for comparison
- The researcher must review conflicts and decide how to proceed:
 - Merge changes manually
 - Overwrite with their version
 - Cancel and reload the latest version
- The researcher returns to step 7

11A1: Validation errors in updated form

- The system detects errors in the form structure (invalid field configurations, broken validation rules, missing required metadata)
- The system highlights the problematic elements
- The researcher corrects the errors and resubmits
- The researcher returns to step 9

12A1: Form update denied by administrator

- Administrator reviews major changes and denies approval (concerns about data consistency, inappropriate modifications, technical issues)
- The system notifies the researcher with an explanation
- The previous version of the form remains active
- The researcher may revise and resubmit changes
- The researcher returns to step 7

13A1: System error during form update

- The system encounters an error while saving the updated form
- The error is logged automatically
- The system notifies the researcher and attempts to preserve their work
- The researcher returns to step 9

Use Case: Researcher Updates Database

Description:

A validated researcher updates the database by maintaining research-controlled data structures and reference data, such as activating/deactivating form templates, updating controlled vocabularies (e.g., health categories, option lists), refreshing cohort definitions, and triggering system-approved data refresh or recalculation processes. Direct modification of individual user submissions is not permitted; all updates are performed through controlled system interfaces to preserve data integrity and privacy.

Actors:

- Researcher (Primary)
- System
- Administrator (Secondary - oversight via audit log and approvals)

Trigger:

- The researcher selects an option such as “Manage Database” from the research tools menu.

Preconditions:

- The researcher is authenticated and has been approved for researcher-level access.
- The database is online and operational.
- The researcher has permission to perform database-maintenance actions.

Post-conditions:

- Approved database updates are applied successfully.
- Any affected reports, aggregates, or cohorts are refreshed as required.
- The update action is recorded in the audit log with timestamp and researcher ID.

Normal Flow:

1. The researcher logs in to their validated researcher account.
2. The researcher navigates to the Database Management or System Data Management section.
3. The system displays available database update actions, such as:
 - Activating or deactivating form templates
 - Updating controlled vocabularies or option lists
 - Refreshing cohort membership calculations
 - Recomputing aggregated statistics
4. The researcher selects a permitted update action.

5. The system displays a confirmation screen describing the scope and impact of the update.
6. The researcher confirms the update.
7. The system performs validation checks (permissions, data integrity, dependency checks).
8. The system applies the update to the database.
9. The system refreshes any dependent data (reports, aggregates, dashboards) if required.
10. The system logs the update event in the audit log.
11. The system confirms successful completion to the researcher.

Alternate Flows:

4A1: Researcher selects an action requiring administrator approval

- The system marks the update as *pending approval*.
- The administrator is notified.
- The use case ends until approval is granted.

6A1: Researcher cancels the update

- No changes are made.
- The use case ends.

7A1: Validation fails (data integrity or permission issue)

- The system blocks the update.
- The system displays an error message explaining the issue.
- The event is logged.
- The researcher returns to Step 3.

8A1: System error during database update

- The system encounters an error while applying the update.
- The system rolls back any partial changes.
- The error is logged automatically.
- The researcher is notified and may retry later.

Use Case: Researcher Creates Cohort (Group)

Description:

Researcher defines how a group is formed using filters. This will enable group-based reporting.

Actors:

- Researcher (Primary)
- System

Trigger:

- The researcher selects create group.

Pre-conditions:

- The researcher is authenticated and approved.

Post-conditions:

- Cohort definition is saved and available for reporting.

Normal Flow:

1. The researcher enters group name and purpose.
2. The system displays allowed filter attributes (non-identifying demographic/health flags).
3. The researcher selects filter rules (e.g., age range, condition flags).
4. The system estimates cohort membership size.
5. The system confirms the cohort meets minimum anonymity size and saves it.
6. The system logs action.

Alternate Flows:

5A1: Cohort too small

- System rejects and suggests broader filters
- Researcher returns to step 2.

Use Case: Healthcare Provider Registers Account

Description:

A healthcare provider creates a new verified account within the Health Data-Bank system by submitting registration details such as their name, email, organization, license, and password after selecting the option to create a new account. The system verifies that the provider is authentic by cross-checking their credentials with a trusted medical registry or administrative authority. Once verified, the system creates an account with specific access and privileges that allow the provider to view and analyze anonymized community health data, as well as patient specific information upon referral. The provider must also complete two-factor authentication (2FA) setup before their account becomes active.

Actors:

- Healthcare Provider (Primary)
- System

Trigger:

- The healthcare provider selects the option to create a new account.

Preconditions:

- The provider has access to a verified organizational email.
- The provider's license is valid and verifiable through an external or internal registry.
- The system's verification and email services are working.

Postconditions:

- A new healthcare provider account is created, verified, and stored within the system.
- The provider has successfully set up 2FA.
- The provider can now log in to their account and access authorized data or community reports.

Normal Flow:

1. The provider navigates to the system's login and account creation page.
2. The provider selects the option to create a healthcare provider account.
3. The system presents the registration form requesting name, organization, email, license/certification ID, and password.
4. The provider fills out the required fields and submits the form.
5. The system validates that the form is complete and properly formatted.

6. The system cross-checks the provided license or certification ID with the appropriate registry.
7. Upon successful verification, the system sends a confirmation email to the provider.
8. The provider accesses their email and completes the verification link.
9. The system activates the provider's account and prompts for 2FA setup.
10. The provider completes the 2FA process.
11. The system confirms successful account creation and redirects the provider to the login page.

Alternate Flows:

4A1: The form contains invalid or incomplete information

- The system detects missing or improperly formatted data.
- The system informs the provider of the errors.
- The provider returns to step 3 to correct the submission.

6A1: License or certification verification fails

- The system cannot validate the provided license or ID with the registry.
- The system informs the provider that verification failed and provides contact options for manual review.
- The provider returns to step 5 and may resubmit or cancel registration.

7A1: Email already associated with an existing account

- The system detects that the email is already registered.
- The provider is prompted to either log in or reset their password instead of creating a new account.
- The provider is redirected to use case: log in

8A1: Email verification not completed within time limit

- The verification link expires after a set time.
- The system notifies the provider and offers to resend the verification email.
- The provider returns to step 8 after receiving the new link.

9A1: Failure during 2FA setup

- The provider experiences an issue while configuring 2FA.
- The system logs the error and instructs the provider to retry setup.
- The provider must complete 2FA to activate the account successfully.
- The provider returns to step 4

Use Case: Healthcare Provider Accesses Patients Data

Description:

An authenticated healthcare provider accesses the system, and logs in to their healthcare provider account. After verifying their credentials, the healthcare provider is presented with an interface to navigate to the data relevant to a selected patient (medical history, prescriptions/medication, basic information about the patient etc.). Each instance of access to a patient's data is added to the audit log.

Actors:

- Healthcare provider (Primary)
- System
- Administrator

Trigger:

- A healthcare provider navigates to the interface to access a patient's medical data.

Preconditions:

- The healthcare provider has registered an account and is validated by the system administrator
- The patient data they are requesting exists within the database

Post-conditions:

- The healthcare provider is granted access to the relevant patient's data
- The event is recorded in the audit log

Normal Flow:

1. The healthcare provider accesses the system and logs in to their validated account
2. The healthcare provider attempts to access the page to search for a specific patient's records
3. The system validates the user's access
4. The healthcare provider is presented with a form to search for a specific patient
5. The healthcare provider submits the relevant information to the form to find the patient records they are looking for
6. The system checks if the patient is in the system
7. The system returns with the patient's data
8. The event is logged
9. The healthcare provider now has access to the data they need

Alternate Flows:1A1: The healthcare provider lacks proper authentication to access patient records

- The system denies access to the healthcare provider
- The system returns an error message explaining that they lack sufficient authorization to access this data
- The healthcare provider is directed to contact an administrator for further questions
- The use case ends

6A1: The patient is not found within the system

- The system returns with an error that the patient could not be found
- The healthcare provider returns to step 4

Use Case: Healthcare Provider Views Graphical Report for Selected Group(s)

Description:

Healthcare Provider views graphs for selected groups, including comparisons between two groups (aggregate views).

Actors:

- Healthcare Provider (Primary)
- System

Trigger:

- Provider selects "Group Analytics".

Pre-conditions:

- Provider authenticated and authorized for group analytics.

Post-conditions:

- Graphical report displayed; export optional.

Normal Flow:

1. The provider selects group A (and optional group B) plus metrics and date range.
2. The system validates minimum group size and anonymity constraints.
3. The system generates graphs and summary tables.
4. The provider views report and filters charts.
5. The system logs report access.

Alternate Flows:

2A1: Group too small

- System blocks and requests broader group
- Provider returns to step 1.

3A1: System failure to generate graphs

- The system fails to provide tables and charts.
- The provider needs to reselect the group and returns to step 1.

Use Case: Healthcare Provider Views Graphical Report for Selected Participants

Description:

A validated healthcare provider generates and views a graphical report (charts/tables) for a selected set of participants (specific users) to support care planning and monitoring. The report may include trends over time and comparisons across the selected participants.

Actors:

- Healthcare Provider (Primary)
- System

Trigger:

- Healthcare provider selects to view graphical report of selected participants.

Preconditions:

- Healthcare provider is authenticated and approved.
- Selected participants exist and have submitted at least one form.

Post-conditions:

- Graphical report is displayed.
- Access/report-generation event is recorded in the audit log.

Normal Flow:

1. Provider logs in and opens the analytics/reporting page.
2. The system displays a list/search of authorized participants.
3. Provider selects multiple participants.
4. Provider selects metrics (e.g., sleep, substance use frequency, symptoms) and a date range.
5. The system validates access permissions for all selected participants.
6. The system retrieves relevant data and computes required summaries.
7. The system generates graphical outputs (charts/tables).
8. The system displays the report to the provider.
9. The system logs the report access/generation event.

Alternate Flows:6A1: No data for selected metrics/date range

- The system displays “No data available” and suggests changing filters.
- Provider returns to Step 4.

7A1: Visualization error

- The system displays tabular summaries and a “graph unavailable” notice.
- Use case continues at Step 8.

Use Case: Healthcare Provider Provides Personalized Feedback

Description:

A healthcare provider can send personalized feedback to a user based on their submitted health data.

Actors:

- User (Secondary)
- System
- Healthcare provider (Primary)

Trigger:

- The healthcare provider chooses the user profile and selects the option to send feedback

Preconditions:

- The user and provider are authenticated.
- The provider has access to the user's data.

Post-conditions:

- The system informs the user that they have received feedback.
- Feedback is stored.
- The event is logged in the audit log

Normal Flow:

1. Provider selects a user profile and selects "Add Feedback"
2. The provider writes notes and recommended actions based on retrieved data.
3. The provider submits feedback.
4. The system saves feedback and notifies the user.
5. The system logs feedback creation..

Alternate Flows:

3A1: System error in sending feedback.

- The system will indicate to the provider that error has occurred.
- The provider returns to step 2.

Use Case: Register Admin Account

Description:

An administrator account is created to grant system-level access for managing governance, security, and maintenance functions of the Health-Data Bank System. Admin account registration establishes a trusted administrative identity with elevated privileges, enabling actions such as approving privileged accounts, viewing audit logs, moderating reports, and performing database backup and restore operations. For security reasons, admin accounts are created through a controlled registration process (e.g., invitation or approval-based setup) and require email verification and two-factor authentication before activation.

Actors:

- Administrator (Primary)
- System

Trigger:

- The administrator indicates that they want to create a new account.

Preconditions:

- Admin has access to an internet connection, email address, and a method of 2 factor authentication
- The system is online and registration is available.

Post-conditions:

- An admin account will be created in the system with admin-level permission.
- The admin will be able to log in and access admin-related functions such as viewing audit logs, deleting reports.

Normal Flow:

1. The administrator will open the "Register" page.
2. The system will display the registration form and required fields (e.g name, email)
3. The admin will enter the required information and submit the form.
4. The system will validate that the form has been filled successfully, then send a verification email to the provided email address.
5. The admin will be prompted to finalize the validation process via the email.
6. The admin will log in to their email account and complete the validation process.
7. The account will be validated and activated, and the admin will be prompted to set up 2FA on their new account.

8. The administrator will follow the instructions to set up their 2FA method of choice.
9. The system will create an Admin account and will assign appropriate permissions.
10. The system will confirm successful registration.
11. The admin will login to access the healthbank.

Alternate Flows:

3A2: The email is already registered

- The submitted email already exists in the system.
- The system informs the admin and provides options to Log In or Reset Password.
- If the administrator chooses to reset the password, a reset email is sent, and this use case ends.
- If admin wishes to Log In, admin is redirected to use case: Log In

4A1: The system detects invalid or missing information.

- The system will highlight the missing or invalid fields.
- The admin will correct the input and resubmit form.
- The use case continues to step 3.

5A1: The verification email fails or expires

- The verification link sent to the administrator is valid only for a set period of time (X hours).
- If not clicked in time, either via admin or system error, the account will not be activated and the admin must request a new link.
- The admin returns to step 6.

Use Case: Admin Views Audit Log

Description:

The admin accesses the audit log to review all activities related to the database system and health data. The log displays a chronological record of actions, including form submissions, data access by healthcare providers, profile modifications, login attempts, and data exports. This lets the admin monitor who has accessed what information within the database and when, allowing them to detect any unauthorized access to sensitive health data.

Actors:

- Admin (Primary)
- System

Trigger:

- The admin selects the option to view the audit log.

Preconditions:

- The admin is authenticated with a role that permits audit-log access.
- The audit log system is online and operational.
- The system has recorded at least one event to populate the audit log with.

Post-conditions:

- The admin has viewed the audit log history.
- Any export action is recorded as an audit event.
- No audit record is modified or deleted as a result of viewing.

Normal Flow:

1. The admin logs in to their administrator account.
2. The admin indicates that they want to open the audit log.
3. The system presents the audit-log interface with default filters (e.g., most recent 24-72 hours).
4. The admin further specifies any more specific filters (e.g., date range, actor, action type, resource) and requests the log.
5. The system validates the filters and queries the audit store.
6. The system displays matching events with relevant fields (timestamp, actor).
7. The admin pages through results or adjusts sorting (e.g., newest first).
8. The admin indicates that the filtered results should be exported (e.g., CSV).
9. The system generates the export and initiates the download.
10. The system records the export event in the audit log.

11. The admin exits the audit-log view.

Alternate Flows:

4A1: Invalid filter parameters

- The admin supplies an invalid date range or unsupported action type.
- The system highlights the invalid input and prompts correction.
- The use case returns to Step 4 and continues.

5A1: Large number of log entries

- The selected time window or filter would return an excessive result set.
- The system prompts the admin to narrow filters or enables server-side pagination.
- The use case returns to Step 3 or proceeds with paginated results to Step 5.

6A1: No matching events

- The query returns zero results.
- The system displays “No events found” and suggests widening filters.
- The use case continues at Step 3 if the admin adjusts filters, or use case ends.

7A1: Export temporarily unavailable

- The export service is unavailable.
- The system displays an error message that export is not available.
- The admin may retry export (return to Step 7) or continue viewing (Step 6).

10A1: System error during log retrieval

- The system encounters an error while fetching audit log entries (database timeout, connection issue, etc).
- The error is logged automatically
- The system notifies the admin and suggests trying again later
- The admin may retry immediately (step 8) or exit

Use Case: Admin Deletes a Problematic User Report

Description:

A system administrator wants to identify and delete user-submitted health records or form submissions that have been flagged as problematic, such as corrupted data, inappropriate content, accidental submissions, or invalid entries. The administrator accesses the flagged submissions queue, reviews the selected submission, and determines whether deletion is necessary. If confirmed, the system removes the submission from active storage, preserves relevant metadata for audit purposes, updates related indexes and analytics, and confirms the deletion to the administrator.

Actors:

- Administrator (Primary)
- System

Trigger:

- An administrator selects the option to delete a flagged or problematic report.

Preconditions:

- The administrator is logged in with verified credentials.
- The report to be deleted is accessible and has been flagged for review.
- The system's deletion and archiving functions are operational.

Post-conditions:

- The problematic report is deleted from the main database.
- Audit logs record the deletion event, including who performed it and when.
- The report is removed from all visible reports and search results.

Normal Flow:

1. The administrator navigates to the flagged reports dashboard.
2. The system displays a list of flagged user submissions marked for potential deletion.
3. The administrator selects a submission to review.
4. The system retrieves and displays the submission details.
5. The administrator confirms that the submission should be deleted.
6. The system prompts for final confirmation to prevent accidental removal.
7. The administrator confirms deletion.
8. The system removes the submission from active storage.
9. The system logs the deletion event with relevant details.

10. The system confirms successful deletion to the administrator.

Alternate Flows:

5A1: Administrator cancels deletion

- The administrator decides the report does not require deletion.
- The administrator exits the process or marks the report as “reviewed.”
- The system retains the report and the use case ends.

7A1: Deletion fails due to server error

- The system encounters a database issue during deletion.
- The error is logged and the administrator is notified.
- The administrator may retry deletion later once the issue is resolved and returns to step 4.

8A1: Report is locked by another process

- The system detects that the report is currently being accessed or modified by another user.
- The system prevents deletion until the lock is released.
- The administrator is notified to retry later and returns to step 4.

Use Case: Admin Backs Up Database

Description:

Admin creates a backup snapshot of the system database on the approved storage location.

Actors:

- Administrator (Primary)
- System
- Storage (Secondary)

Triggers:

- Admin selects "Backup Database".

Pre-conditions:

- Admin authenticated.
- Backup storage location is available.

Post-conditions:

- Backup file created and catalogued.
- The event is logged.

Normal Flow:

1. Admin opens the backup/restore page.
2. Admin selects "Create Backup".
3. The system validates storage access and capacity.
4. The system creates a backup snapshot and stores it.
5. The system logs backup event and confirms.

Alternate Flows:

3A1: Storage unavailable

- The system does not have enough storage, the system logs failure.
- The use case ends.

Use Case: Admin Restores Database

Description:

The Admin restores the database to a chosen backup version after confirmation.

Actors:

- Administrator (Primary)
- System

Triggers:

- Admin selects "Restore Database".

Pre-conditions:

- Admin authenticated.
- At least one backup exists.

Post-conditions:

- Database restored; event logged.

Normal Flow:

1. Admin selects a backup version.
2. The system displays a warning (overwrites current data).
3. Admin confirms restore.
4. The system performs restore and verifies completion.
5. The system logs restore event and confirms.

Alternate Flow:

4A1: Restore integrity check fails

- System blocks restore and logs error
- The use case ends.

Use Case: Admin Approve/Reject Forms

Description:

The Admin reviews a submitted form template and decides to either approve it for use in the system or reject it with a provided reason.

Actors:

- Administrator (Primary)
- System

Triggers:

- Admin selects a form from the “Pending Forms” list for review.

Pre-conditions:

- Admin is authenticated.
- Form template exists with status “pending”.

Post-conditions:

-
- Form status is updated to “approved” or “rejected”.
- If rejected, a reason is stored.
- Event is logged in the audit system.
-

Normal Flow:

1. Admin navigates to the form review section.
2. The system displays a list of pending form templates.
3. Admin selects a form to review.
4. The system displays form details (fields, description, version, etc.).
5. Admin chooses to approve or reject the form.

For Approval:

- 6A. Admin clicks “Approve”.
- 7A. The system updates the form status to “approved”.
- 8A. The system records approval metadata (admin ID, timestamp).
- 9A. The system logs the approval event and confirms success.

For Rejection:

- 6B. Admin clicks “Reject”.
- 7B. The system prompts for a rejection reason.

8B. Admin submits the reason.

9B. The system updates the form status to “rejected” and stores the reason.

10B. The system logs the rejection event and confirms success.

Alternate Flow:

4A1: Form is no longer pending (already reviewed)

- System prevents action and notifies admin.
- The use case ends.

7B1: Rejection reason is empty or invalid

- System prompts admin to provide a valid reason.
- Admin retries submission.

Use Case: Log In

Description:

A registered account logs in and is routed to a role-specific landing page.

Actors:

- User/Researcher/Provider/Admin (Primary)
- System
- 2FA Service (Secondary)

Triggers:

- The actor selects “Log In”.

Pre-conditions:

- The account exists and is active.
- 2FA is configured.

Post-conditions:

- An authenticated session is created.
- A view event will be recorded in the audit log.

Normal Flow:

1. The actor enters email + password.
2. System validates credentials.
3. System prompts for 2FA code.
4. The actor enters code.
5. The system creates a session and loads a landing page.

Alternate Flows:

2A1: Invalid credentials

- The system informs actor that the credentials used are invalid
- The actor returns to step 1.

2A2: Pending approval

- System shows status
- The use case ends.

4A1: Invalid 2FA

- The actor fails to authenticate the account
- Return to step 3.

Use Case: Update Account Information (All Roles)

Description:

- A logged-in user updates profile details (contact info, preferences). Profiles may be updated, and changes are logged.

Actors:

- Logged-in User (Primary)
- System

Triggers:

- Actor selects the “Settings” options on their profile.

Pre-conditions:

- Actor is authenticated.

Post-conditions:

- Profile data is updated; audit log entry created.

Normal Flow:

1. Actor opens Settings.
2. The system displays editable profile fields allowed for that role.
3. The actor edits fields and selects “Save”.
4. System validates input.
5. The system saves changes and records an audit log entry.
6. The system confirms success and returns to profile page.

Alternate Flows:

3A1: Actor cancels

- No changes saved, actor returns to dashboard
- The use case ends.

4A1: Validation fails

- The system shows errors in input data.
- The actor returns to step 2.

Stretch Goal Use Cases

Stretch Goal Use Case: Reminders for Data Entry

Description:

The system sends reminders to users to complete forms or enter data according to a schedule (daily/weekly) or based on incomplete to-do items.

Actors:

- System (Primary)
- User (Secondary)

Trigger:

A reminder schedule is reached OR a to-do form remains incomplete for a configured time period.

Preconditions:

- The user has an active account.
- The user has notification settings enabled (email/app notification).

Post-conditions:

- Reminder is sent and recorded.

Normal Flow:

1. The system checks which users have yet to complete forms according to the relevant reminder rules.
2. The system checks each of those users' notification preferences.
3. The system takes this information into account, and notifies each user through their preferred method
4. The user receives the reminder notification with the relevant information according to their preferences.
5. The user completes and submits the required form(s).
6. The system records the reminder event (and delivery status).

Alternate Flows:

2A1: No notification preferences set

- The system uses default settings (TBD).

- The system prompts the user to set their own preferences along with the reminder message.

4A1: Notification delivery fails

- The system attempts to notify the user via an alternate channel, if available.
- Should all available channels fail, the system logs the failure and ends notification attempts for a period.

5A1: The user receives the notification, but dismisses it without any action

- The user selects the "Remind me later" option.
- The system schedules a follow-up reminder to be delivered shortly after..

Stretch Goal Use Case: System-Generated Suggestions (Automated Insight)

Description:

The system generates automated, non-clinical insights and suggestions for users based on trends in their data, and may recommend resources or prompt goal updates.

Actors:

- System (Primary)
- User (Secondary)
- Healthcare Provider (may view suggestion summary)

Trigger:

- Periodic analysis job runs OR new form submission triggers analysis.

Preconditions:

- The user has sufficient data history for analysis.
- Suggestion feature enabled in settings.

Post-conditions:

- Suggestions are generated and displayed on the user dashboard.
- Event logged.

Normal Flow:

1. The system detects that an analysis should be run, whether scheduled or via a trigger
2. The system retrieves the user's recent health data and baseline history.
3. The system identifies any patterns, trends, or anomalies.
4. The system evaluates if any discovered patterns warrant generation of a suggestion.
5. The system generates an appropriate suggestion.
6. The suggestion is sent to the user's dashboard to be displayed, and the user is notified.
7. The user views the suggestion.
8. The system logs the suggestion generation event

Alternate Flows:

2A1: Insufficient data for analysis

- The user has less than the minimum data threshold required to generate a suggestion.
- The system skips suggestion generation and logs "insufficient data".

3A1: Analysis error

- The system encounters an error when analyzing user data for patterns
- The error event is logged and the system tries again on the next scheduled run

4A1: The user has disabled suggestions

- The user has opted out of automated insight generation.
- The system skips generating suggestions for the user.

Feature List

Database

Description: Design database architecture containing all fields, tables, and relationships necessary to support required user, researcher, healthcare provider, and admin features. Implement database architecture specified in the database architecture plan. Run tests to ensure databases are working correctly.

Priority: 1 (Very High)

Acceptance Criteria: Database architecture supports all required entities and relationships The database functions with the correct architecture. All tests passed.

Account Management

Login/Registration Page User Interface (All Roles)

Description: Design a login/registration page for users of all roles to log in to an existing account with the correct credentials or register for an account. Implement the login/registration page UI based on the design specified in the plan. Ensure that UI contains correct elements and they behave properly

Priority: 2 (High)

Acceptance Criteria: Login and registration page The UI contains all required elements. The UI behaves correctly for user registration and login. All UI tests pass successfully

User Registration System (All Roles)

Description: Design a system to allow new users of all roles to register for an account. Users will be prompted for what type of user they are (user, researcher, healthcare provider, or admin) and then asked to enter their email address along with a password and additional credentials related to their role. Create a new account using the information entered, assign permissions based on role, and add it to the database. Implement the user registration system based on the plan. Test the registration system to ensure correct behavior.

Priority: 2 (High)

Acceptance Criteria: Users can successfully register for an account based on their selected role Accounts are created with correct role-based permissions. The registration system behaves correctly under expected conditions. All tests pass successfully

Password Reset System

Description: Users of the project may forget their passwords, causing them to be unable to access their accounts. To remedy this problem, a password reset system needs to be designed. Implement the password reset system based on the plan. Test the password reset system to ensure correct behavior

Priority: 3 (Medium)

Acceptance Criteria: Users can securely reset their password via email and log in with the new password. Functioning password reset system. All tests pass

Design Account Settings Page User Interface (All Roles)

Description: Users may wish to alter their account settings/info after registration, an account settings user interface will need to be designed to support this.

Priority: 3 (Medium)

Acceptance Criteria: Documentation detailing plan for account settings UI is produced.

Implement Account Settings Page User Interface (All Roles)

Description: Implement account settings UI based on plan.

Priority: 3 (Medium)

Acceptance Criteria: Functioning account settings UI.

Test Account Settings Page User Interface (All Roles)

Description: Test account settings page to ensure correct behavior.

Priority: 4 (Low)

Acceptance Criteria: All tests pass.

Account Settings System (All Roles)

Description: Users may wish to alter their account settings/info over time. The project needs a system to update the relevant settings/info stored in the database to the new values to support this. Implement the account settings system based on the plan. Test the account settings system to ensure correct behavior.

Priority: 3 (Medium)

Acceptance Criteria: Users' account settings can be updated in the system, and changes are correctly saved in the database. Functioning account settings system. All tests pass.

Authentication

User Authentication System (All Roles)

Description: Users of all roles (users, researchers, healthcare providers, and admins) need a way to securely log in. The project needs an authentication system to authenticate users and give them the correct permissions for data access. Implement the user authentication based on the plan. Test the user authentication system to ensure correct behavior.

Priority: 2 (High)

Acceptance Criteria: Users of all roles can log in securely, and the system grants the correct access permissions based on their role. Functioning user authentication system. All tests pass.

User Permissions System

Description: Different users (user, researcher, healthcare provider, admin) have access to different data based on their role. The project needs a permissions system to determine who has access to the database. Implement the user permissions system based on the plan. Test the user permissions system to ensure correct behavior

Priority: 2 (High)

Acceptance Criteria: The system correctly enforces role-based access, allowing each user type to access only the data permitted for their role. Functioning user permissions system. All tests pass.

Form Submission

Design Form Fillout Page User Interface

Description: Users need a way to fill out forms with relevant health information. The project needs a user interface to support this.

Priority: 2 (High)

Acceptance Criteria: Documentation detailing the form fill out UI plan is produced.

Implement Form Fillout User Interface

Description: Implement form fill out page based on plan.

Priority: 2 (High)

Acceptance Criteria: Functioning form fill out UI.

Test Form Fillout User Interface

Description: Test form fill out UI to ensure all necessary elements exist and behave correctly

Priority: 4 (Low)

Acceptance Criteria: All tests pass

Form Submission System

Description: The project needs a system to take in a form submitted by a user, check that the input fields are valid, and if so, add the new information to the database for future use.

Implement the form submission system based on the plan. Test the form submission system to ensure correct behavior

Priority: 2 (High)

Acceptance Criteria: The system validates submitted form data and, if valid, stores it correctly in the database for future use. Functioning form submission system. All tests pass.

Form Input Validation System

Description: The input fields of submitted forms must be validated before being added to the database; otherwise, the user must redo the invalid fields. The project needs a way to determine whether or not a form's input values are valid. Implement the form validation system based on the plan. Test the form input validation system to ensure correct behavior.

Priority: 2 (High)

Acceptance Criteria: The system checks all form inputs for validity and prompts the user to correct any invalid fields before saving the data to the database. Functioning form input validation system. All tests pass.

Form Creation/Modification

Design Form Creation/Modification User Interface

Description: The project needs to provide a user interface for the creation of new forms and the modification of existing forms.

Priority: 3 (Medium)

Acceptance Criteria: Documentation detailing the form creation/modification UI plan is produced.

Implement Form Creation/Modification User Interface

Description: Implement the form creation/modification UI based on the plan.

Priority: 3 (Medium)

Acceptance Criteria: Functioning form creation/modification UI.

Test Form Creation/Modification User Interface

Description: Test the form creation/modification UI to ensure all necessary elements exist and behave correctly.

Priority: 4 (Low)

Acceptance Criteria: All tests pass.

Form Creation/Modification System

Description: The project needs a system to support the creation of new forms and the modification of existing forms. Upon receiving a new or changed form, the system will check if the form is valid, and if so, the new form is added to the database or the changed form replaces the old version. Implement the form creation/modification system based on the plan. Test the form creation/modification system to ensure correct behavior

Priority: 3 (Medium)

Acceptance Criteria: Documentation detailing the form creation/modification system plan is produced. Functioning form creation/modification system. All tests pass.

Design Form Validation System

Description: The project needs a system to validate newly created forms and updated existing forms before they can be added to the database.

Priority: 3 (Medium)

Acceptance Criteria: Documentation detailing the form validation system plan is produced

Implement Form Validation System

Description: Implement the form validation system based on the plan.

Priority: 3 (Medium)

Acceptance Criteria: Functioning form validation system.

Test Form Validation System

Description: Test the form validation system to ensure correct behavior.

Priority: 4 (Low)

Acceptance Criteria: All tests pass.

User Features

Design Personal Dashboard User Interface

Description: Create a dashboard layout and navigation plan showing where key user information will appear. This will include to-do forms, recent submissions, goal progress, and visual summaries.

Priority: 1 (Very High)

Acceptance Criteria: Documentation showing all required dashboard navigation elements and sections.

Implement Personal Dashboard User Interface

Description: Build the dashboard page in the frontend based on the design. Implement widgets for to-do forms, recent entries, goal progress, and chart dependencies. Display empty states if the user has no data.

Priority: 1 (Very High)

Acceptance Criteria: A backend API endpoint successfully returns dashboard only for the authenticated user.

Test Personal Dashboard User Interface

Description: Verify the API returns correct results per user and blocks unauthorized access.

Priority: 1 (Very High)

Acceptance Criteria: All tests pass. Unauthorized access to dashboard data is blocked and valid access events are correctly recorded in the audit log.

Design Health Goals User Interface

Description: Create the UI layout for viewing, creating, editing and tracking health goals, including empty states and navigation from dashboard.

Priority: 2 (High)

Acceptance Criteria: Documentation for the Health Goals page exists showing all the required screens and action

Implement Health Goals User Interface

Description: Build the Health Goals page in the frontend, including goal list view, add/edit forms, and display of goal status/progress.

Priority: 2 (High)

Acceptance Criteria: The Health Goals UI allows a user to create, update, and view goals using live backend data.

Test Health Goals User Interface

Description: Test goal UI rendering, validation messages, navigation, and empty states across common screen sizes.

Priority: 2 (High)

Acceptance Criteria: All tests pass. The Health Goals UI works correctly without layout or navigation errors on mobile and desktop views.

Design Health Goals System

Description: Define the backend goal data model, API endpoints, validation rules, and how goal progress is calculated from submissions. Implement backend CRUD for goals (create/read/update), input validation, RBAC checks, and persistence in the database. Add unit/integration tests for goal endpoints, including invalid input, unauthorized access, and correct persistence.

Priority: 2 (High)

Acceptance Criteria: Documentation exists describing goal storage, APIs, and progress calculation rules.

Design Personal Health Summary User Interface

Description: Design the UI for a personal health summary page showing key metrics, trends, and simple charts/tables for a selected time range.

Priority: 1 (Very High)

Acceptance Criteria: Documentation for the Personal Health Summary page exists with defined metrics and chart placements.

Implement Personal Health Summary User Interface

Description: Implement the summary page UI, including metric cards, charts, date filters, and loading/error states.

Priority: 1 (Very High)

Acceptance Criteria: The Personal Health Summary UI displays charts and metrics correctly for a selected user and date range.

Test Personal Health Summary User Interface

Description: Test chart rendering, filtering, and responsiveness, including “no data” scenarios.

Priority: 1 (Very High)

Acceptance Criteria: All tests pass. The Personal Health Summary UI produces correct displays for normal and empty datasets without UI errors.

Personal Health Summary System

Description: Define how summary metrics are computed from submissions (aggregation rules, time windows, required fields) and what endpoints support the UI. Implement backend aggregation logic and API endpoints to return computed metrics and chart-ready time-series data for the authenticated user. Create automated tests verifying metric calculations, filtering correctness, and access control for summary endpoints.

Priority: 1 (Very High)

Acceptance Criteria: Documentation exists describing required inputs, calculations, and API responses. The summary API returns accurate computed metrics and time-series data for the authenticated user only. All tests pass. Automated tests confirm summary calculations match expected results and are protected by role permissions.

Design Personal Health Comparison User Interface

Description: Design the UI for comparing a user’s metrics to anonymized group aggregates, including group selection, date filters, and comparison charts.

Priority: 2 (High)

Acceptance Criteria: Layout exists for the comparison page showing group selection and comparison visualizations.

Implement Personal Health Comparison User Interface

Description: Build the comparison page UI, including group dropdowns, metric selection, comparison charts, and handling of “group too small” warnings.

Priority: 2 (High)

Acceptance Criteria: The comparison UI displays the user metric alongside group aggregate values using live backend results.

Test Personal Health Comparison User Interface

Description: Test comparison filtering, chart rendering, and correct error handling when aggregates are unavailable or suppressed.

Priority: 2 (High)

Acceptance Criteria: All tests pass. The comparison UI correctly handles valid comparisons and displays suppression/empty states without breaking.

Personal Health Comparison System

Description: Define anonymization rules, minimum group size thresholds, aggregate calculation methods, and API contracts for comparison outputs. Implement backend endpoints that compute anonymized aggregate metrics for selected cohorts and return comparison-ready data while enforcing suppression rules. Add automated tests verifying suppression behavior, correct aggregates, and access control for comparison endpoints.

Priority: 2 (High)

Acceptance Criteria: A comparison system design exists defining cohort rules, suppression thresholds, and API response formats. The comparison API returns anonymized aggregates that obey minimum group-size rules and contain no identifiers. All tests pass. Automated tests confirm aggregates are correct and suppressed outputs occur when cohort size is below threshold.

Researchers Features

Design Researcher Home Page User Interface

Description: Design the layout and navigation of the researcher home page, including access to reports, cohorts, filters, and export tools.

Priority: 1 (Very High)

Acceptance Criteria: Wireframe exists showing all researcher home page sections and navigation paths.

Implement Researcher Home Page User Interface

Description: Implement the researcher home page UI based on the design, including links to report generation, filtering, and exports.

Priority: 1 (Very High)

Acceptance Criteria: The researcher home page renders correctly and provides access to all researcher features.

Test Researcher Home Page User Interface

Description: Test layout, navigation, and responsiveness of the researcher home page across supported browsers and screen sizes.

Priority: 1 (Very High)

Acceptance Criteria: All tests pass. The researcher home page functions correctly without layout or navigation errors.

Aggregated Report Generation System

Description: Allows users (e.g., researchers or admins) to select cohorts, metrics, and date ranges to generate aggregated reports. Includes UI for selection, backend report generation logic, and validation of results.

Priority: 1 (Very High)

Acceptance Criteria: Users can select cohorts, metrics, and date ranges via the interface. The system generates aggregated reports correctly based on selected parameters. All UI elements behave correctly and the reported data is accurate. All tests for report generation pass successfully.

Design Cohort Definition User Interface

Description: Design the researcher screens for creating, editing, viewing, and selecting cohorts, including cohort name, filter rules, and cohort size preview.

Priority: 1 (Very High)

Acceptance Criteria: A finalized cohort management wireframe exists showing cohort creation and edit workflows.

Implement Cohort Definition User Interface

Description: Implement the cohort management UI with controls for selecting filters (demographics/attributes), saving cohorts, and viewing saved cohorts.

Priority: 1 (Very High)

Acceptance Criteria: The cohort UI allows a researcher to create and edit cohort definitions using live backend data.

Test Cohort Definition User Interface

Description: Test cohort creation/edit flows, input validation, error states, and responsiveness across supported devices.

Priority: 1 (Very High)

Acceptance Criteria: The cohort UI works correctly for valid and invalid inputs without layout or navigation errors.

Cohort Definition System

Description: Define cohort storage format, allowed filter fields, cohort size calculation, minimum cohort-size threshold rules, and cohort versioning. Implement backend endpoints and database logic to create, retrieve, update, and list cohort definitions, including cohort size estimation and threshold enforcement. Add automated tests for cohort creation/update, filter validation, size estimation, threshold enforcement, and researcher-only permissions.

Priority: 1 (Very High)

Acceptance Criteria: A cohort system design exists defining allowed filters, size rules, and how cohorts are stored and versioned. The system stores cohort definitions and enforces minimum cohort-size rules when cohorts are used. Automated tests confirm cohort endpoints enforce permissions and correctly apply size and validation rules.

Design Researcher Database Update Workflow

Description: Define what “updating the database” means for researchers through safe, controlled operations (e.g., reference data updates, bulk imports of approved datasets, and recalculation triggers), including validation rules and approval requirements.

Priority: 2 (High)

Acceptance Criteria: A database update workflow design document exists defining permitted researcher update actions and required approvals.

Design Researcher Database Update User Interface

Description: Design UI screens that allow researchers to submit a database update request, upload a file if needed (e.g., CSV), view validation results, and track approval status.

Priority: 2 (High)

Acceptance Criteria: Layout exists for submitting and tracking researcher database update requests.

Implement Researcher Database Update User Interface

Description: Implement the UI for submitting update requests, uploading files, viewing request history, and displaying approval status and error messages.

Priority: 2 (High)

Acceptance Criteria: The researcher UI allows submission of a database update request and displays its validation and approval status.

Researcher Database Update System

Description: Allows researchers to submit controlled database update requests, including file uploads, validation, approval workflows, safe application of updates, rollback on failure, and audit logging. The system ensures updates cannot be applied without approval and provides feedback to researchers on validation and approval status.

Priority: 2 (High)

Acceptance Criteria: Researchers can submit update requests and track approval status. All updates are validated according to defined rules before approval. Updates cannot be applied without approval. Approved updates are applied safely with rollback on failure and full audit logging.

Implement Aggregated Report Generation User Interface

Description: Implement the UI for configuring and generating aggregated reports, including loading and error states.

Priority: 1 (Very High)

Acceptance Criteria: The aggregated report UI allows a researcher to generate reports using live backend data.

Test Aggregated Report Generation User Interface

Description: Test report configuration controls, result display, and handling of invalid or empty selections.

Priority: 1 (Very High)

Acceptance Criteria: The aggregated report UI correctly handles valid, invalid, and empty report requests.

Data Filtering System

Description: Define filtering rules and parameters (date range, cohort, metric selection) used across researcher reports. Implement backend logic to apply filters consistently to aggregated report queries. Test filter combinations, boundary cases, and invalid inputs for researcher queries.

Priority: 1 (Very High)

Acceptance Criteria: A filtering system design exists defining supported filters and validation rules. Filtered queries return correct aggregated results based on selected criteria. Automated tests confirm filtering behaves correctly for valid and invalid inputs.

Aggregated Report Generation System

Description: Define how anonymized aggregates are computed, including suppression rules and metric calculations. Implement backend services that compute anonymized aggregated reports from stored health data. Test aggregation accuracy, suppression behavior, and role-based access control.

Priority: 1 (Very High)

Acceptance Criteria: A system design exists describing aggregation logic and anonymization constraints. The system generates anonymized aggregated reports that follow defined rules. Tests confirm aggregates are correct and privacy rules are enforced.

Report Export System

Description: Design the process for exporting aggregated reports to CSV format with anonymization preserved. Implement CSV export functionality for aggregated reports. Test CSV exports for correctness, formatting, and anonymization.

Priority: 2 (High)

Acceptance Criteria: A design exists specifying export format and privacy constraints.

Aggregated reports can be exported to CSV without including identifiable data. Exported CSV files match the expected structure and content.

Implement Report Appending User Interface

Description: Implement the UI for adding and viewing appended notes on reports.

Priority: 3 (Medium)

Acceptance Criteria: Researchers can add and view appended notes through the UI.

Test Report Appending User Interface

Description: Test note submission, display, and error handling in the report appending UI.

Priority: 3 (Medium)

Acceptance Criteria: The report appending UI works correctly without submission or display errors.

Report Appending System

Description: Define how appended notes are stored, versioned, and linked to reports. Implement backend logic to store appended notes and maintain report version history. Test note persistence, version history, and permission enforcement.

Priority: 3 (Medium)

Acceptance Criteria: A design exists describing report versioning and note storage. Report notes are stored correctly and older versions remain accessible. Tests confirm appended notes persist and permissions are enforced.

Healthcare Provider Features

Design Healthcare Provider Home Page User Interface

Description: Design the healthcare provider home page user interface, healthcare providers are presented with options to view health data of patients in their care and give patient feedback.

Priority: 2 (High)

Acceptance Criteria: Documentation detailing the healthcare provider home page UI plan.

Implement Healthcare Provider Home Page User Interface

Description: Implement the healthcare provider home page UI based on the plan.

Priority: 2 (High)

Acceptance Criteria: Functioning healthcare provider home page UI.

Test Healthcare Provider Home Page User Interface

Description: Test the healthcare provider home page user interface to ensure all necessary elements are present and behave correctly.

Priority: 4 (Low)

Acceptance Criteria: All tests pass.

Design Healthcare Provider Patient Data User Interface

Description: Design a user interface that allows healthcare providers to view health data of patients in their care.

Priority: 2 (High)

Acceptance Criteria: Documentation detailing the healthcare provider patient data UI design plan.

Implement Healthcare Provider Patient Data User Interface

Description: Implement the healthcare provider patient data UI based on the plan.

Priority: 2 (High)

Acceptance Criteria: Functioning healthcare provider patient data UI.

Test Healthcare Provider Patient Data User Interface

Description: Test the healthcare provider patient data UI to ensure all necessary elements are present and behave correctly.

Priority: 4 (Low)

Acceptance Criteria: All tests pass.

Healthcare Provider Patient Data Retrieval System

Description: Design a system to allow healthcare providers to retrieve health data of patients in their care from the database. The system must ensure that healthcare providers cannot access data of patients not in their care. Implement the healthcare provider patient data retrieval system based on the plan. Test the healthcare provider patient data retrieval system to ensure correct behavior.

Priority: 2 (High)

Acceptance Criteria: Documentation detailing the healthcare provider patient data retrieval system plan. Functioning healthcare provider patient data retrieval system. All tests pass.

Design Healthcare Provider Feedback User Interface

Description: Design a user interface for healthcare providers to give feedback to patients in their care.

Priority: 3 (Medium)

Acceptance Criteria: Documentation detailing the healthcare provider patient feedback UI design plan.

Implement Healthcare Provider Feedback User Interface

Description: Implement the healthcare provider feedback UI based on the plan.

Priority: 3 (Medium)

Acceptance Criteria: Functioning healthcare provider feedback UI.

Test Healthcare Provider Feedback User Interface

Description: Test the healthcare provider feedback UI to ensure all necessary elements are present and behave correctly.

Priority: 4 (Low)

Acceptance Criteria: All tests pass.

Design Healthcare Provider Feedback System

Description: Design a system that allows healthcare providers to send feedback messages to patients in their care. Implement the healthcare provider feedback system based on the plan. Test the healthcare provider feedback system to ensure correct behavior.

Priority: 3 (Medium)

Acceptance Criteria: Healthcare providers can send feedback messages to patients in their care, and patients can receive and view that feedback securely. Functioning healthcare provider feedback system. All tests pass.

Admin Features

Design Admin Page User Interface

Description: Design the home page layout that links to account approvals, audit logs, report moderation and database backup/restore tools.

Priority: 1 (Very High)

Acceptance Criteria: Documentation exists for admin home page showing navigation to all admin tools.

Implement Admin Page User Interface

Description: Implement the admin home page UI with functional navigation and role-protected access to admin features.

Priority: 1 (Very High)

Acceptance Criteria: The admin page renders correctly and provides access only to admin-authorized functionality.

Test Admin Page User Interface

Description: Test admin page rendering, responsiveness and navigation, including unauthorized access attempts.

Priority: 1 (Very High)

Acceptance Criteria: All tests pass.

Audit Log System

Description: Define what events must be recorded (logins, data access, exports, approvals, reports deletion, backup/restore) and what fields are stored per entry. Implement backend logging that records required events in an append-only manner with timestamps and actor identity. Add automated tests that verify audit events are created for key actions and cannot be modified by non-admin users.

Priority: 1 (Very High)

Acceptance Criteria: An audit log specification exists defining required events and audit log record fields. Required security and data-access events are recorded as audit log entries in the database. All tests pass. Automated tests confirm audit events are created for key actions and audit records are not editable.

Design Audit Log User Interface

Description: Design an audit UI to view and filter audit logs by date, actor, role, action type and target object.

Priority: 1 (Very High)

Acceptance Criteria: Wireframe exists for an audit log viewer with filter controls.

Implement Audit Log User Interface

Description: Implement the audit log viewer UI, including filtering and viewing detailed log entries.

Priority: 1 (Very High)

Acceptance Criteria: Admin users can view and filter audit logs through the UI using live backend data.

Test Audit Log User Interface

Description: Test filtering, empty states and performance for the audit log viewer.

Priority: 1 (Very High)

Acceptance Criteria: All tests pass.

Design Admin Report Review User Interface

Description: Design the UI for reviewing generated reports and taking moderation actions such as flagging or deleting a problematic report.

Priority: 1 (Very High)

Acceptance Criteria: Wireframe exists for report review and moderation actions.

Implement Admin Report Review User Interface

Description: Implement the admin UI for listing reports, viewing report details and selecting moderation actions.

Priority: 1 (Very High)

Acceptance Criteria: The admin report review UI allows an admin to view reports and initiate moderation actions.

Test Admin Report Review User Interface

Description: Test report listing , report viewing, moderation workflow and error handling.

Priority: 1 (Very High)

Acceptance Criteria: All tests pass.

Admin Report Review System

Description: Define how reports are stored, how a report is marked as problematic and how deletion affects visibility to other roles. Implement backend endpoints that support report listings, report retrieval, and report deletion with permission checks and audit logging. Add automated tests verifying only admins can moderate reports and moderated reports are no longer accessible to other roles.

Priority: 1 (Very High)

Acceptance Criteria: A moderation system design exists defining report lifecycle states and moderation effects. Admin moderation actions successfully change report visibility and are logged in the audit log. Tests confirm report moderation is admin-only and moderated reports are inaccessible to non-admin users.

Design Database Backup/Restoration User Interface

Description: Design an admin UI to create a manual backup, view backup history and select a restore point with confirmations.

Priority: 1 (Very High)

Acceptance Criteria: Wireframe exists for backup creation, backup history, and restoration confirmation screens.

Implement Database Backup/Restoration User Interface

Description: Implement the UI for backup/restore actions, including status messages, progress indicators and error handling.

Priority: 1 (Very High)

Acceptance Criteria: Admin users can trigger backups and restore operations through the UI and see success or failure status.

Test Database Backup/Restoration User Interface

Description: Test backup/restore UI workflows, confirmations and error states.

Priority: 1 (Very High)

Acceptance Criteria: The backup/restore UI correctly handles success, failure, and confirmation flows without UI errors.

Database Backup/Restoration System

Description: Define backup frequency options, backup storage location strategy, retention rules, and the restore procedure, including rollback and downtime considerations. Implement backend support for backup creation and restoration, including permission checks and audit log recording. Test backup generation, restore execution, failure handling, and verification that the restored state matches the selected backup.

Priority: 1 (Very High)

Acceptance Criteria: A backup/restore design exists defining backup creation, retention, and restore procedures. Database backups can be created and restored successfully and all actions are recorded in the audit log. Automated tests confirm backups restore the system to an expected previous state.

Database Schema Modification System

Description: Define how schema changes will be managed using mitigations under admin control, including review and rollback procedures. Implement mitigation tooling and an admin-controlled execution process for applying approved schema mitigations in non-production and production environments. Test migrations on a staging dataset to verify schema updates do not break core workflows and can be rolled back.

Priority: 3 (Medium)

Acceptance Criteria: A migration-based schema update plan exists defining how schema changes are applied and rolled back. Approved migrations can be applied and rolled back using a documented process without corrupting data. All tests pass.

Design Admin Form Review User Interface

Description: Design the admin interface for reviewing submitted form templates, viewing form details, and taking approval or rejection actions.

Priority: 1 (Very High)

Acceptance Criteria: Wireframe exists for form review page, form detail view, and approve/reject actions.

Implement Admin Form Review User Interface

Description: Implement the admin UI for listing pending forms, opening individual form details, and allowing the admin to approve or reject a form.

Priority: 1 (Very High)

Acceptance Criteria: The admin form review UI allows an admin to view pending forms, inspect form details, and initiate approve or reject actions.

Implement Form Approval Workflow

Description: Implement the backend logic for approving a pending form, updating its status, and recording approval metadata such as approver ID and timestamp.

Priority: 1 (Very High)

Acceptance Criteria: When an admin approves a pending form, the form status changes to approved and approval metadata is stored correctly.

Implement Form Rejection Workflow

Description: Implement the backend logic for rejecting a pending form, requiring a rejection reason, and storing that reason with the updated rejected status.

Priority: 1 (Very High)

Acceptance Criteria: When an admin rejects a pending form, the form status changes to rejected and the rejection reason is saved.

Implement Audit Logging for Form Review Actions

Description: Record admin approval and rejection actions in the audit log for accountability and traceability.

Priority: 1 (Very High)

Acceptance Criteria: Each form approval or rejection action creates an audit log entry with the admin ID, action type, and timestamp.

Validate Admin Permissions for Form Review

Description: Ensure only authenticated admins can access the form review page and perform approve or reject actions.

Priority: 1 (Very High)

Acceptance Criteria: Non-admin users are denied access to form review features, while admins can access them successfully.

Validate Rejection Reason Input

Description: Ensure that a rejection reason is provided and validated before a form can be rejected.

Priority: 2 (High)

Acceptance Criteria: The system prevents rejection if the reason is missing or invalid and displays an appropriate validation message.

Prevent Review of Already Processed Forms

Description: Ensure the system blocks approval or rejection actions if the form is no longer in pending status.

Priority: 2 (High)

Acceptance Criteria: The system displays an error and prevents duplicate review actions on already approved or rejected forms.

Test Admin Form Review User Interface

Description: Test pending form listing, form detail viewing, approval flow, rejection flow, and validation/error handling.

Priority: 1 (Very High)

Acceptance Criteria: All UI and workflow tests pass.

Test Admin Form Approval and Rejection Workflow

Description: Test backend status changes, metadata storage, rejection reason storage, permission checks, and audit log creation.

Priority: 1 (Very High)

Acceptance Criteria: All approval and rejection workflow tests pass.

Notifications/Reminders

Reminder System

Description: Design a system to send reminders to users notifying them to fill out their forms. Reminders can be sent on a schedule or after a certain time that the user has not logged in. Implement the reminder system based on the plan. Test the reminder system to ensure correct behavior.

Priority: 3 (medium)

Acceptance Criteria: The system automatically sends reminders to users based on a schedule or inactivity to prompt them to complete and submit forms.

Functioning reminder system. All tests pass.

Design Notification Center User Interface

Description: Design a notification center user interface allowing users to view a list of their notifications such as feedback from their healthcare provider or reminders to fill out and submit forms.

Priority: 3 (Medium)

Acceptance Criteria: Documentation detailing notification center UI plan.

Implement Notification Center User Interface

Description: Implement the notification center UI based on the plan.

Priority: 3 (Medium)

Acceptance Criteria: Functioning notification center UI.

Test Notification Center User Interface

Description: Test the notification center UI to ensure all necessary elements are present and behave correctly.

Priority: 4 (Low)

Acceptance Criteria: All tests pass.

Notification System

Description: Design a system to send notifications to the user's notification center with the option to also send the notification to the user's email account. Implement the notification system based on the plan. Test the notification system to ensure correct behavior.

Priority: 2 (High)

Acceptance Criteria: The system delivers notifications to the user's notification center and optionally sends the same notifications to the user's email. Functioning notification system. All tests pass.

System Documentation

Description: Produce comprehensive system documentation, including a user manual for all roles and technical documentation for developers and administrators.

Priority: 3 (Medium)

Acceptance Criteria: A complete user manual exists for all user roles. Technical documentation exists covering system architecture, workflows, and backend logic. Documentation is clear, accurate, and usable by intended audiences.

Stretch Goal Features

Stretch Goal: Reminders for Data Entry

Data Entry Reminder System

Description: Define how reminder schedules are configured, triggered, and delivered to users who have not submitted data within a defined time period. Implement backend logic to track user submissions and trigger reminder notifications based on configured rules. Test reminder triggers, timing accuracy, and failure handling.

Priority: 4 (Low)

Acceptance Criteria: A reminder system design exists defining trigger conditions and delivery timing. The system sends a reminder when a user misses a configured data submission window. Reminders trigger correctly under defined conditions without duplicate notifications.

Stretch Goal: System-Generated Health Suggestions (Automated Insights)

Automated Insight System

Description: Define rules or basic algorithms for generating health suggestions based on user data trends and thresholds. Implement logic to generate and store health suggestions when sufficient user data is available. Test insight generation accuracy and suppression when insufficient data exists.

Priority: 5 (Very Low)

Acceptance Criteria: An insight generation design exists defining input data and output rules. The system generates at least one suggestion when defined conditions are met. Insights are generated only when sufficient data is available.

Class Definitions

Visibility: Attributes should be made private to protect data integrity, access should be managed through public methods.

1. Identity, Accounts, and Access Control

This module manages user identities, security credentials, and Role-Based Access Control (RBAC) to ensure data privacy.

- **Account (Abstract):**
 - **Role:** Abstract base class representing any authenticated system actor.
 - **Attributes:** accountId: UUID, email: String, status: AccountStatus (Enum)
 - **Methods:** deactivateAccount(), requestDataExport()
 - **Relationships:**
 - 1 Account ↔ 1 AuthenticationCredential
 - 1 Account ↔ 0..* TwoFactorMethod
 - 1 Account ↔ 1 Role
 - 1 Account ↔ 0..* AuditLog (as actor)
 - **Structural Logic:**
 - Cannot be instantiated directly.
 - Encapsulates identity, authentication, and audit concerns shared by all accounts.
 - *deactivateAccount()* performs a soft delete by updating status.
- **User (extends Account):** The primary account holder (precariously housed individuals).
 - **Role:** Core domain entity; owns health data and dashboards.
 - **Attributes:** userID: UUID, name: String
 - **Methods:** updateProfile()
 - **Relationships:**
 - User ⊂ Account
 - 1 User ↔ 0..* HealthEntry
 - 1 User ↔ 1 Dashboard
 - 1 User ↔ 0..* HealthGoal
 - 1 User ↔ 0..* FormSubmission
 - 1 User ↔ 0..1 FamilyAccount

- **Structural Logic:**
 - *deactivateAccount()* soft-deletes the user (status change), preserving health data.
 - *requestDataExport()* triggers CSVExport and logs an audit entry.
- **Researcher (extends Account):** Specialized account for academic or community health analysis.
 - **Role:** Restricted analytical access to anonymized data only.
 - **Attributes:** researcherID: UUID, institution: String, verificationStatus: Boolean
 - **Methods:** applyForAccess(), viewAggregatedReport(), submitVerificationDocs()
 - **Relationships:**
 - Researcher $\subset$ Account
 - Researcher $\leftrightarrow$ 0..* Report
 - Researcher $\leftrightarrow$ 0..* Cohort
 - **Structural Logic:**
 - Cannot access *HealthEntry* directly.
 - All data access passes through *AggregatedData*.
- **HealthcareProvider (extends Account):** Authenticated healthcare professional account.
 - **Role:** Scoped access to patient data for care coordination and feedback.
 - **Attributes:** providerID: UUID, name: String, specialty: String, organization: String, verificationStatus: VerificationStatus (Enum)
 - **Methods:** reviewPatientData(), provideFeedback(), viewAppointments(), updateAvailability()
 - **Relationships:**
 - HealthcareProvider $\subset$ Account
 - HealthcareProvider $\leftrightarrow$ 0..* Appointment
 - HealthcareProvider $\leftrightarrow$ 0..* AuditLog
 - **Structural Logic:**
 - Cannot access raw *HealthEntry* records unless explicitly authorized through role permissions

- All data views are read-only by default; feedback is stored separately from original health data.
 - Provider actions (data access, feedback submission) generate audit log entries.
- **Administrator (extends Account):** High-level system users responsible for maintenance and oversight.
 - **Role:** System governance and compliance.
 - **Attributes:** adminID: UUID
 - **Methods:** approveResearcher(), reviewAuditLogs(), manageSystemConfig()
 - **Relationships:**
 - Administrator $\subset$ Account
 - Administrator $\leftrightarrow$ SystemConfiguration
 - **Structural Logic:**
 - Has permissions to override workflow states (e.g., researcher approval).
 - No direct access to raw health values unless explicitly permitted.
- **Role:** Defines the set of permissions associated with a user type.
 - **Role:** RBAC abstraction.
 - **Attributes:** roleName (User, Researcher, Provider, Admin): RoleType (Enum)
 - **Methods:** checkPermission(), assignToAccount()
 - **Relationships:**
 - 1 Role $\leftrightarrow$ 1.. Permission*
 - 1 Role $\leftrightarrow$ 0..* Account
 - **Structural Logic:**
 - Evaluated at request-time via *checkPermission(action, resource)*.
- **Permission:** Defines specific allowed actions and their scope.
 - **Role:** Authorization unit.
 - **Attributes:** permissionName: String, scope: String
 - **Methods:** validateScope(), isAuthorized()
 - **Relationships:**
 - Used by Role

- **Structural Logic:**
 - *scope* limits access (own data vs aggregated vs system-wide).
- **AuthenticationCredential:** Manages the secure storage and verification of login data.
 - **Role:** Authentication boundary.
 - **Attributes:** passwordHash: String, lastLogin: DateTime
 - **Methods:** verifyPassword(), updatePassword(), resetPassword()
 - **Relationships:**
 - 1 AuthenticationCredential ↔ 1 Account
- **Structural Logic:**
 - Passwords never leave this class.
 - Login success updates *lastLogin*.
- **TwoFactorMethod:** Handles the secondary layer of security for identity verification.
 - **Role:** Optional security enhancement.
 - **Attributes:** authenticationType: String, secretKey: String, enabled: Boolean
 - **Methods:** generateSecret(), verifyToken(), toggleStatus()
 - **Relationships:**
 - 0..* TwoFactorMethod ↔ 1 Account
- **Structural Logic:**
 - Enforced only when *enabled == true*.
 - Multiple methods allow fallback (E.g. SMS + Authenticator).

2. Health Data Collection and Storage

This module handles the intake of all sensitive health information via append-only logs.

- **HealthEntry:** A single granular piece of health data recorded by a user.
 - **Role:** Unit of health data.
 - **Attributes:** entryID: UUID, timestamp: DateTime, values: Map<String, Object>
 - **Methods:** encryptData(), validateData(), getFormattedValue()
 - **Relationships:**
 - 1 HealthEntry ↔ 1 User
 - 1 HealthEntry ↔ 1 FormSubmission
 - Source for AggregatedData and AutomatedInsight
 - **Structural Logic:**
 - Cannot be edited or deleted.
 - Encryption happens before persistence.
- **FormTemplate:** The structural definition of a health questionnaire.
 - **Role:** Blueprint for structured data collection.
 - **Attributes:** formID: UUID, version: Integer, status: String, description: String
 - **Methods:** publishNewVersion(), archiveTemplate(), renderForm()
 - **Relationships:**
 - 1 FormTemplate ↔ 1.. FormField*
 - 1 FormTemplate ↔ 0.. FormSubmission*
 - **Structural Logic:**
 - Versioned to preserve historical consistency.
 - Archived templates remain readable.
- **FormField:** Individual data points within a form.
 - **Role:** Validation and semantic definition of inputs.
 - **Attributes:** fieldID: String, label: String, typeFieldType (Enum), validationRules: JSON
 - **Methods:** executeValidation(), getPlaceholderText()
 - **Relationships:**
 - Belongs to exactly 1 FormTemplate

- **Structural Logic:**
 - *validationRules* executed client-side and server-side.
- **FormSubmission:** Represents the action of a user completing and submitting a specific form.
 - **Role:** Event capturing user interaction.
 - **Attributes:** submissionID: UUID, status: SubmissionStatus (Enum), submittedAt: DateTime
 - **Methods:** processSubmission(), linkToUser(), flagForReview()
 - **Relationships:**
 - 1 FormSubmission ↔ 1 User
 - 1 FormSubmission ↔ 1 FormTemplate
 - 1 FormSubmission ↔ 1.. HealthEntry*
 - **Structural Logic:**
 - Acts as the transactional boundary.
 - *processSubmission()* creates HealthEntry records.

3. Dashboards, Goals, and Insights

This module provides the user visualization and motivational tools for personal health management.

- **Dashboard**
 - **Attributes:** dashboardID: UUID
 - **Methods:** loadUserSummary(), refreshVisualizations()
 - **Relationships:**
 - 1 Dashboard ↔ 1 User
 - Dashboard ↔ 0.. HealthGoal*
 - Dashboard ↔ 0.. AutomatedInsight*
 - **Structural Logic:**
 - Pulls summaries, never raw data.
- **HealthGoal**
 - **Attributes:** goalID: UUID, targetValue: Float, timeFrame: DateRange, status: GoalStatus (Enum)
 - **Methods:** calculateProgress(), updateStatus(), checkDeadline()
 - **Relationships:**
 - 1 HealthGoal ↔ 1 User
 - Uses HealthEntry for progress calculation
 - **Structural Logic:**
 - Progress recalculated on new data entry.
- **AutomatedInsight** (stretch)
 - **Attributes:** insightID: UUID, message: String, createdAt: DateTime, confidenceLevel: Float
 - **Methods:** generateMessage(), dismissInsight()
 - **Relationships:**
 - Generated from HealthEntry
 - Governed by InsightRule
 - Displayed on Dashboard
 - **Structural Logic:**
 - Non-deterministic confidence-based insights.

- Can be dismissed but remains logged.
- **InsightRule** (stretch)
 - **Attributes:** ruleID: UUID, threshold: Float, condition: String
 - **Methods:** evaluateCondition(), triggerAction()
 - **Relationships:**
 - 1 InsightRule ↔ 0.. AutomatedInsight*
 - **Structural Logic:**
 - Acts as a rule engine input, not user-visible.

4. Reporting, Aggregation, and Research

This module facilitates the transformation of raw data into anonymized community insights.

- **Report**
 - **Attributes:** reportID: UUID, type: ReportType (Enum), createdAt: DateTime
 - **Methods:** generateSummary(), applyPrivacyFilters()
 - **Relationships:**
 - 1 Report ↔ 1.. AggregatedData*
 - 1 Report ↔ 0.. Visualization*
 - Generated by Researcher
- **AggregatedData**
 - **Attributes:** aggregateID: UUID, metrics: Map<String, Float>, anonymizationLevel: Integer
 - **Methods:** calculateMean(), checkMinimumCohortSize(), anonymize()
 - **Relationships:**
 - Derived from *HealthEntry*
 - Used by Report
 - **Structural Logic:**
 - Enforces minimum cohort sizes.
 - Cannot be reverse-mapped to users.
- **Visualization**
 - **Attributes:** visualizationType: ChartType (Enum)
 - **Methods:** generateChartData(), setDimensions()
 - **Relationships:**
 - Visualization ↔ 1 Report
- **CSVExport**
 - **Attributes:** exportID: UUID, createdAt: DateTime, filtersUsed: List<String>
 - **Methods:** prepareCSV(), logExportAction()
 - **Relationships:**
 - Triggered by User or Researcher
 - Logged in AuditLog

5. Cohorts, Groups, and Comparisons

This module is used to define sub-populations for targeted health analysis.

- **Cohort**
 - **Attributes:** cohortID: UUID, definitionCriteria: String
 - **Methods:** addMember(), calculateDemographics(), createdBy()
 - **Relationships:**
 - Cohort ↔ 1 CohortDefinition
 - Cohort ↔ 0.. User (logical, not direct reference)*
- **CohortDefinition**
 - **Attributes:** filters: List<Filter>, demographicRules: JSON
 - **Methods:** buildQuery(), validateCriteria()
 - **Structural Logic:**
 - Query-only object; no data ownership.

6. System Integrity, Auditing, and Operations

This module ensures the system is secure and reliable.

- **AuditLog**
 - **Attributes:** logID: Long, actionType: String, timestamp: DateTime, actorID: UUID
 - **Methods:** createEntry(), getImmutableRecord()
 - **Relationships:**
 - AuditLog ↔ 1 Actor (User / Admin / Researcher)
 - **Structural Logic:**
 - Append-only, immutable.
- **Notification**
 - **Attributes:** notificationID: UUID, messageString, channel: NotificationChannel (Enum)
 - **Methods:** sendEmail(), markAsRead(), dispatchPush()
 - **Relationships:**
 - Notification ↔ 1 User

- **SystemConfiguration**
 - **Attributes:** configKey: String, value: String
 - **Methods:** getValue(), updateSetting()
 - **Relationships:**
 - Managed by Administrator

7. Family and Shared Access (stretch)

Links multiple users under one household for shared management.

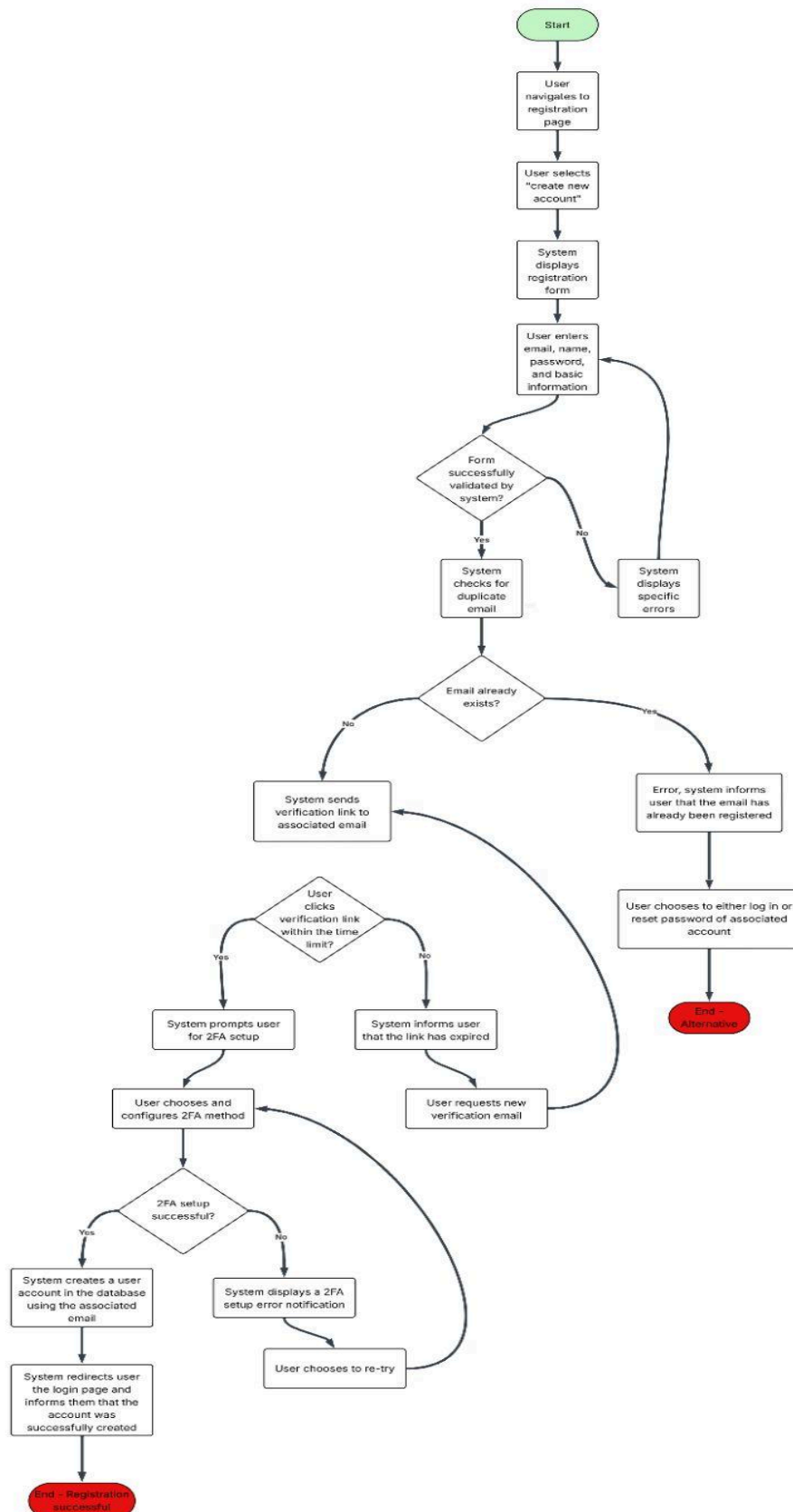
- **FamilyAccount**
 - **Attributes:** familyID: UUID
 - **Methods:** inviteMember(), removeMember()
 - **Relationships:**
 - FamilyAccount ↔ 1.. User*
 - FamilyAccount ↔ 0.. FamilyPermission*
- **FamilyPermission**
 - **Attributes:** accessLevel: Level (Enum)
 - **Methods:** setAccessLevel()
 - **Structural Logic:**
 - Restricts visibility (e.g., read-only vs manage).

8. Appointments (stretch)

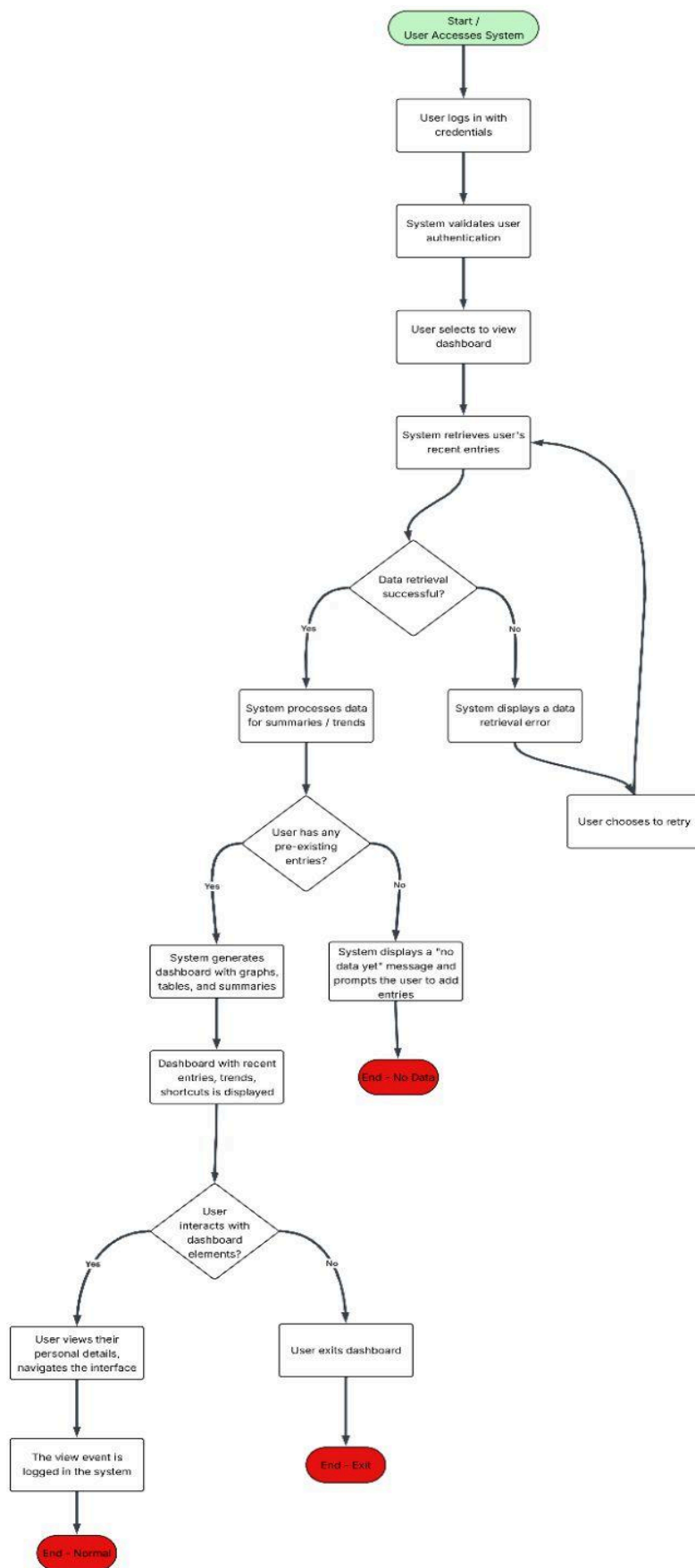
Tracks health-related visits or consultations.

- **Appointment**
 - **Attributes:** appointmentID: UUID, dateTime: DateTime, status: ApptStatus (Enum)
 - **Methods:** schedule(), cancel(), sendReminder()
 - **Relationships:**
 - Appointment ↔ 1 User
 - Appointment ↔ 1 HealthcareProvider
- **Healthcare Provider**
 - **Attributes:** providerID: UUID, name: String, specialty: String
 - **Methods:** reviewPatientData(), provideFeedback()
 - **Structural Logic:**
 - Access is scoped and auditable.

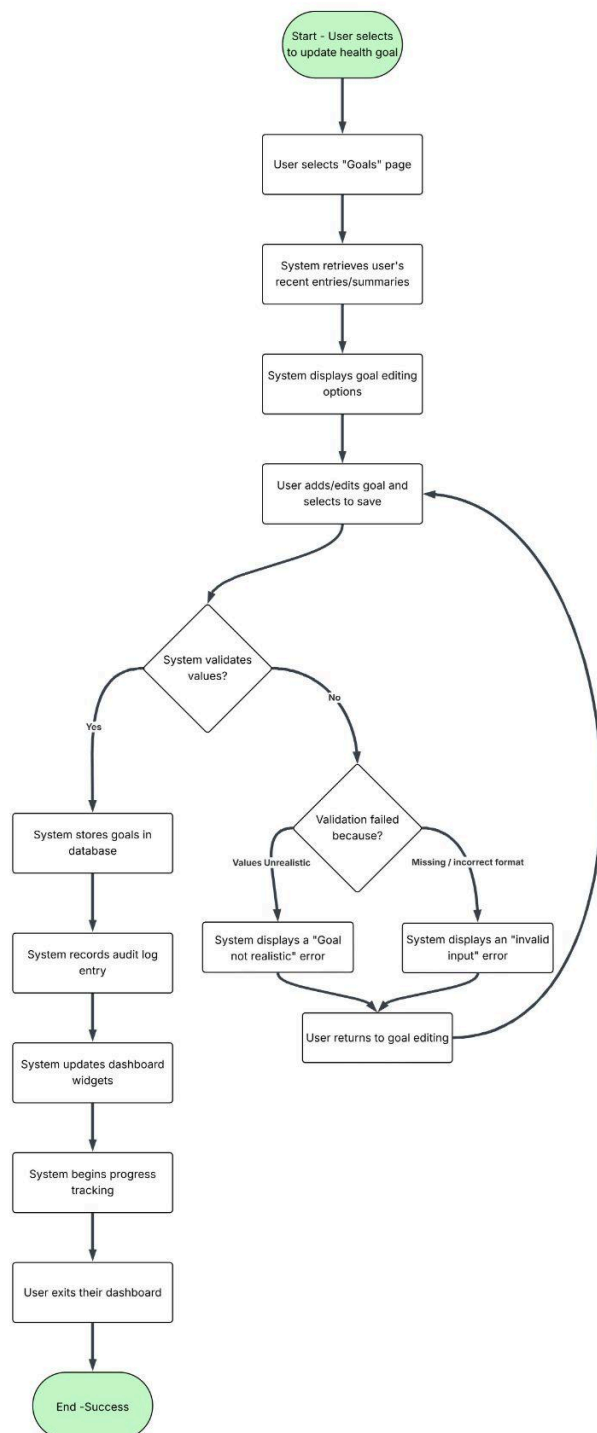
Activity Diagram 1: User Registers Profile/Account (UC1)



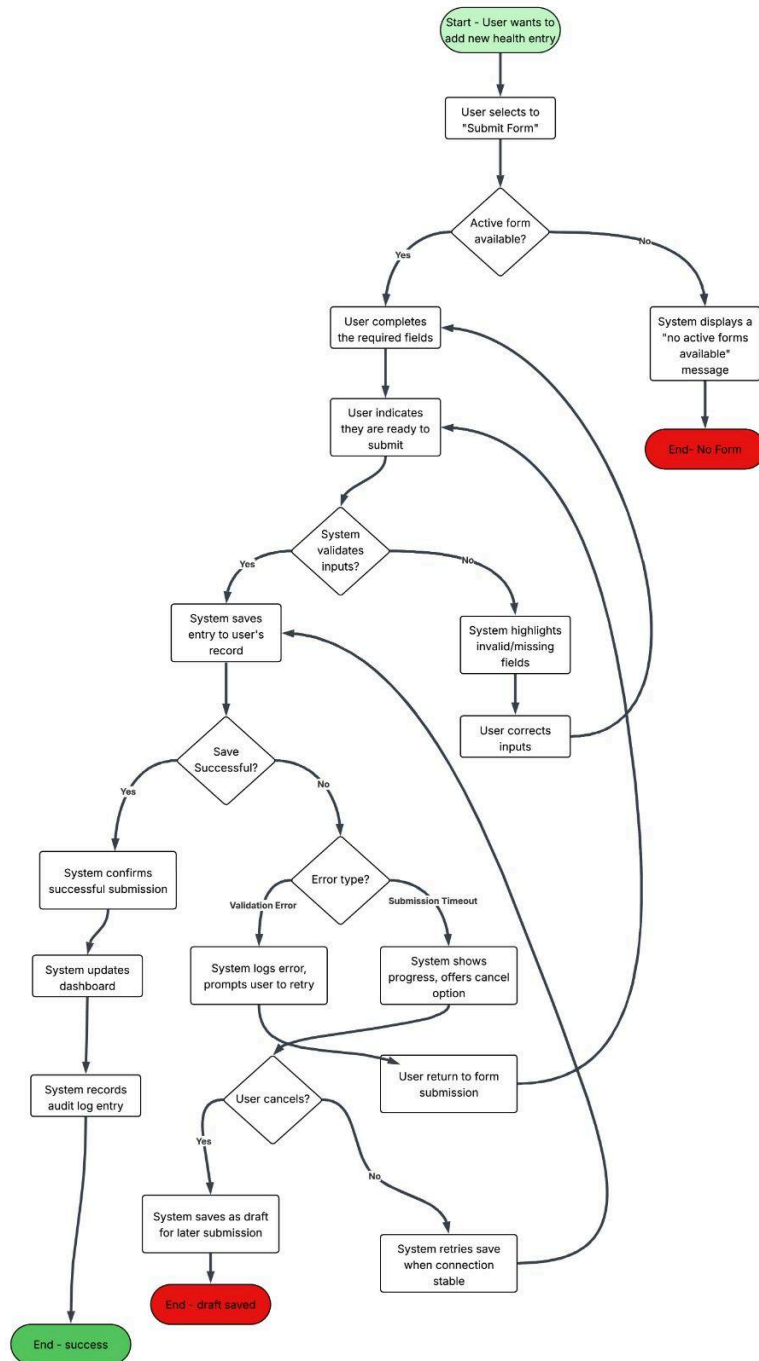
User Views Personal Dashboard



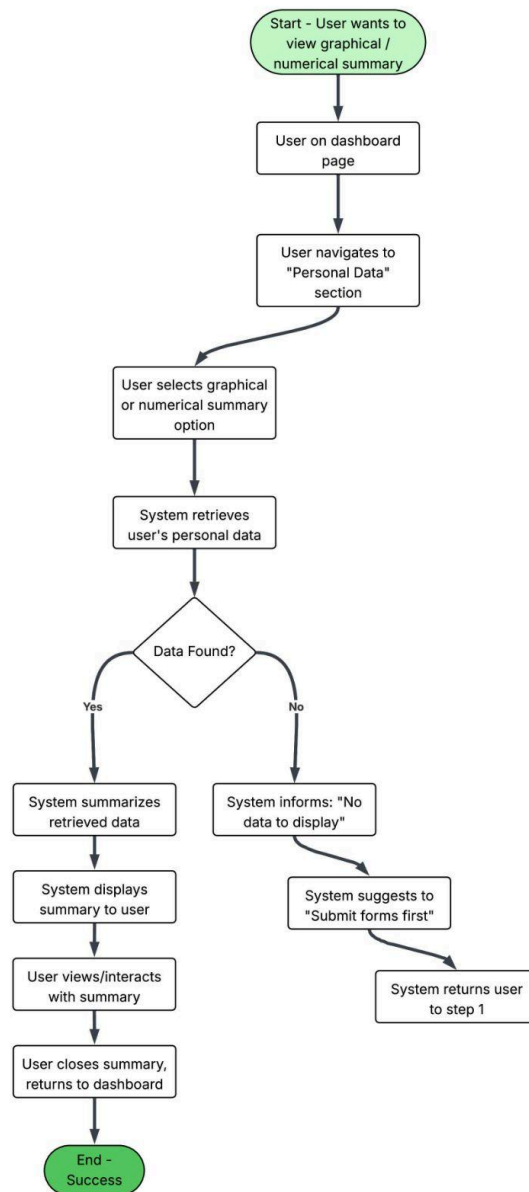
Activity Diagram: User updates Personal Health Goals (UC3)



Activity Diagram: User Completes and Submits Form (UC4)

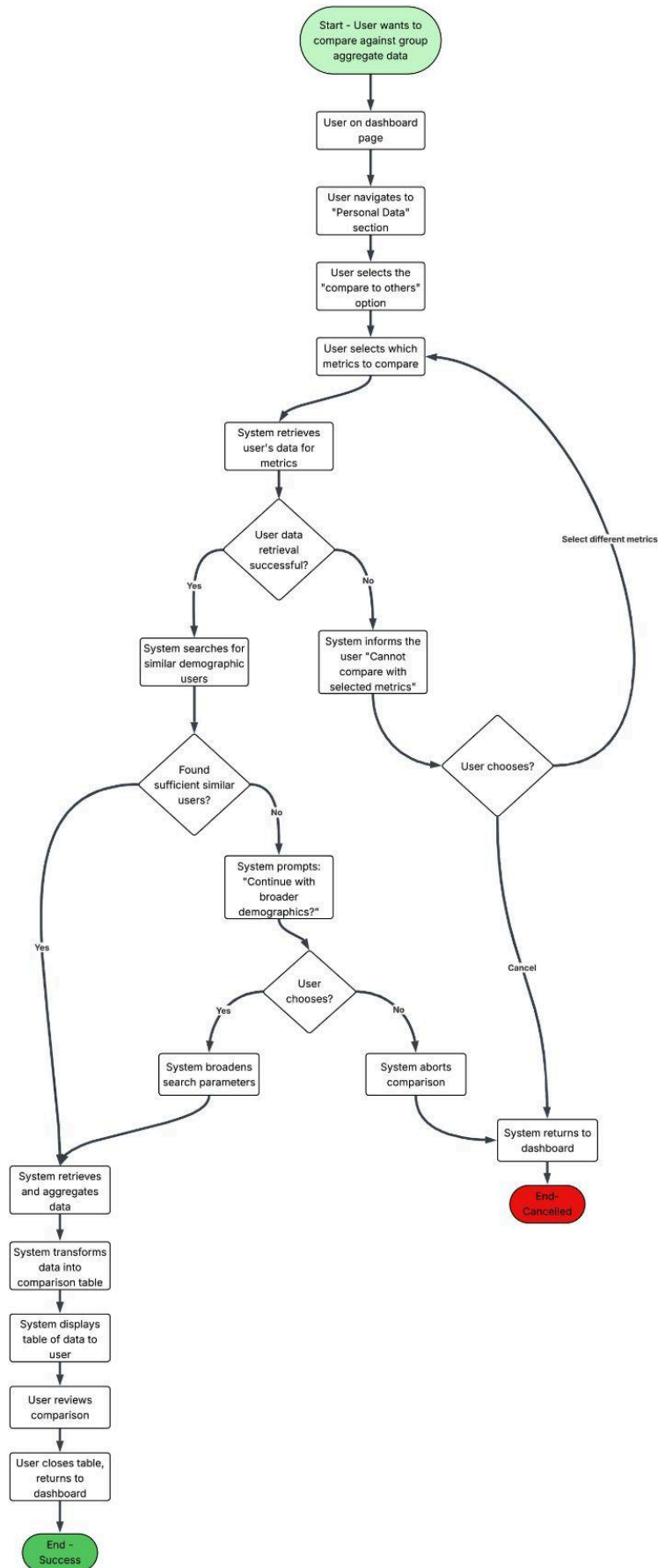


Activity Diagram: User Generates Personal Report (UC4)



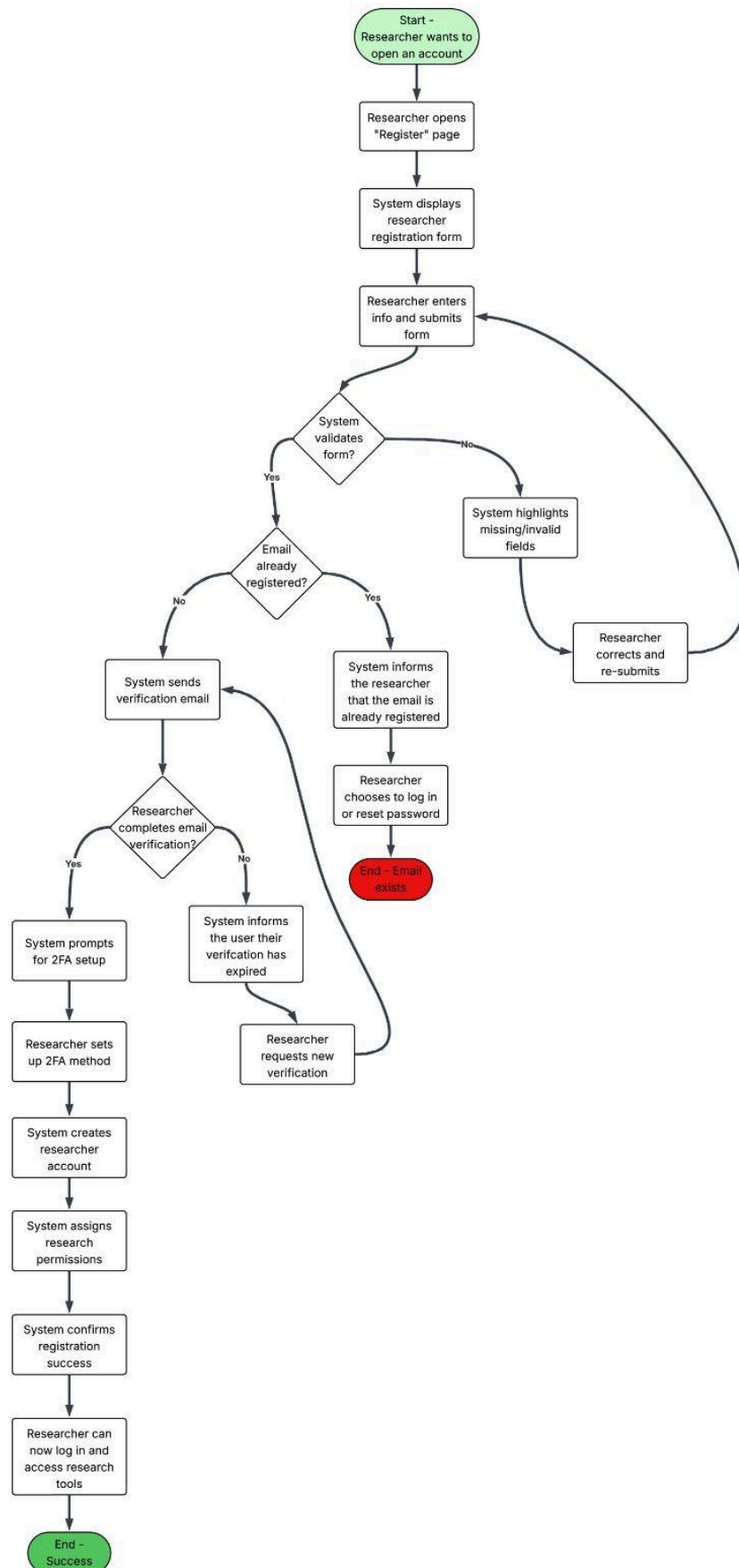
Activity Diagram: User Reviews Personal Data Against Group Aggregate Values (UC6)

102

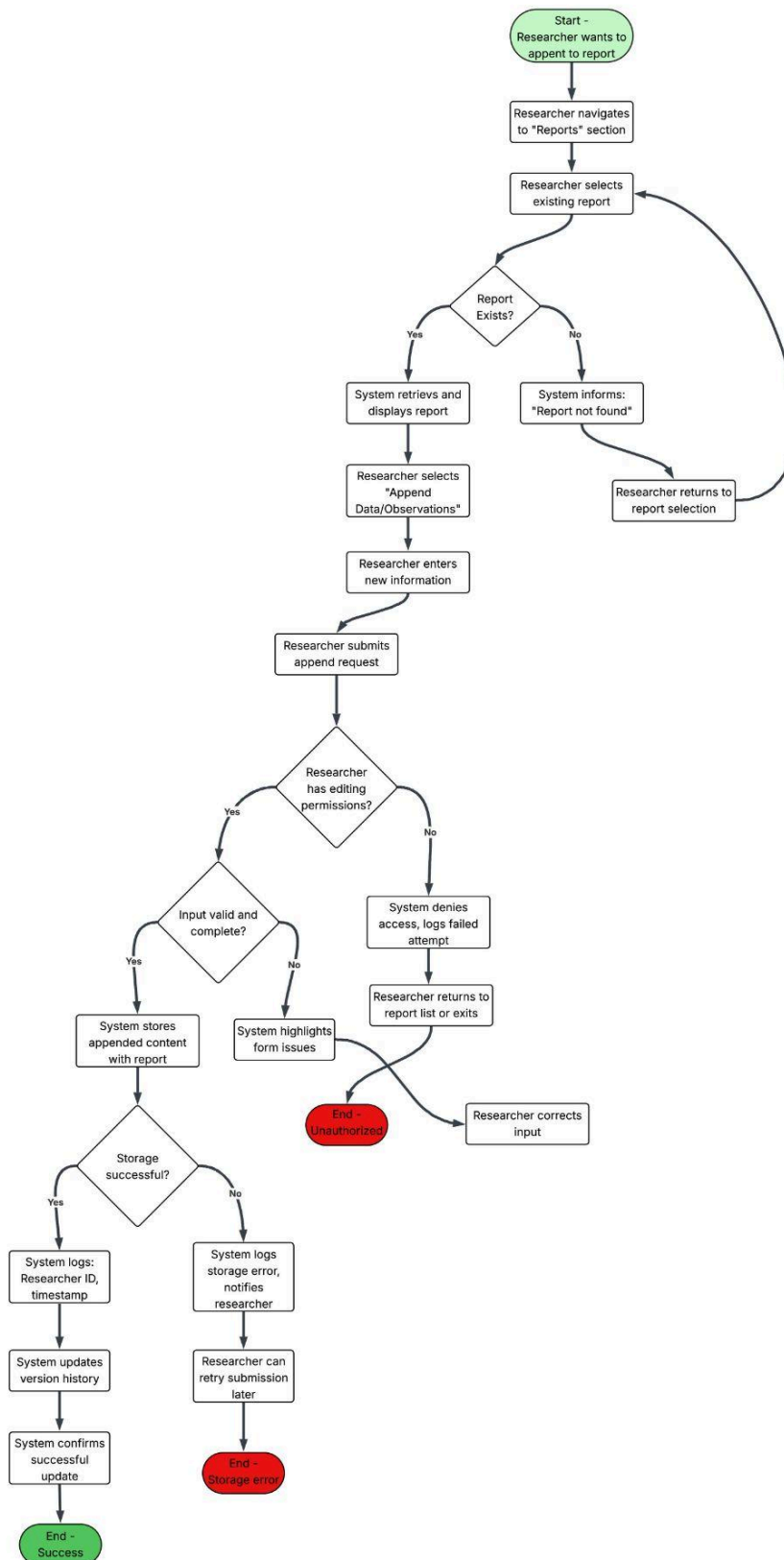


102

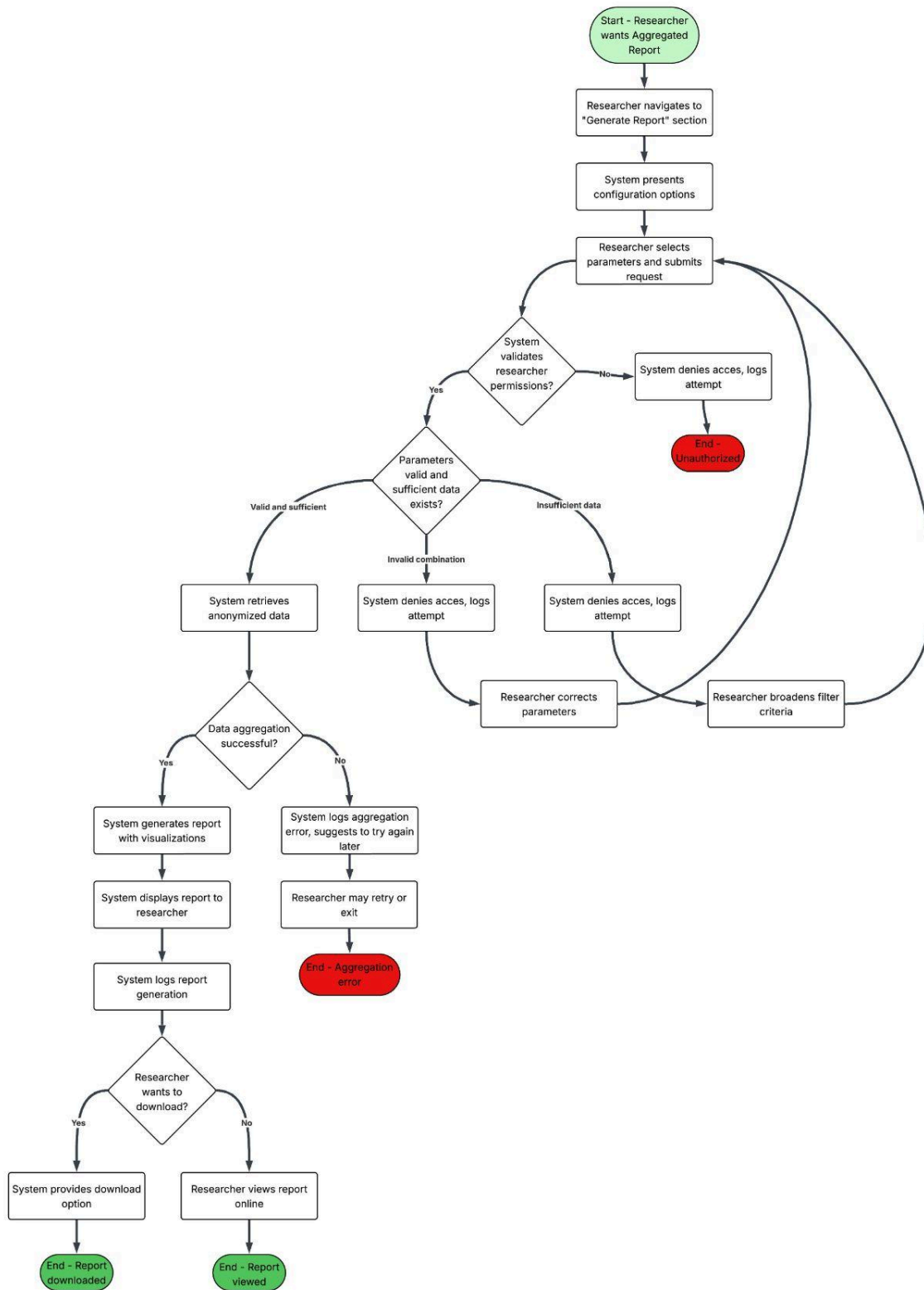
Activity Diagram: Register Researcher Account (UC7)



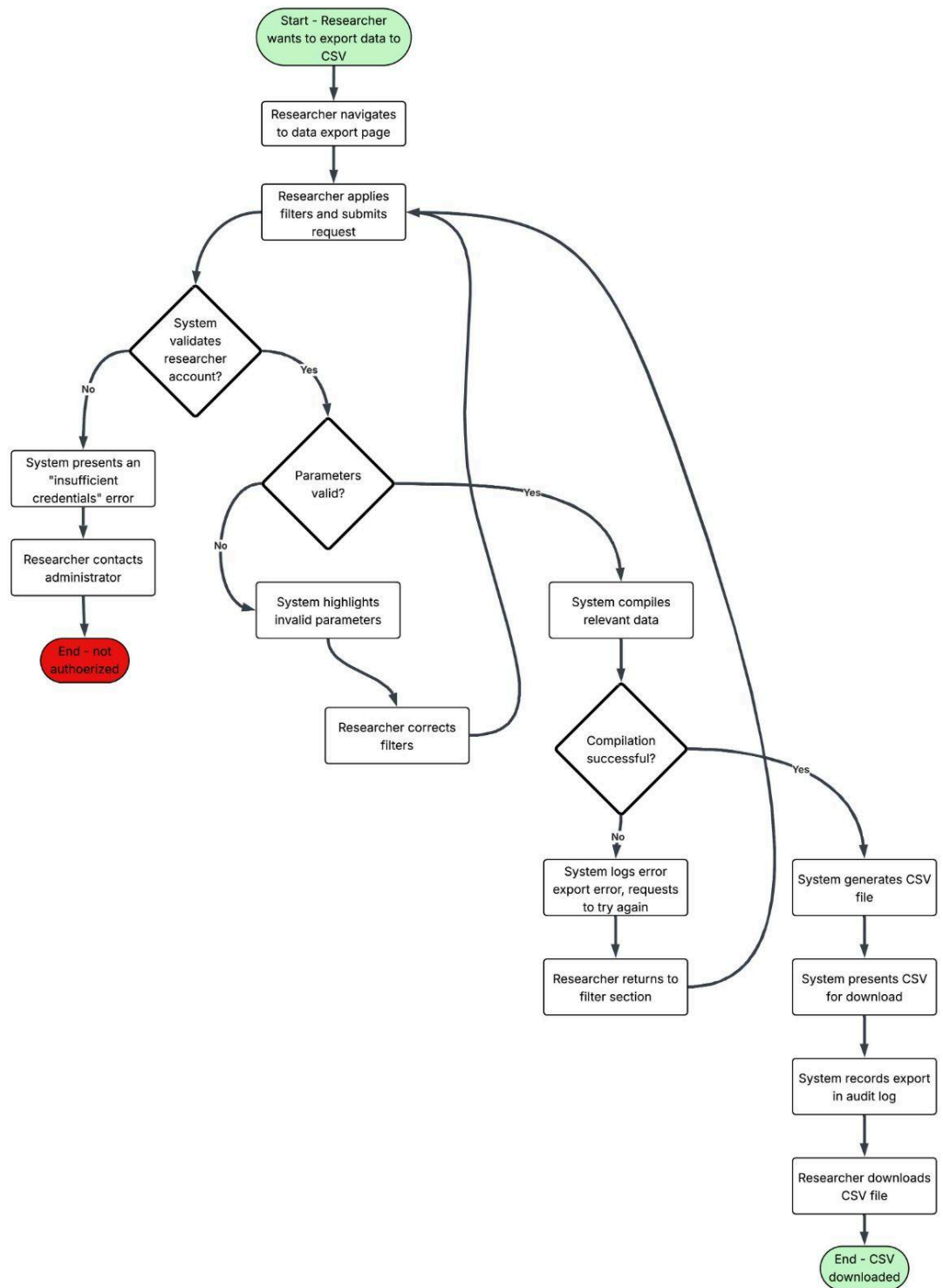
Activity Diagram: Researcher Appends Report (UC8)



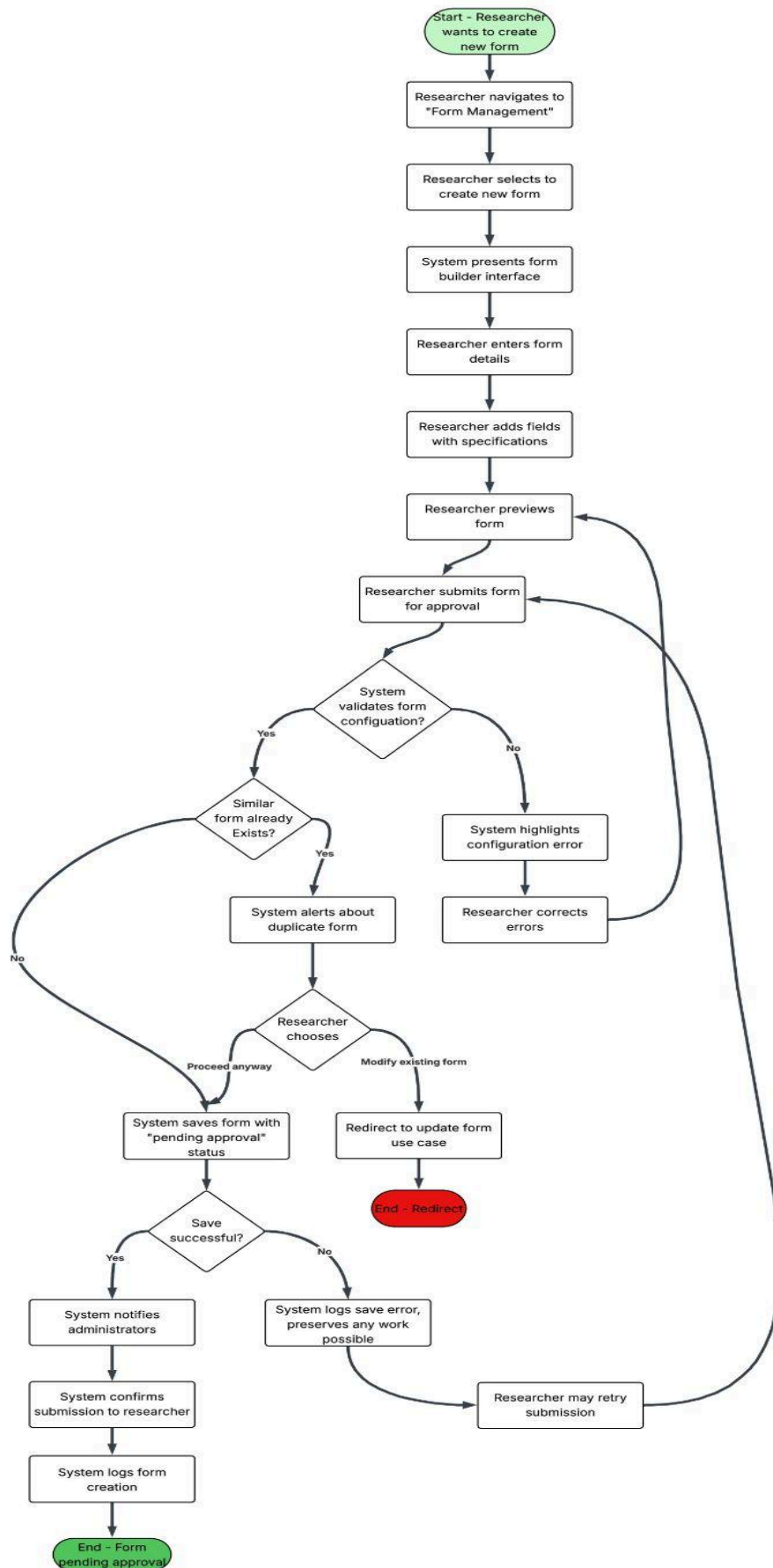
Activity Diagram: Researcher Generates Anonymous Aggregated Report (UC9)

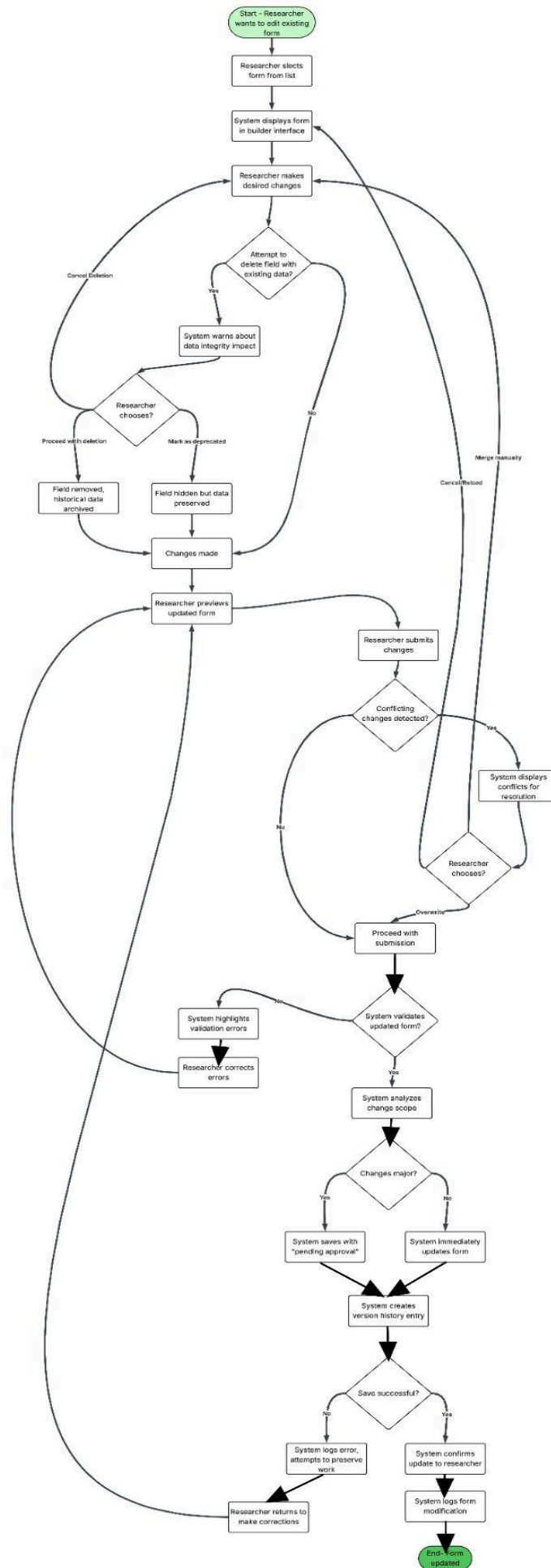


Activity Diagram: Researcher Exports Data to CSV (UC10)

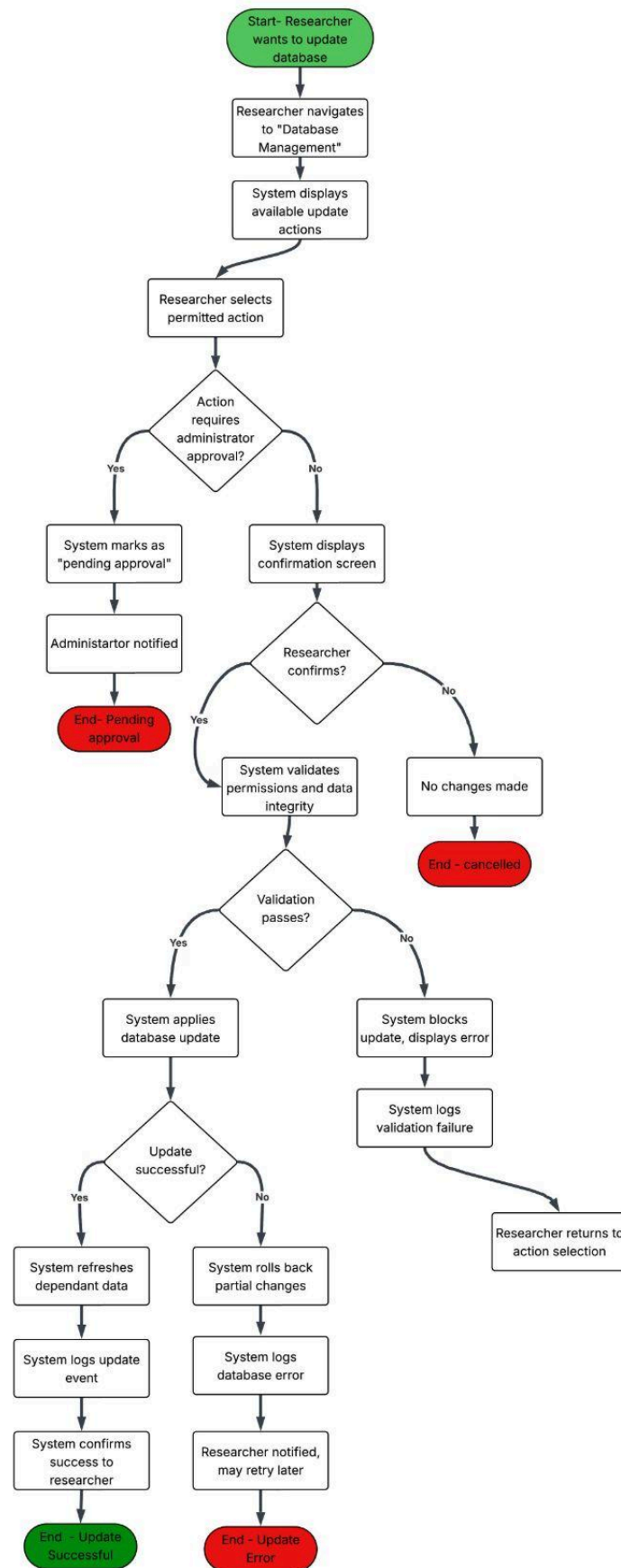


Activity Diagram: Researcher Adds New Form (UC11)

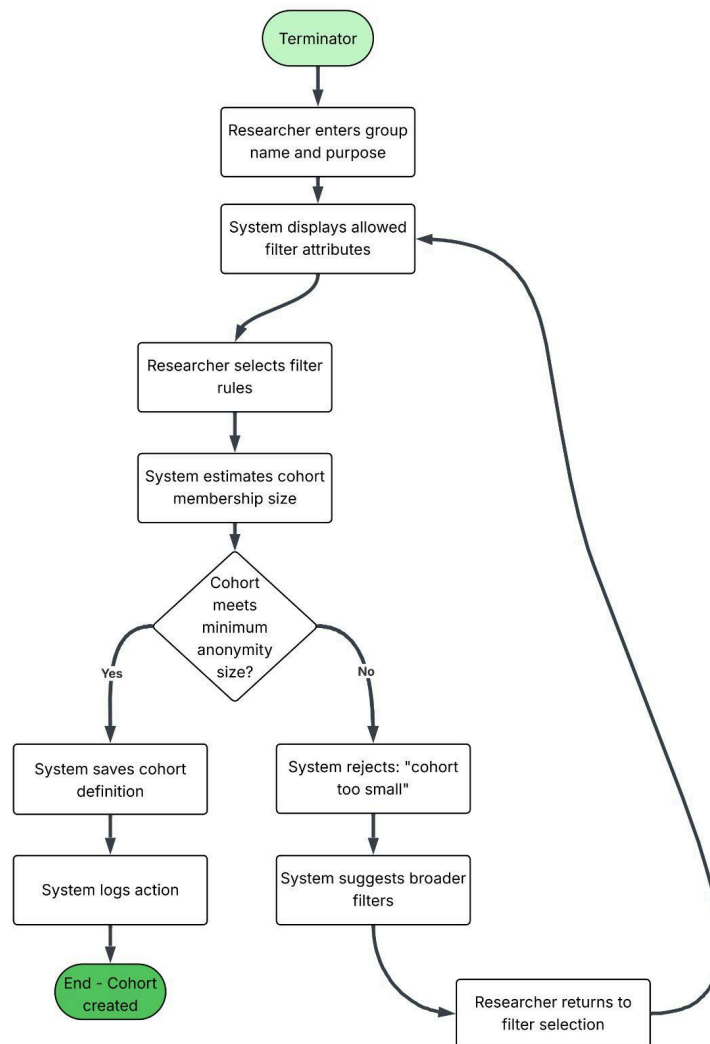




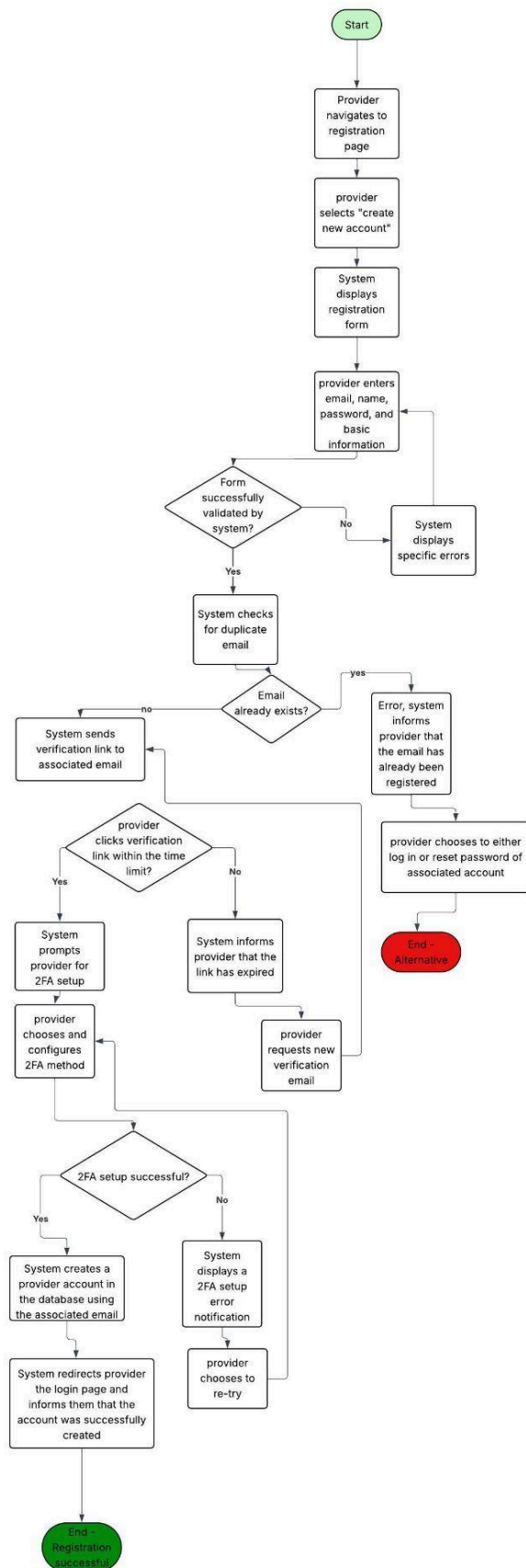
Activity Diagram: Researcher Updates Database (UC13)



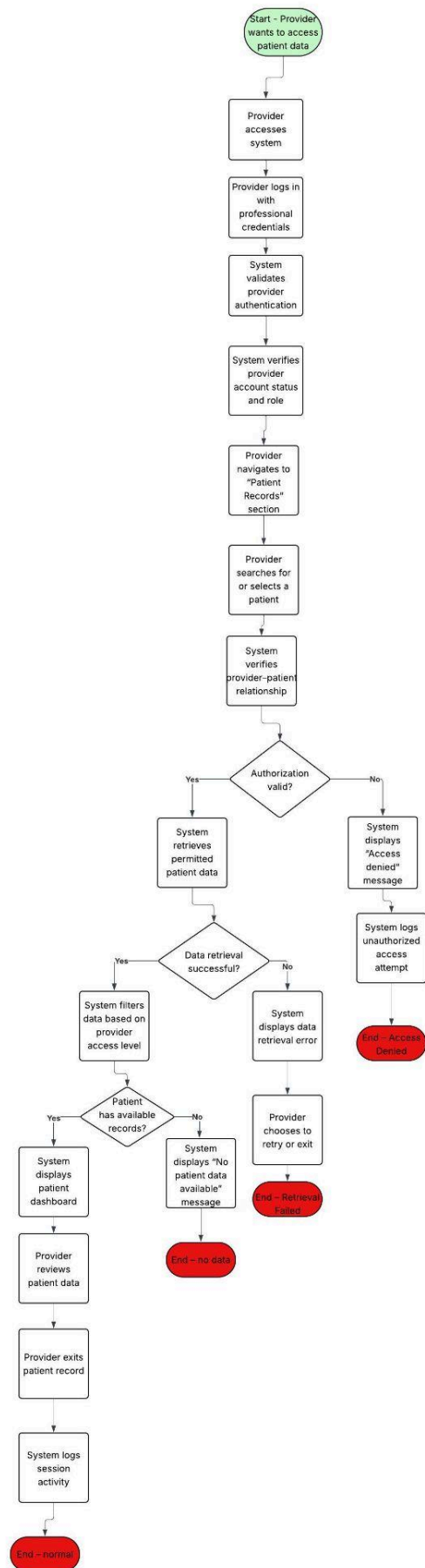
Activity Diagram: Researcher Creates Cohort (Group) (UC14)

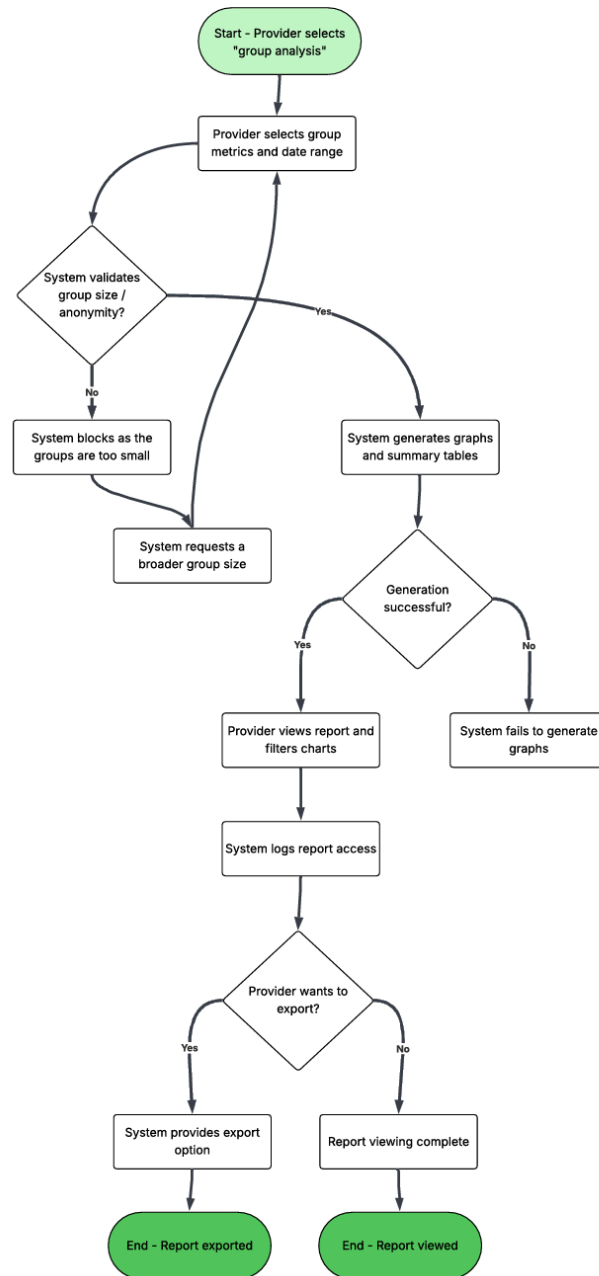


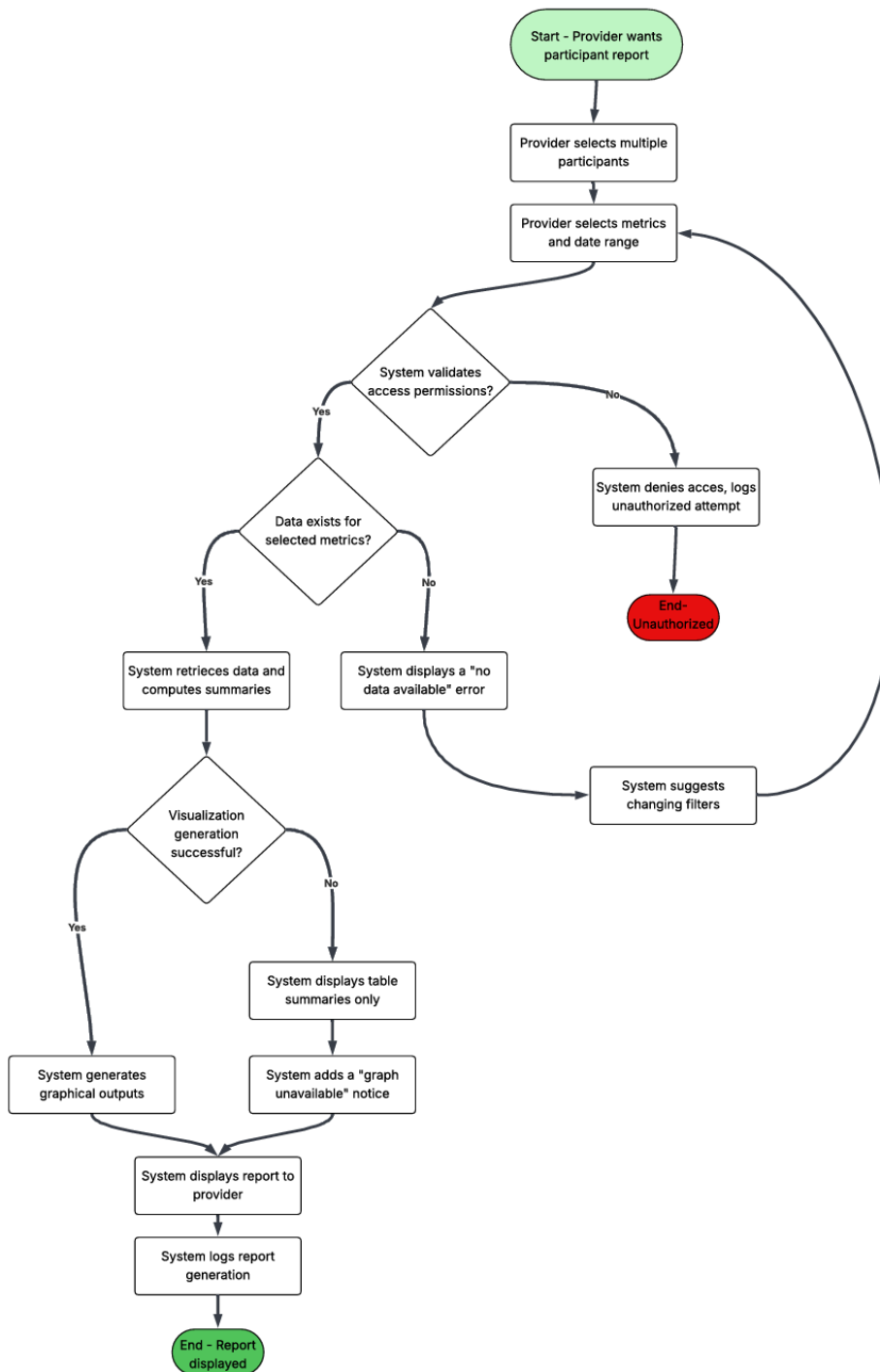
Provider Registers Account



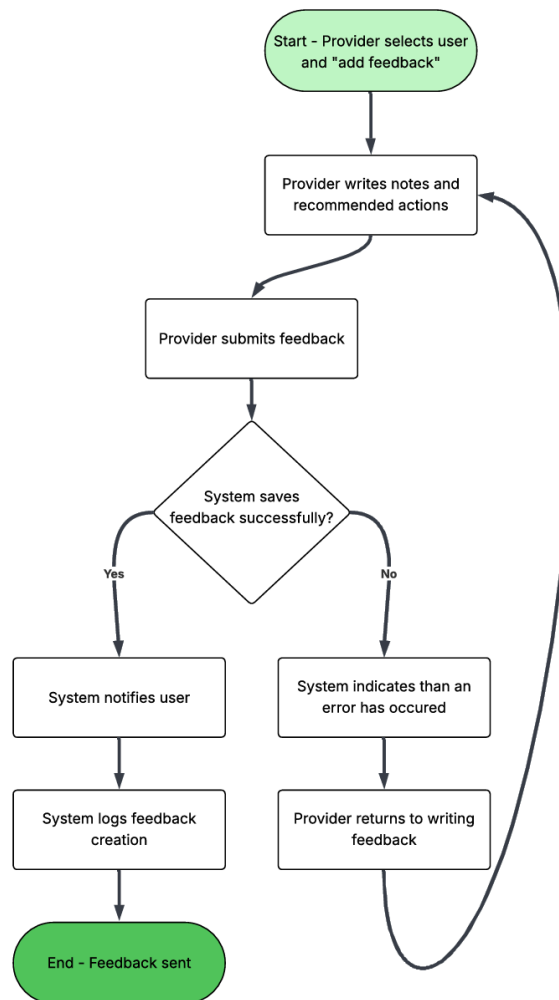
Provider Accesses Patient Data



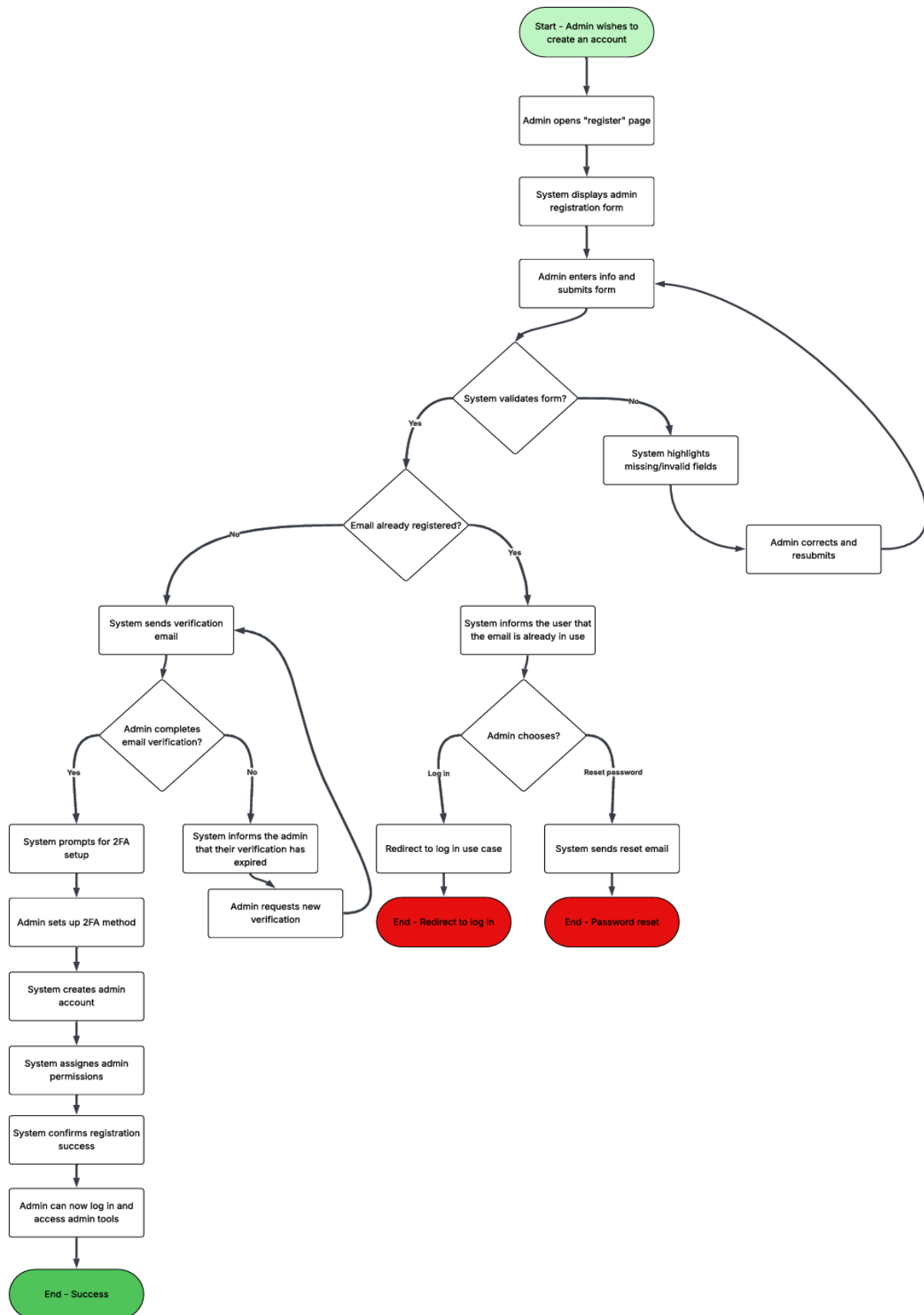
Activity Diagram: Healthcare Provider Views Graphical Report for Groups (UC17)

Activity Diagram: Healthcare Provider Views Graphical Report for Participants (UC18)

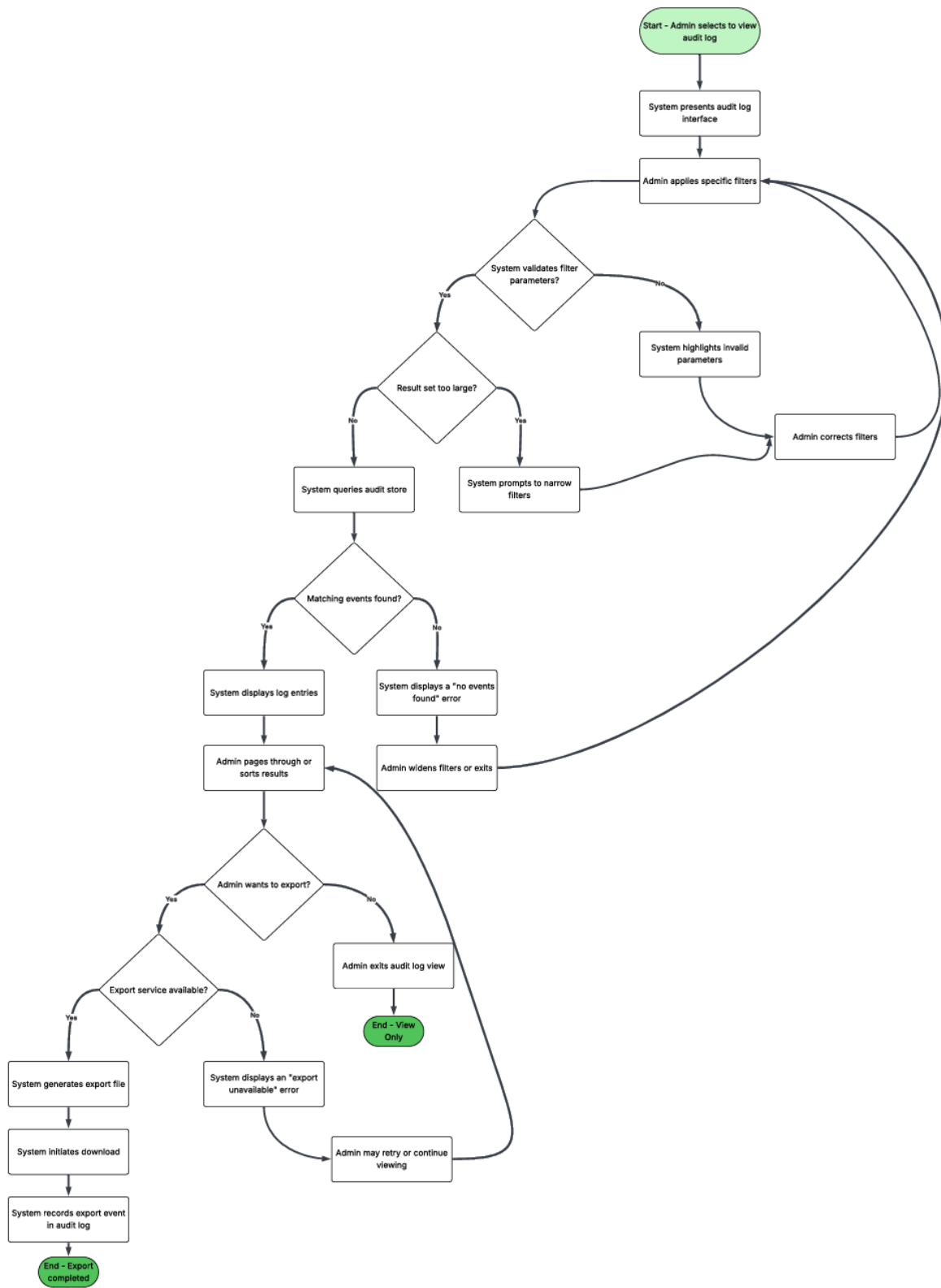
Activity Diagram - Healthcare Provider Provides Personalized Feedback (UC19)



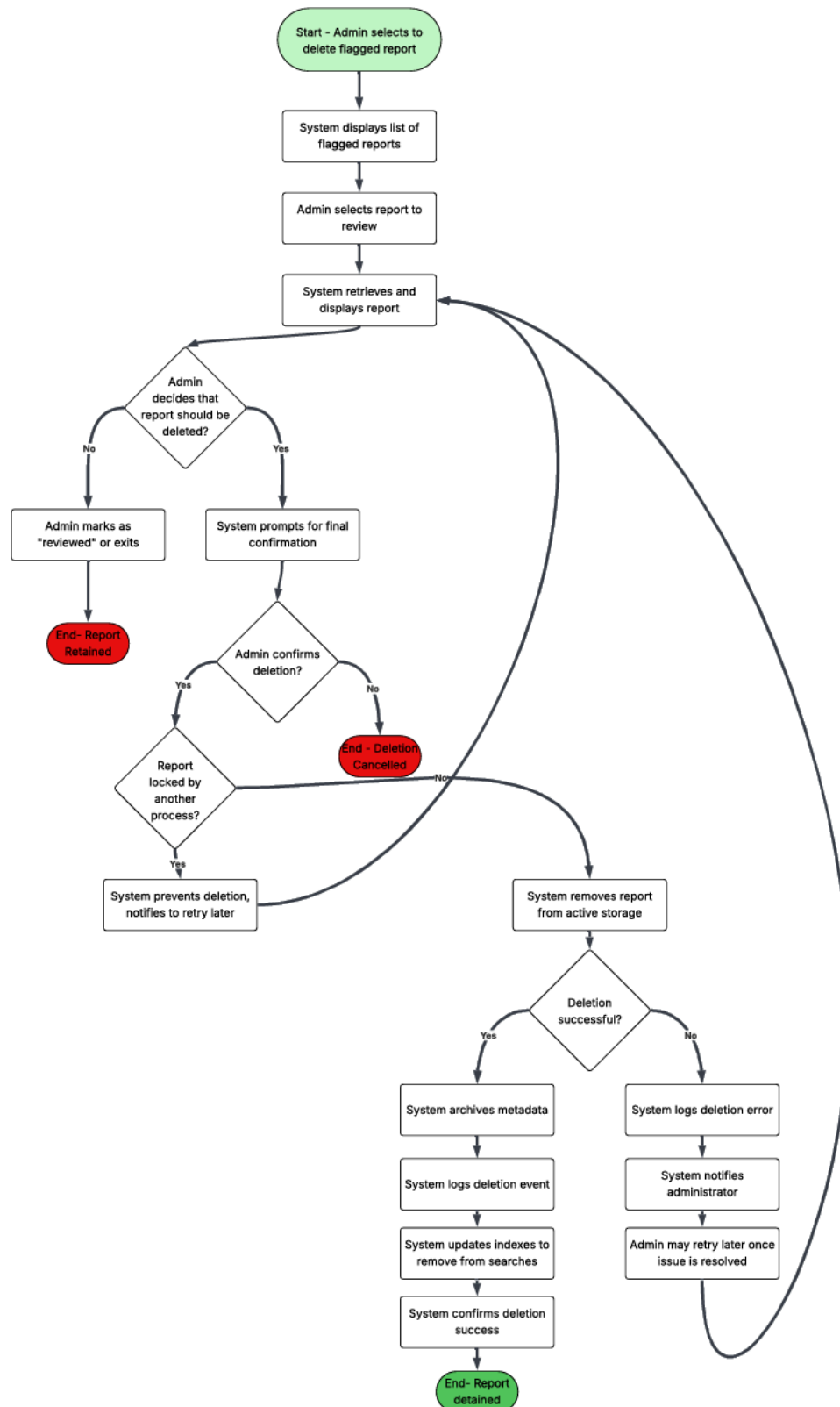
Activity Diagram - Register Admin Account (UC20)



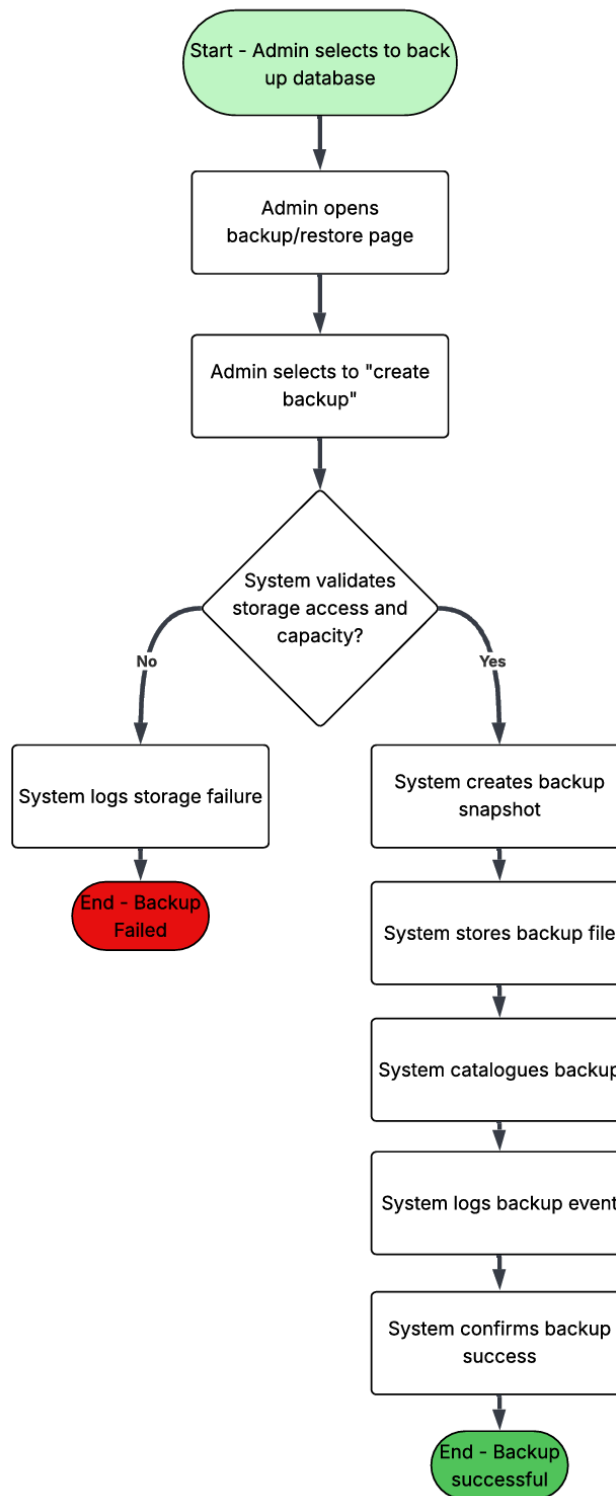
Activity Diagram - Admin Views Audit Log (UC21)



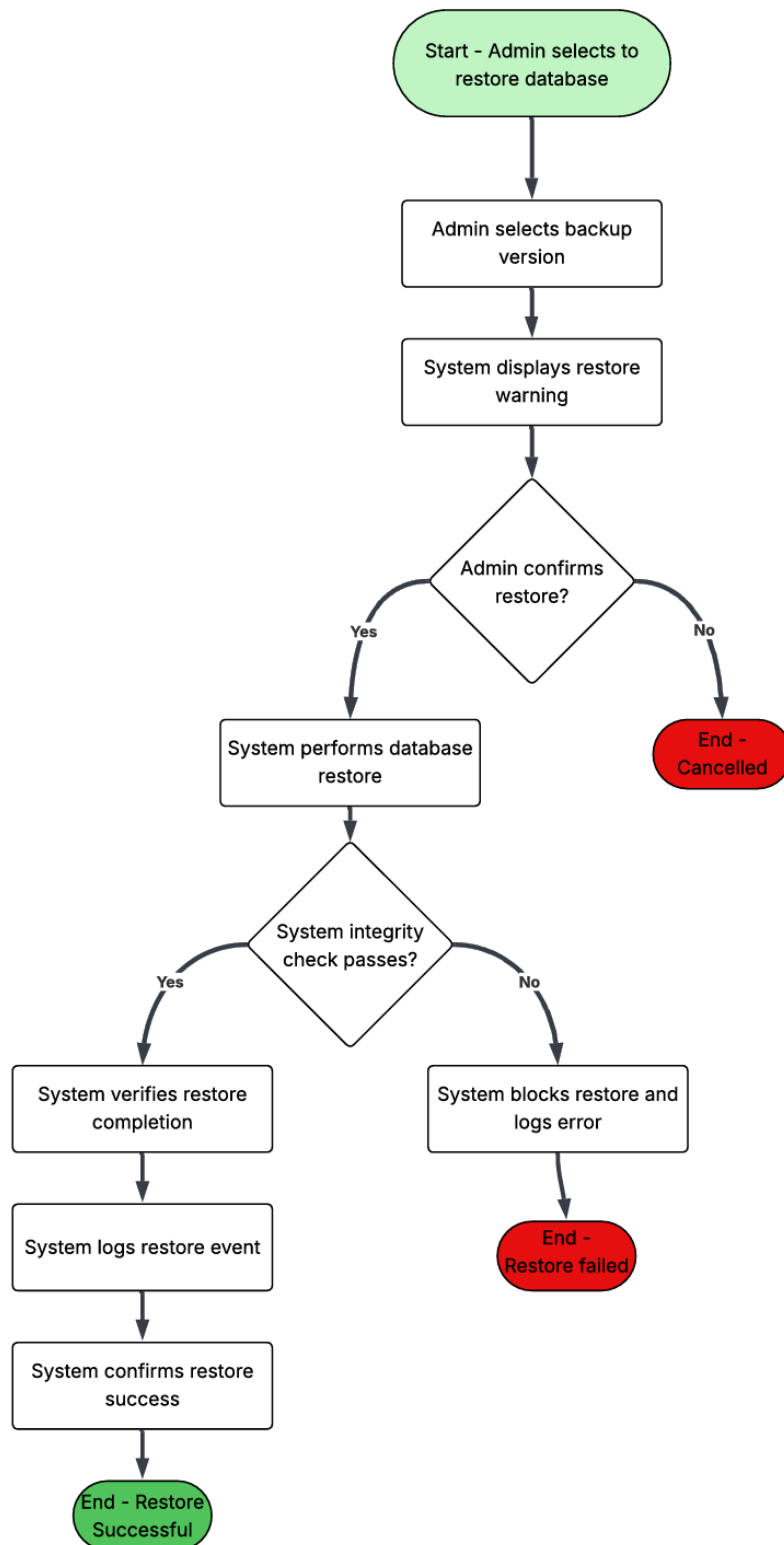
Activity Diagram - Admin Deletes Problematic Report



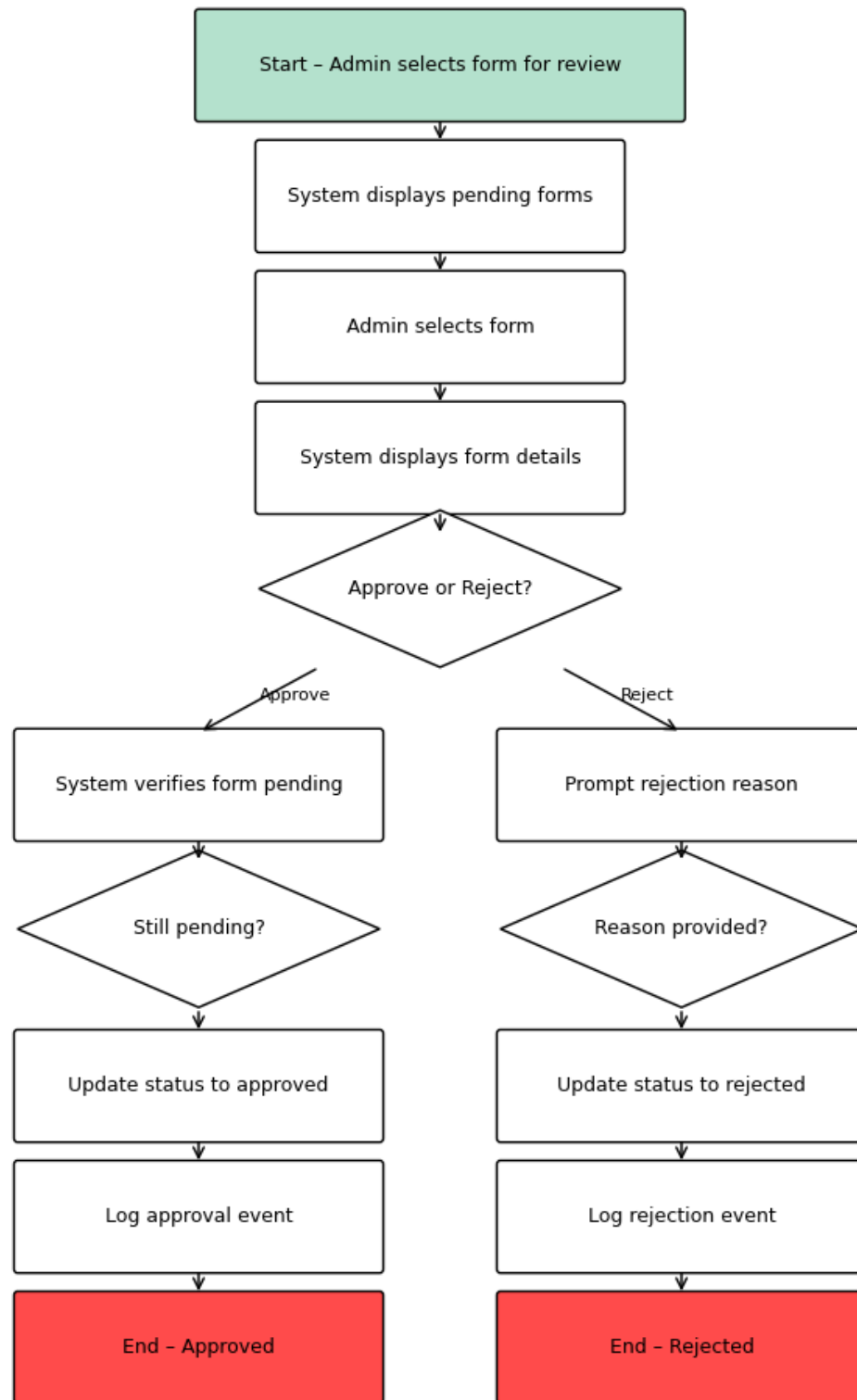
Activity Diagram - Admin Backs up Database (UC22)



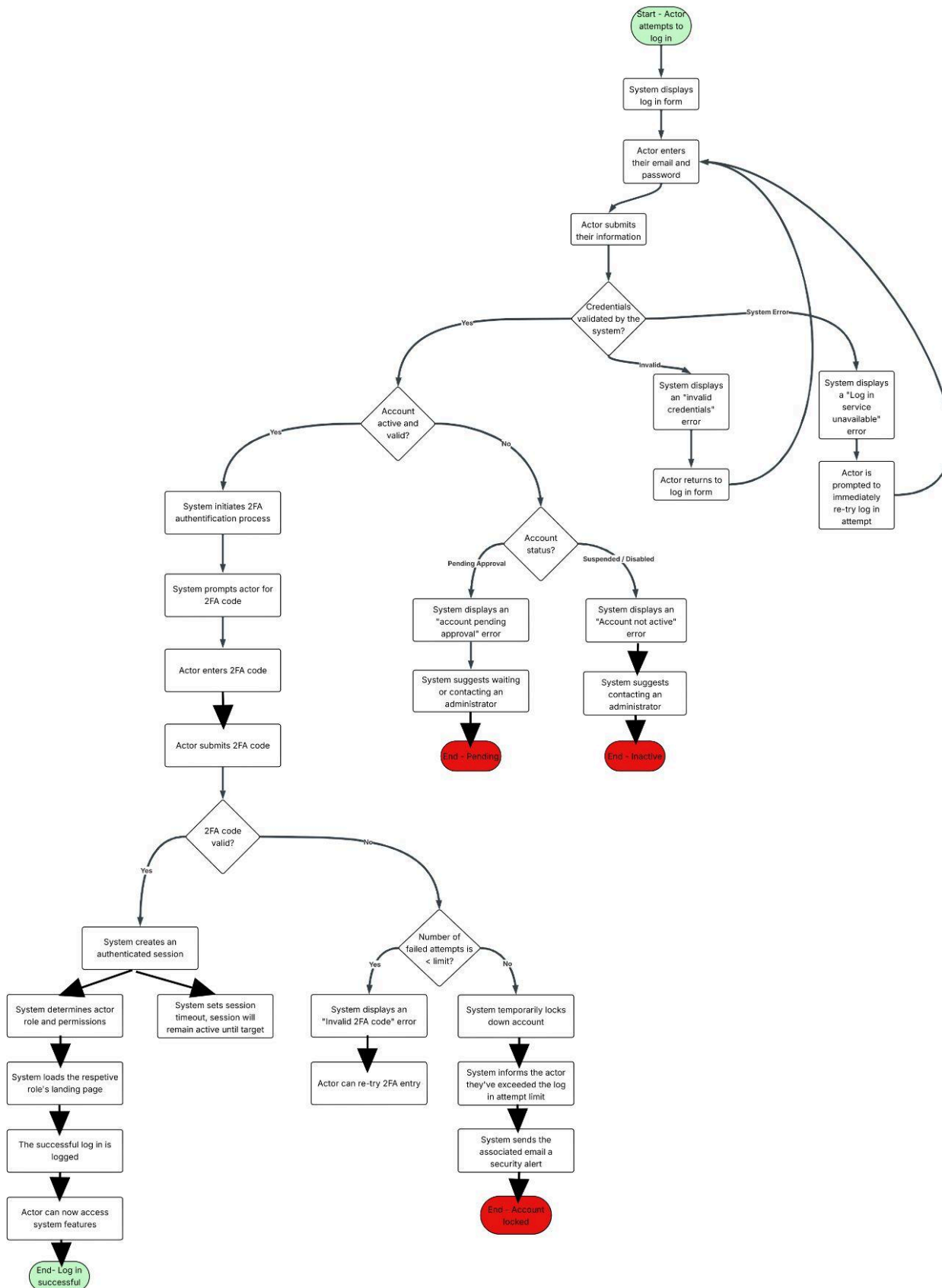
Activity Diagram - Admin Restores Database (UC23)



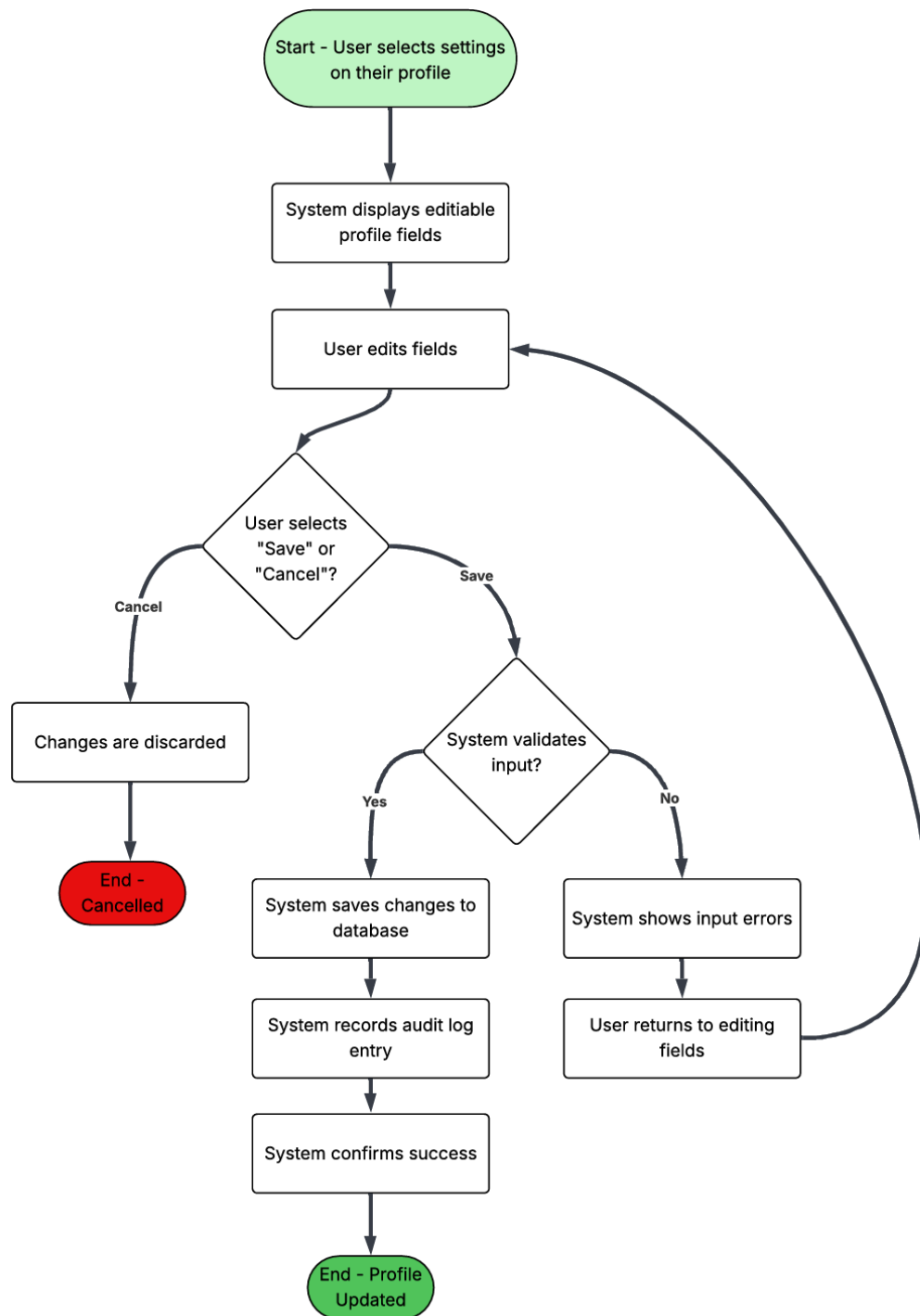
Admin Approve/Reject From



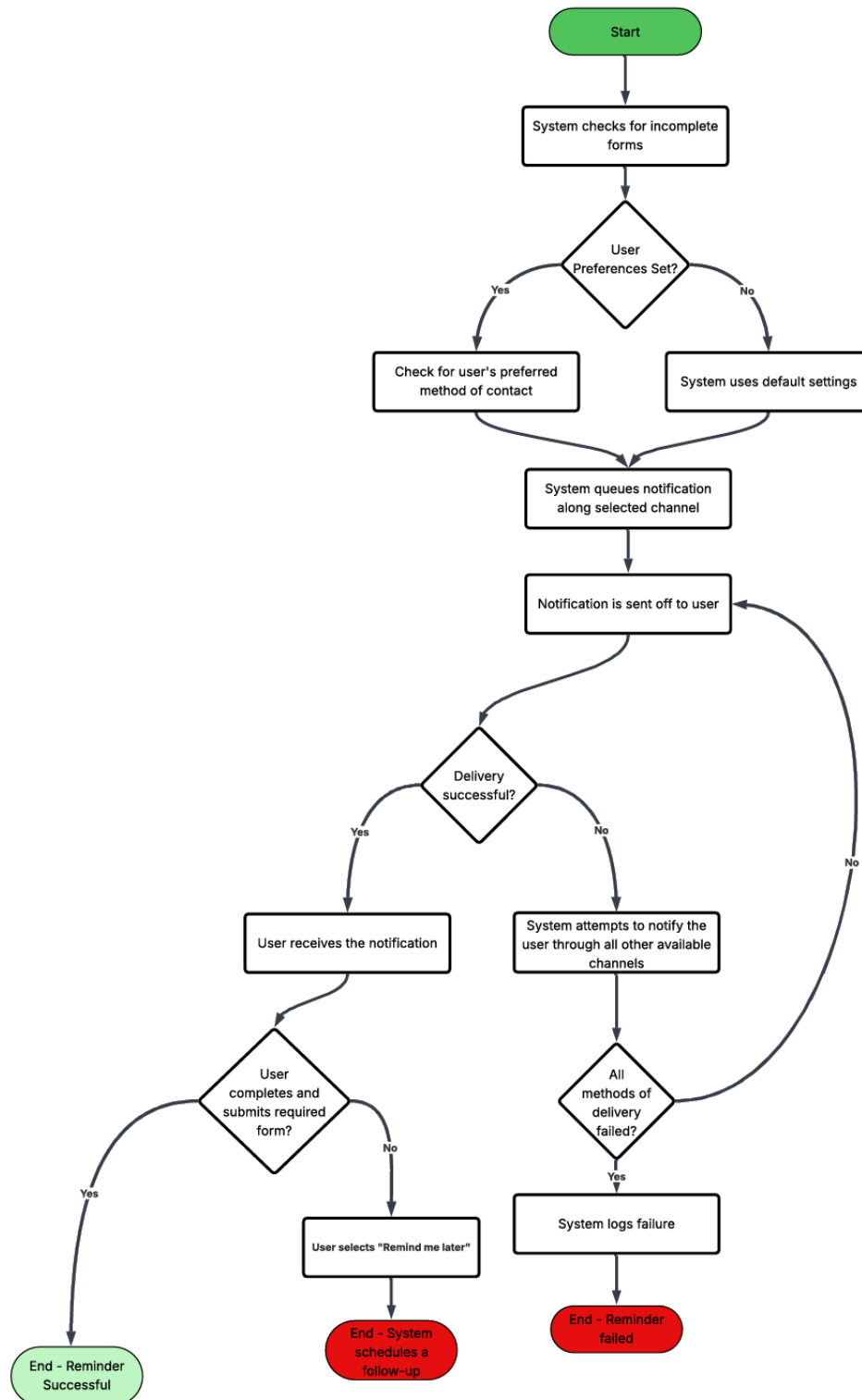
Actor Attempts to Log In



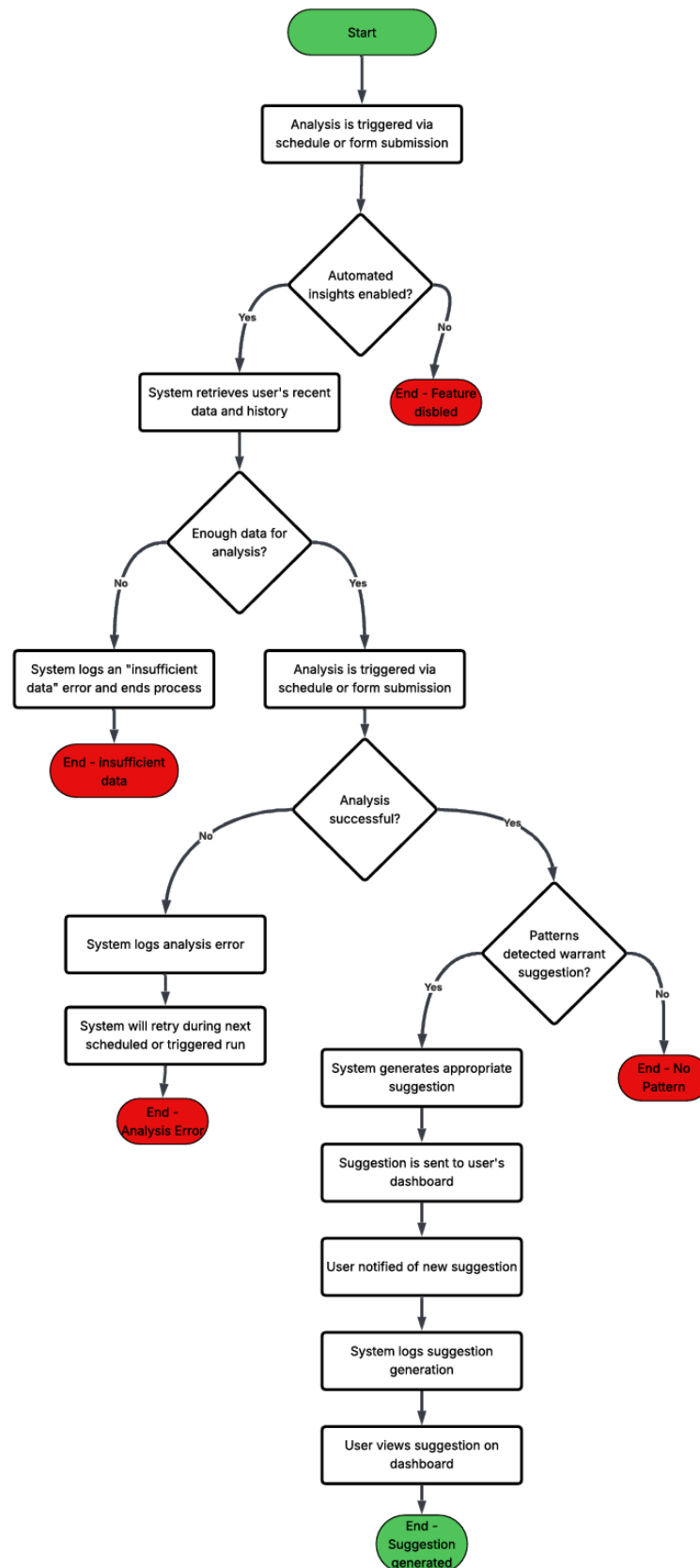
Activity Diagram - Update Account Information (UC25)



Activity Diagram: Reminders for Data Entry (Stretch Goal 1)

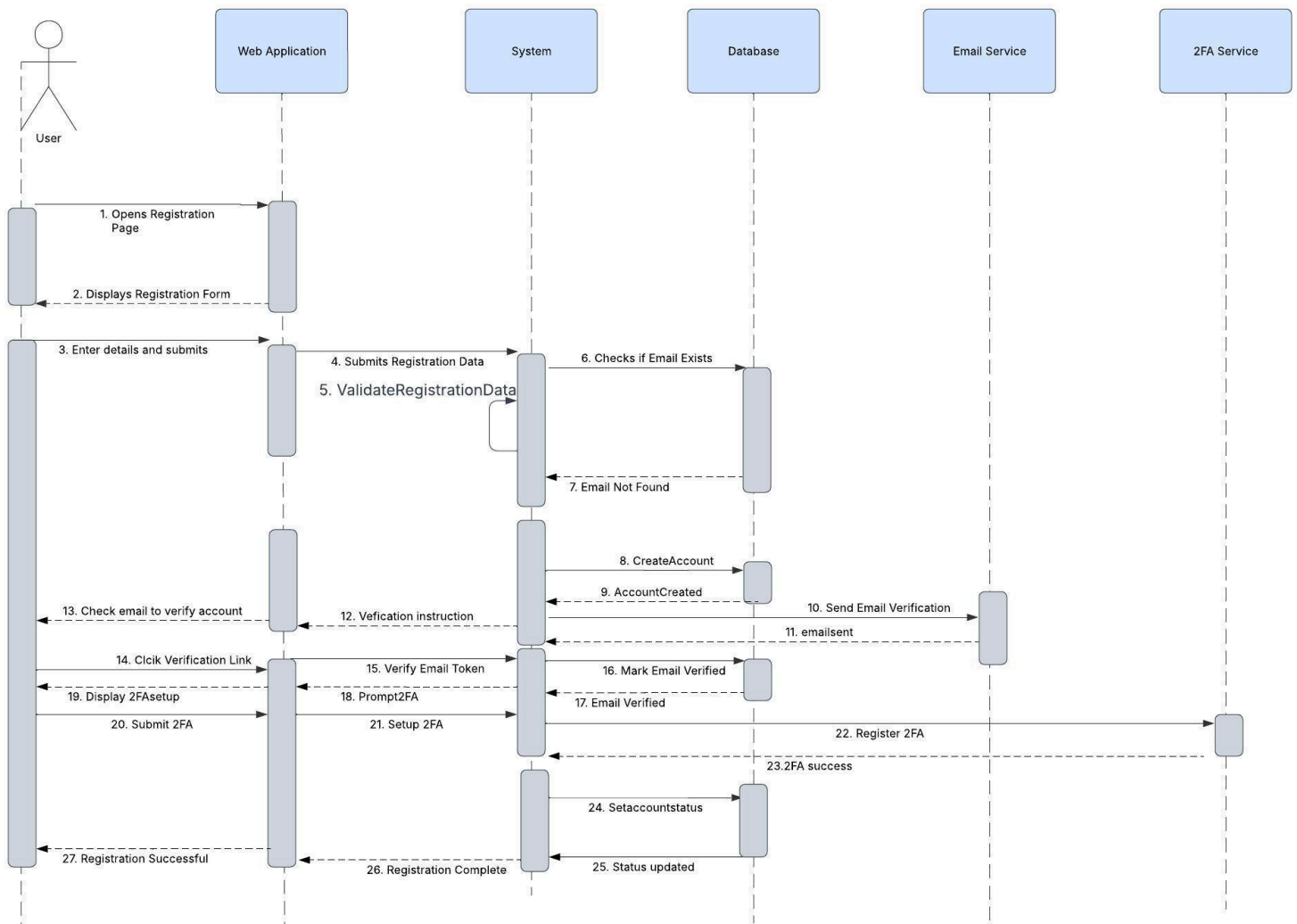


Activity Diagram: System-Generated Suggestions (Stretch Goal 4)

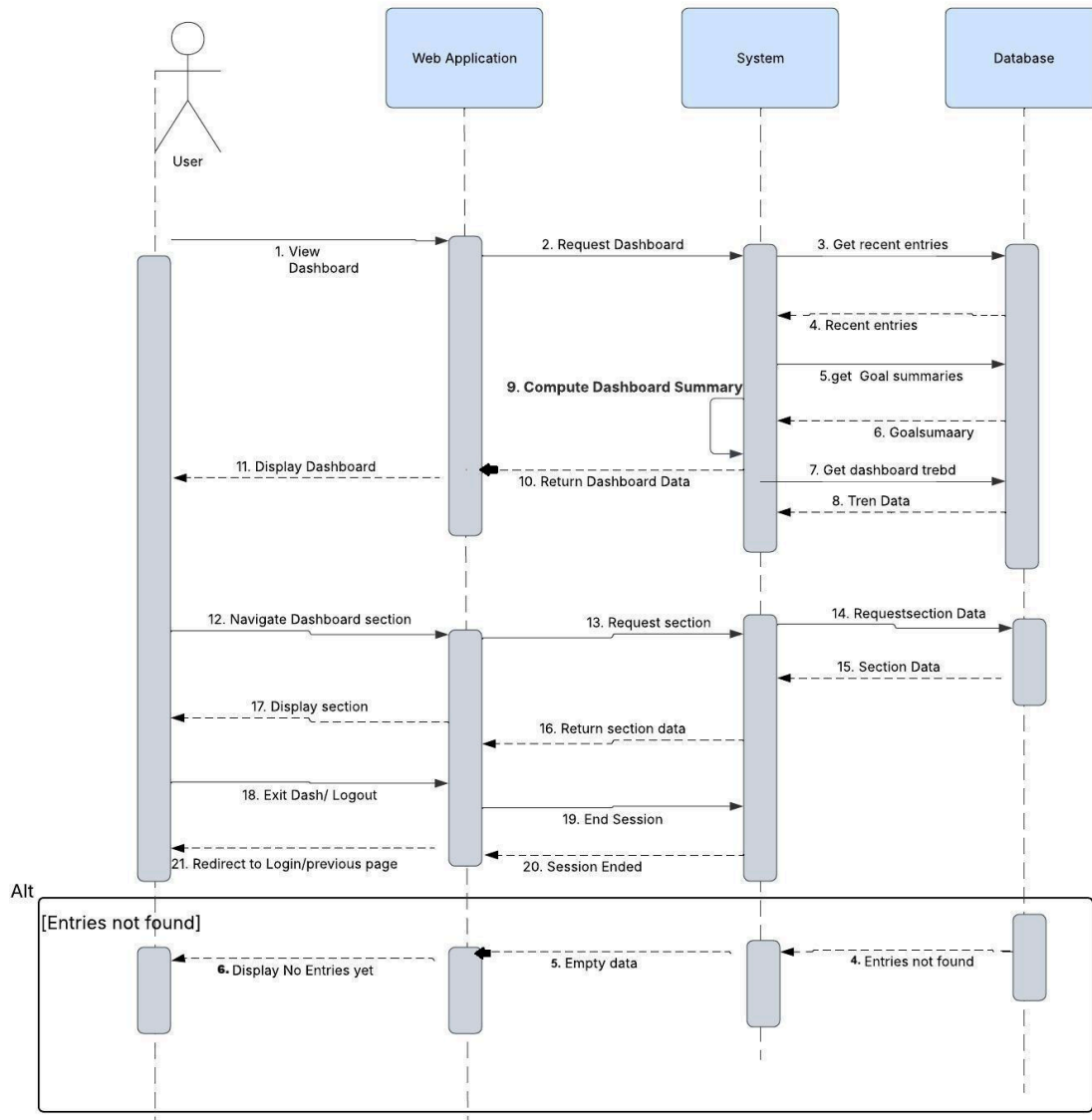


Sequence Diagrams

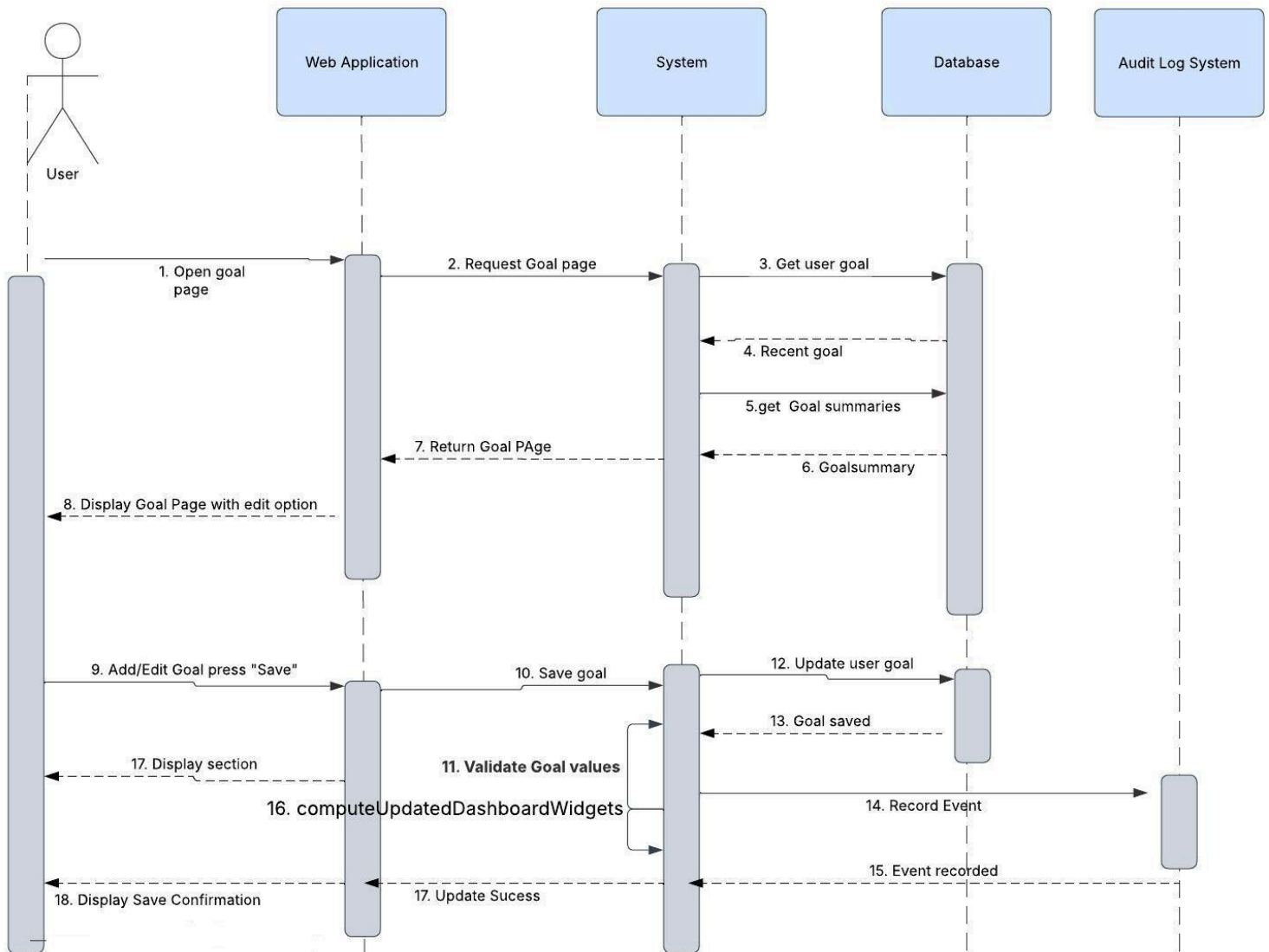
User Registers Account



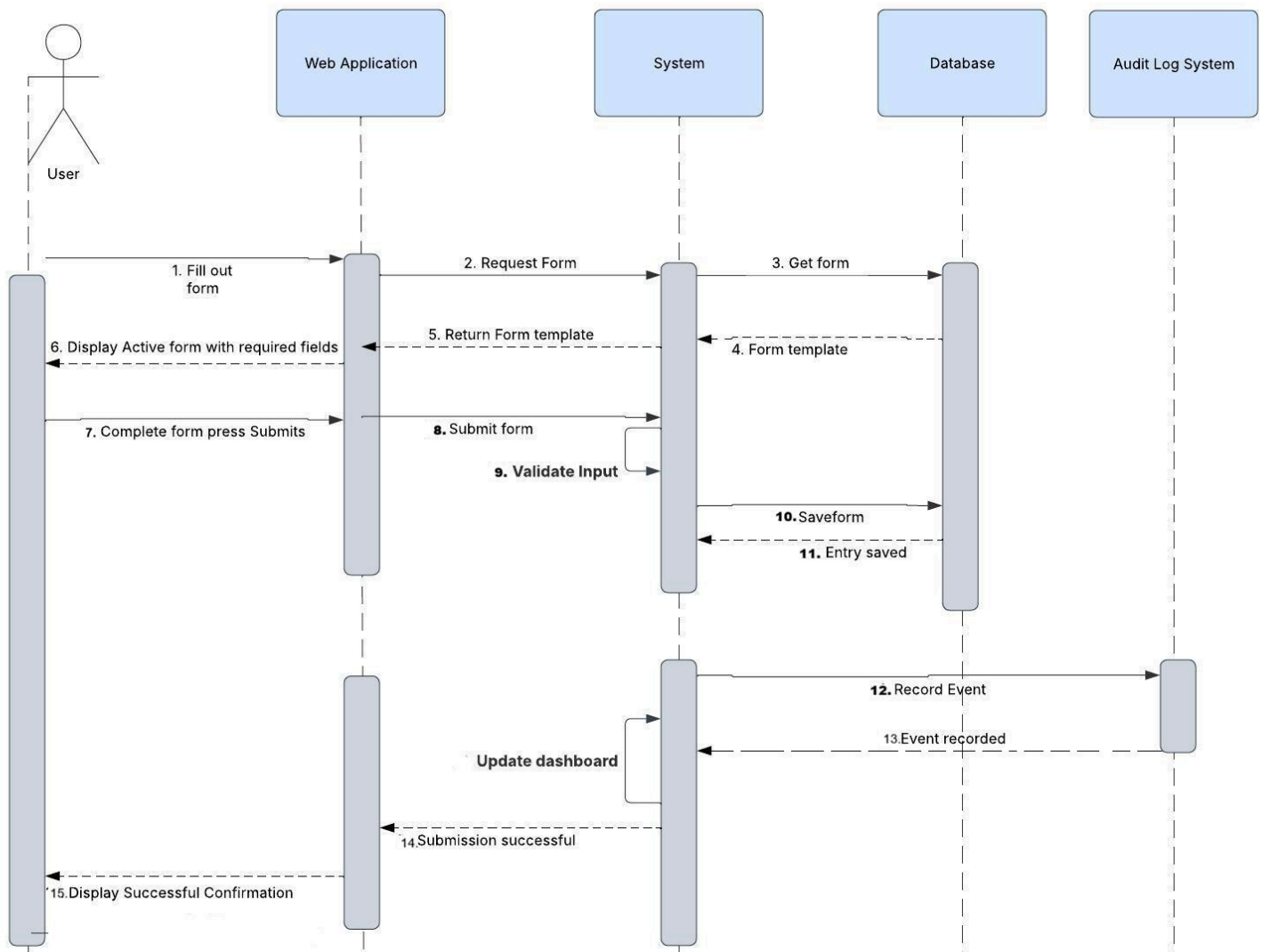
User Views Personal Dashboard



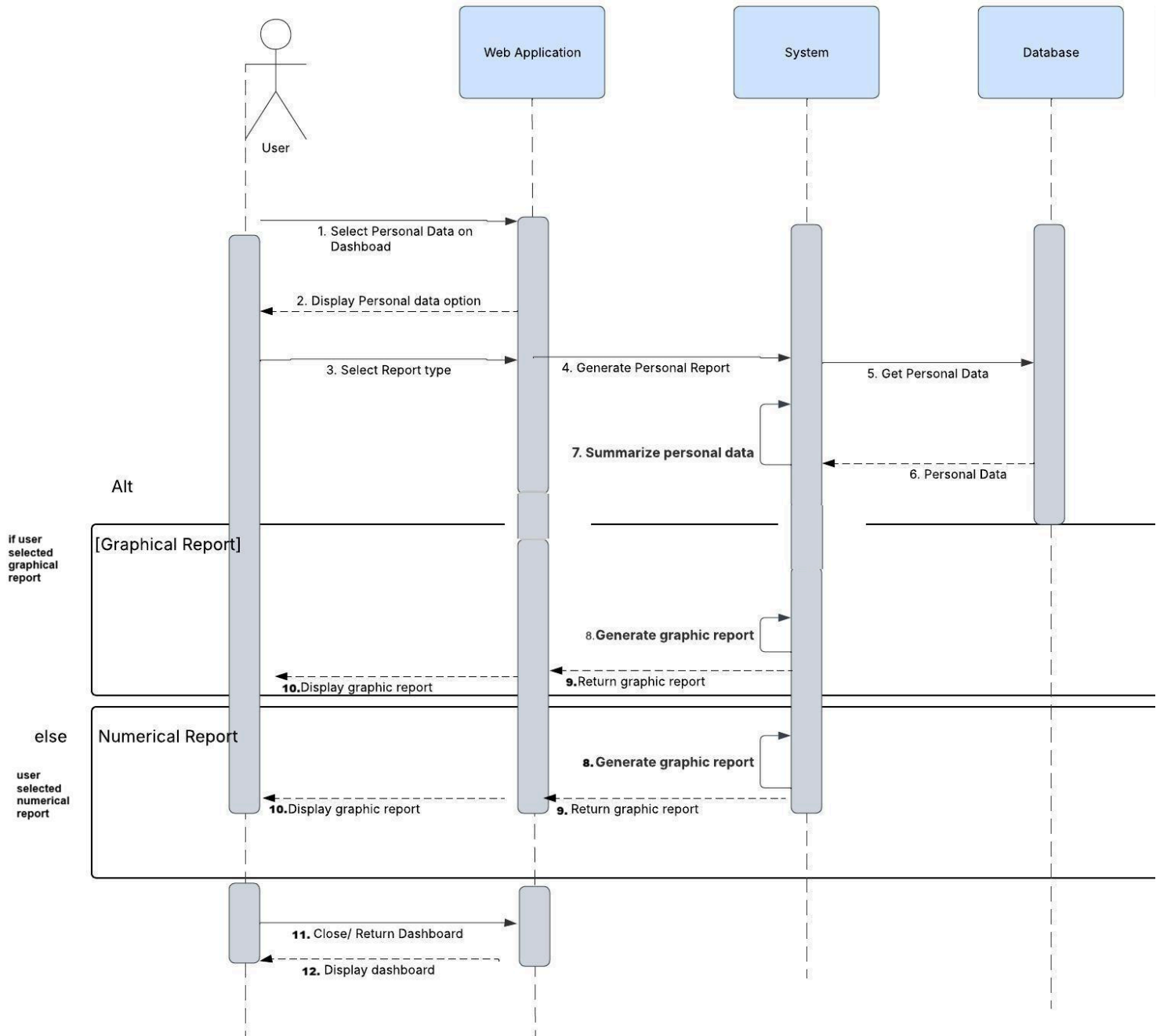
User Updates Personal Health Goals



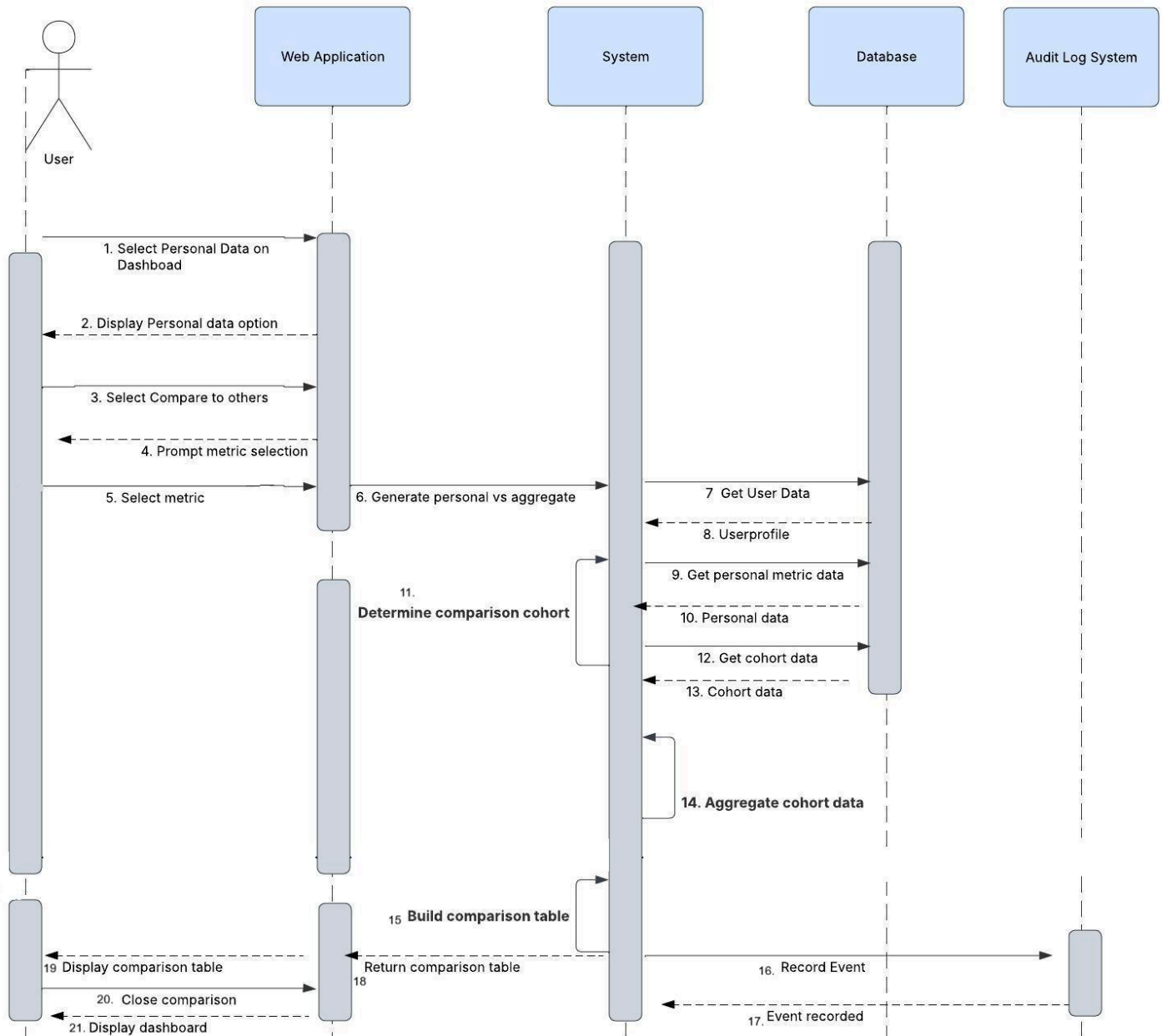
User Completes/Submits Form



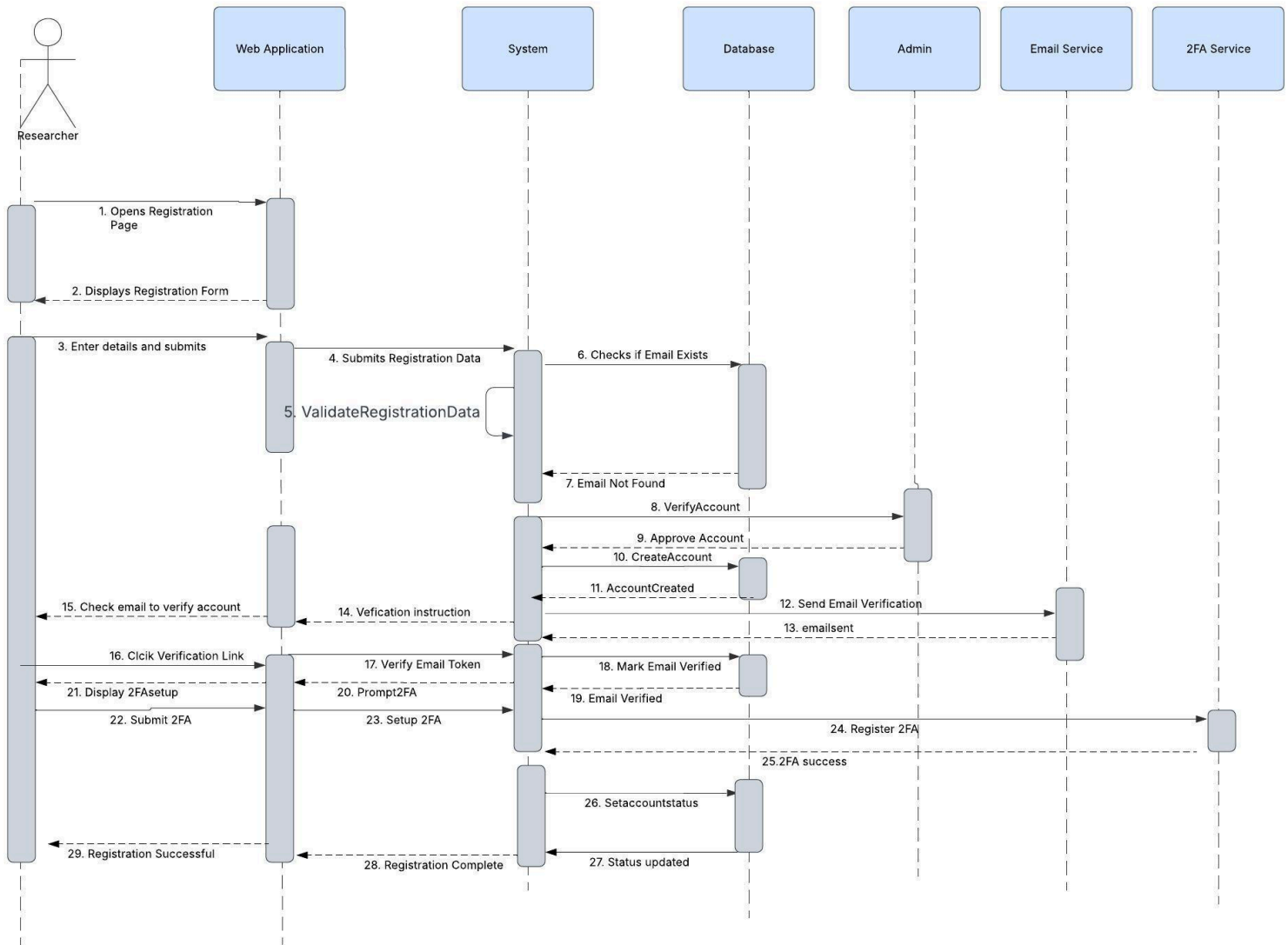
User Generates Personal Report



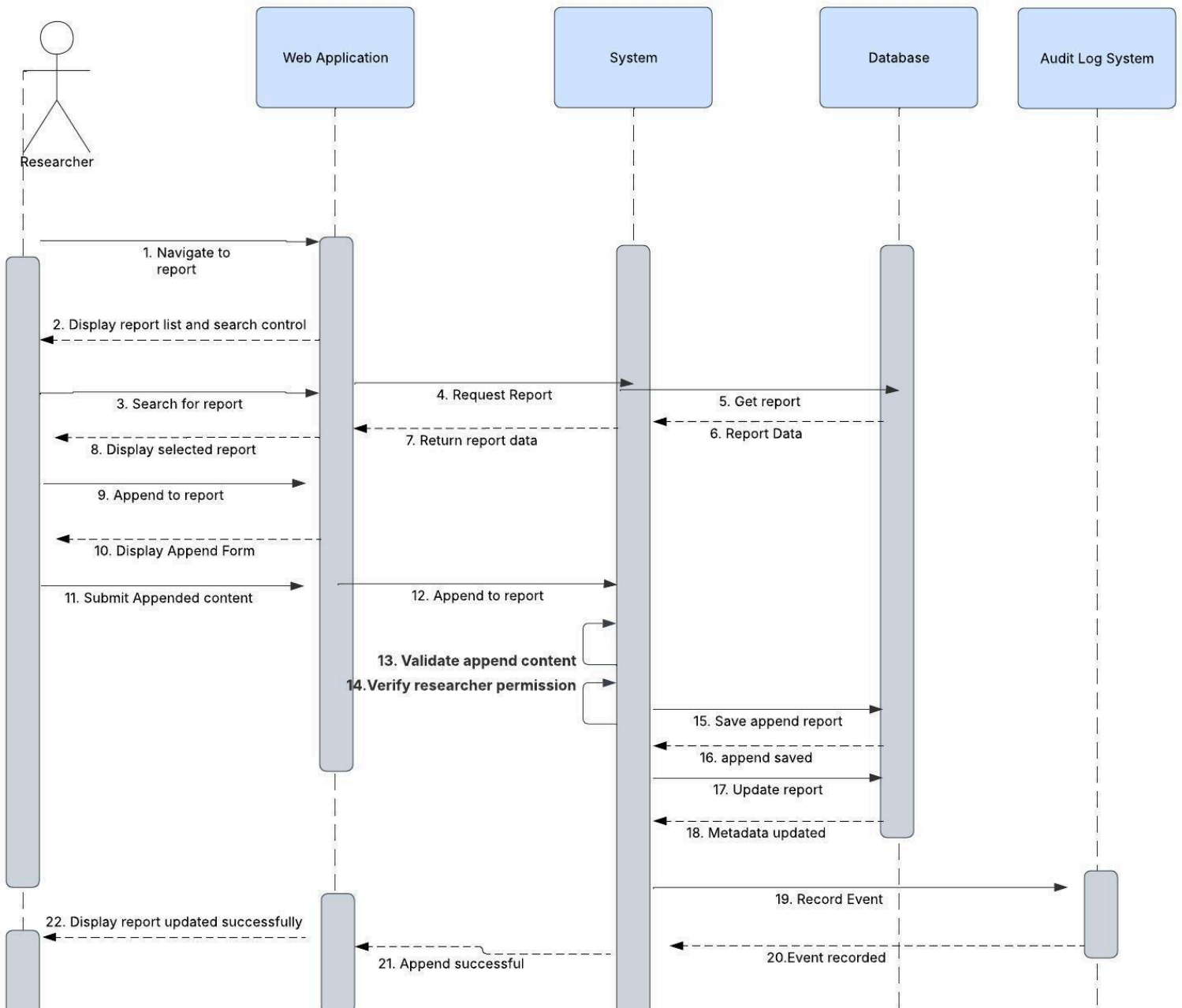
User Reviews Personal Data v/s Aggregate



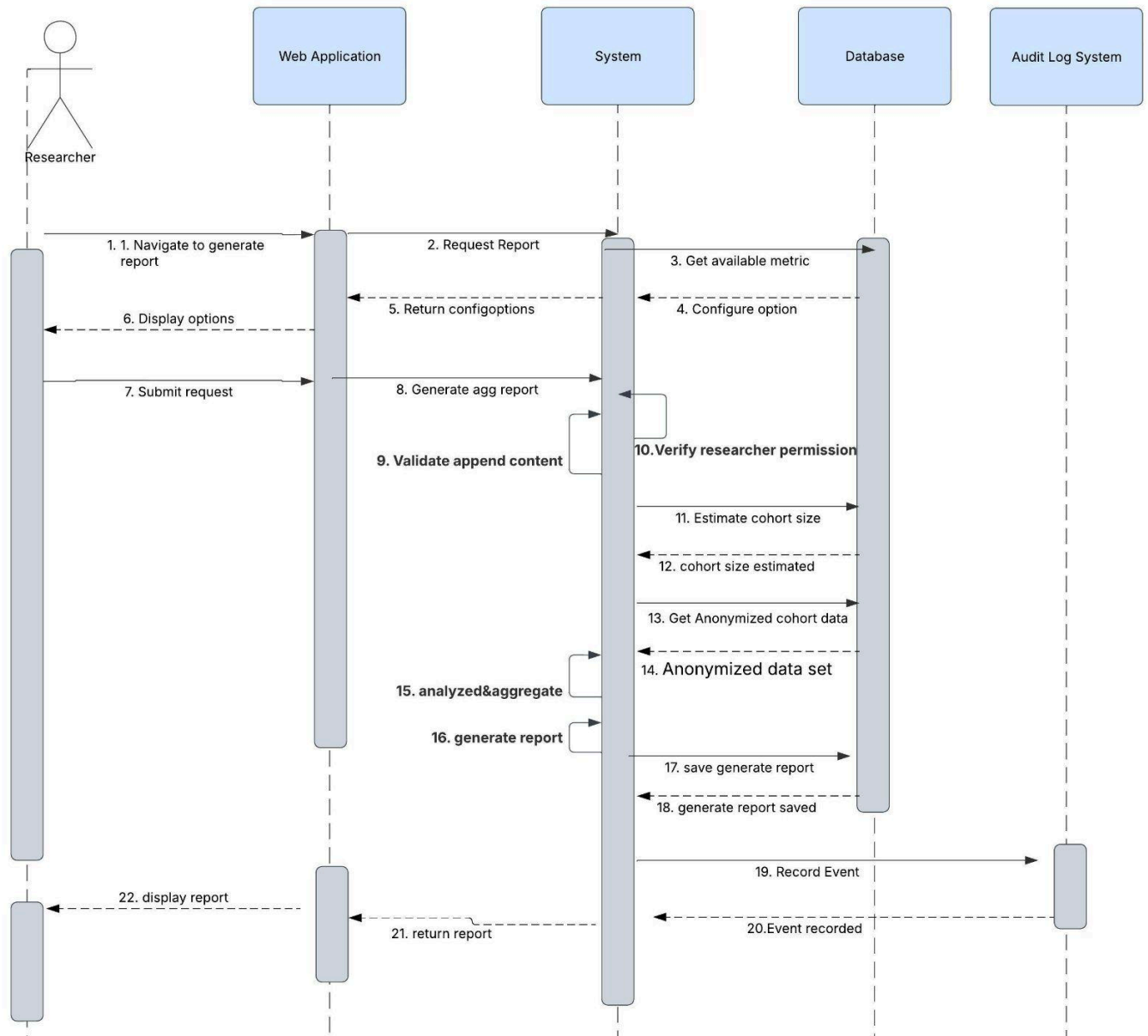
Researcher Registers Account



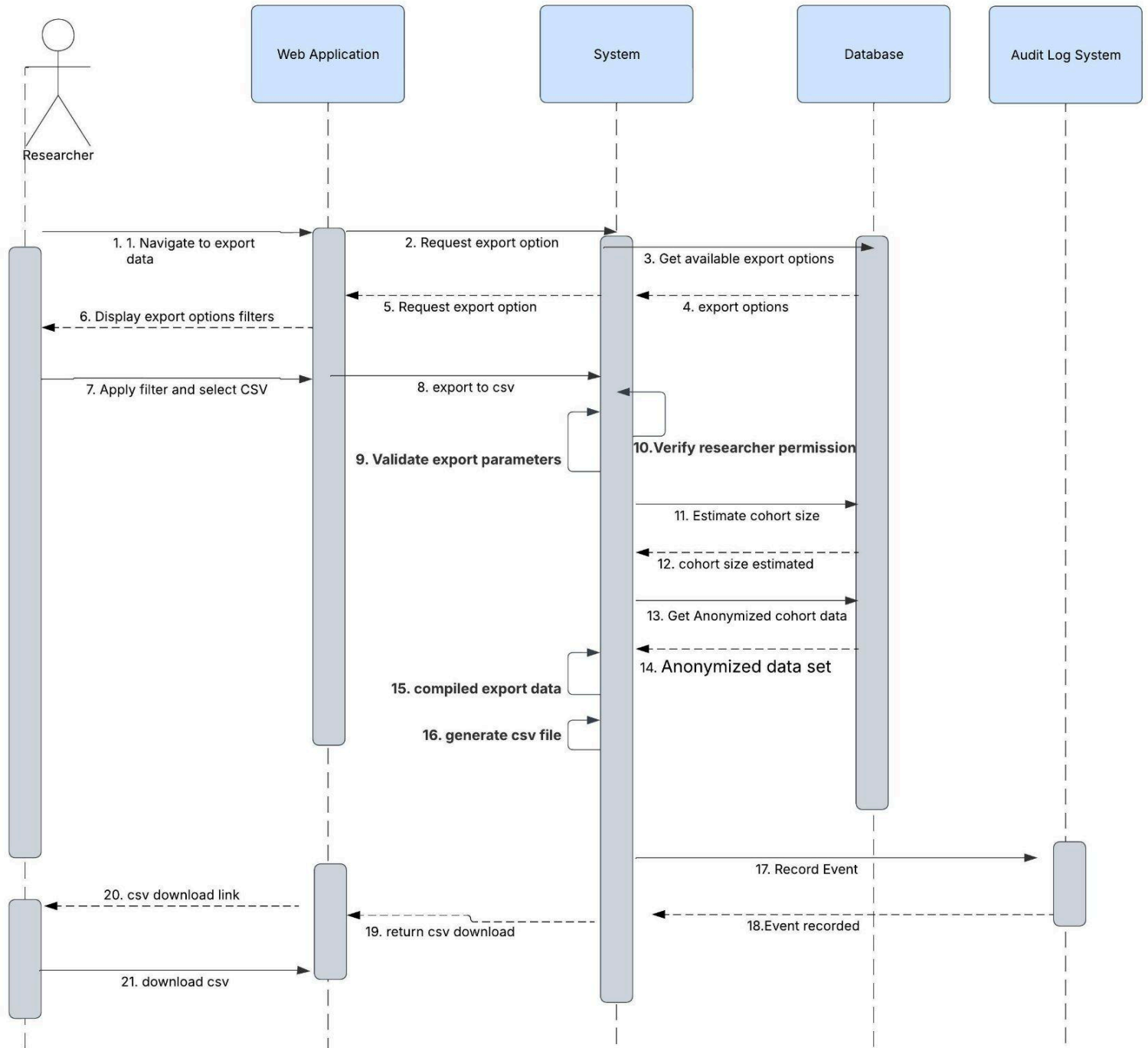
Researcher Appends Report



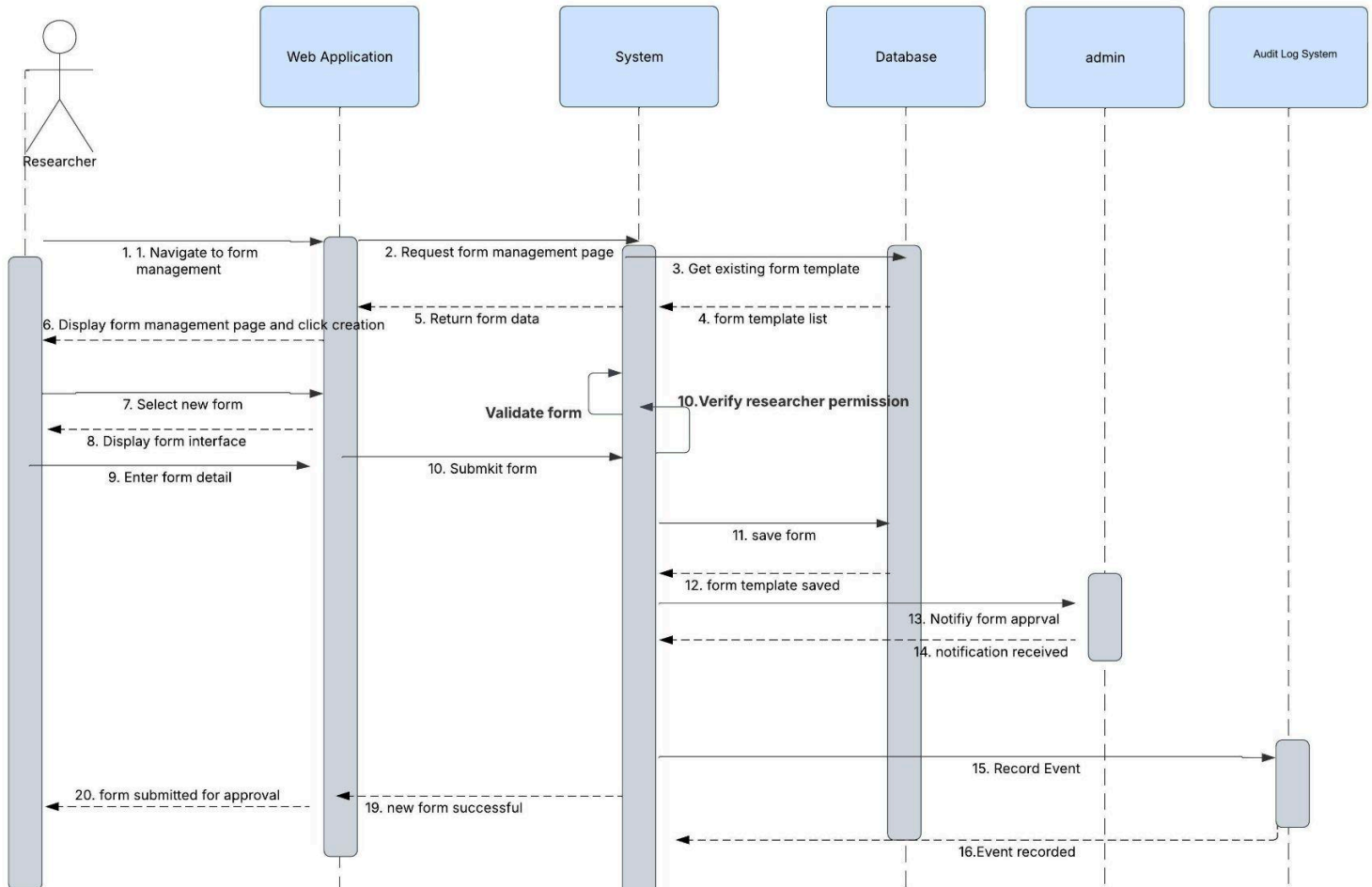
Researcher Generates Anonymous Report



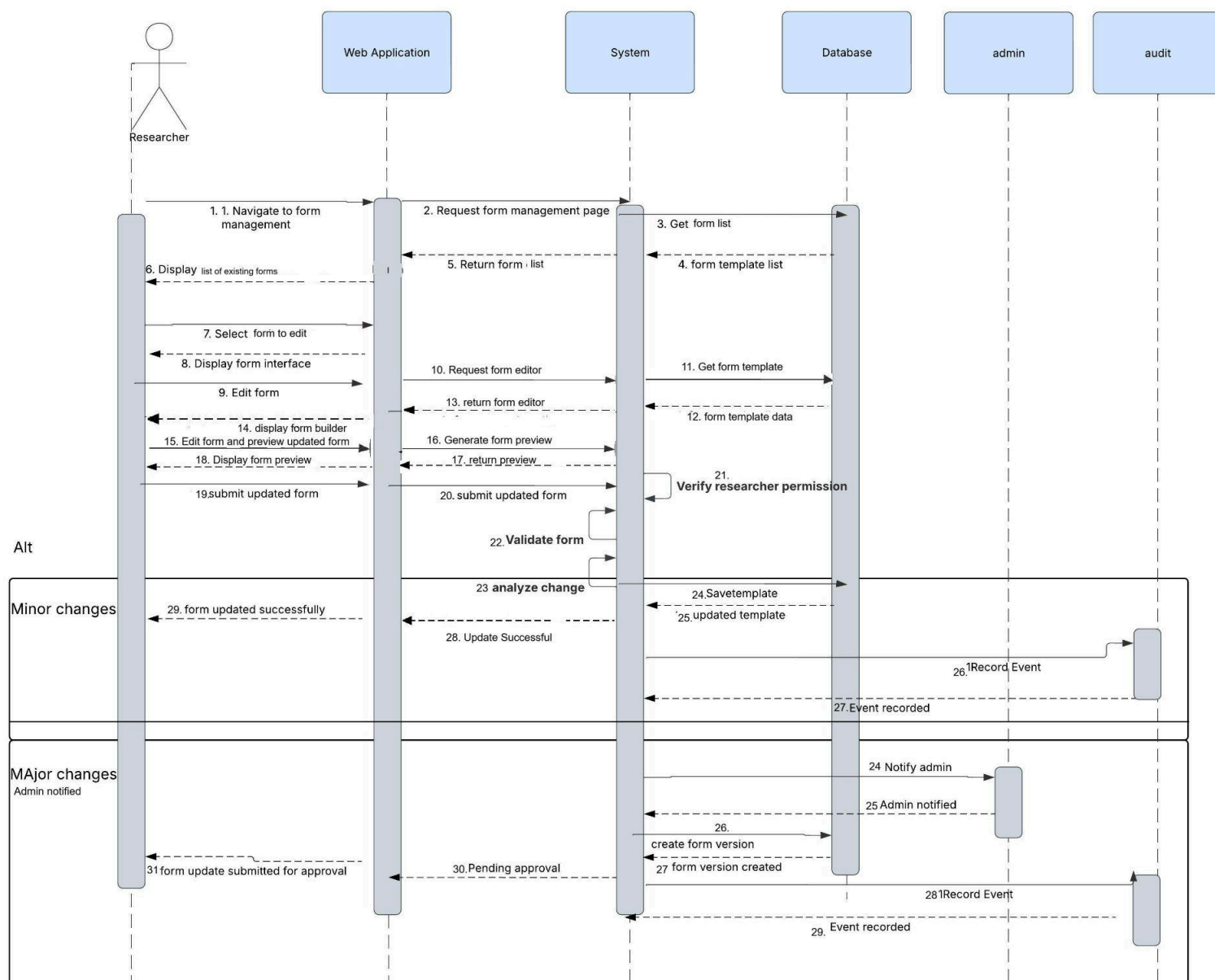
Researcher Exports Data



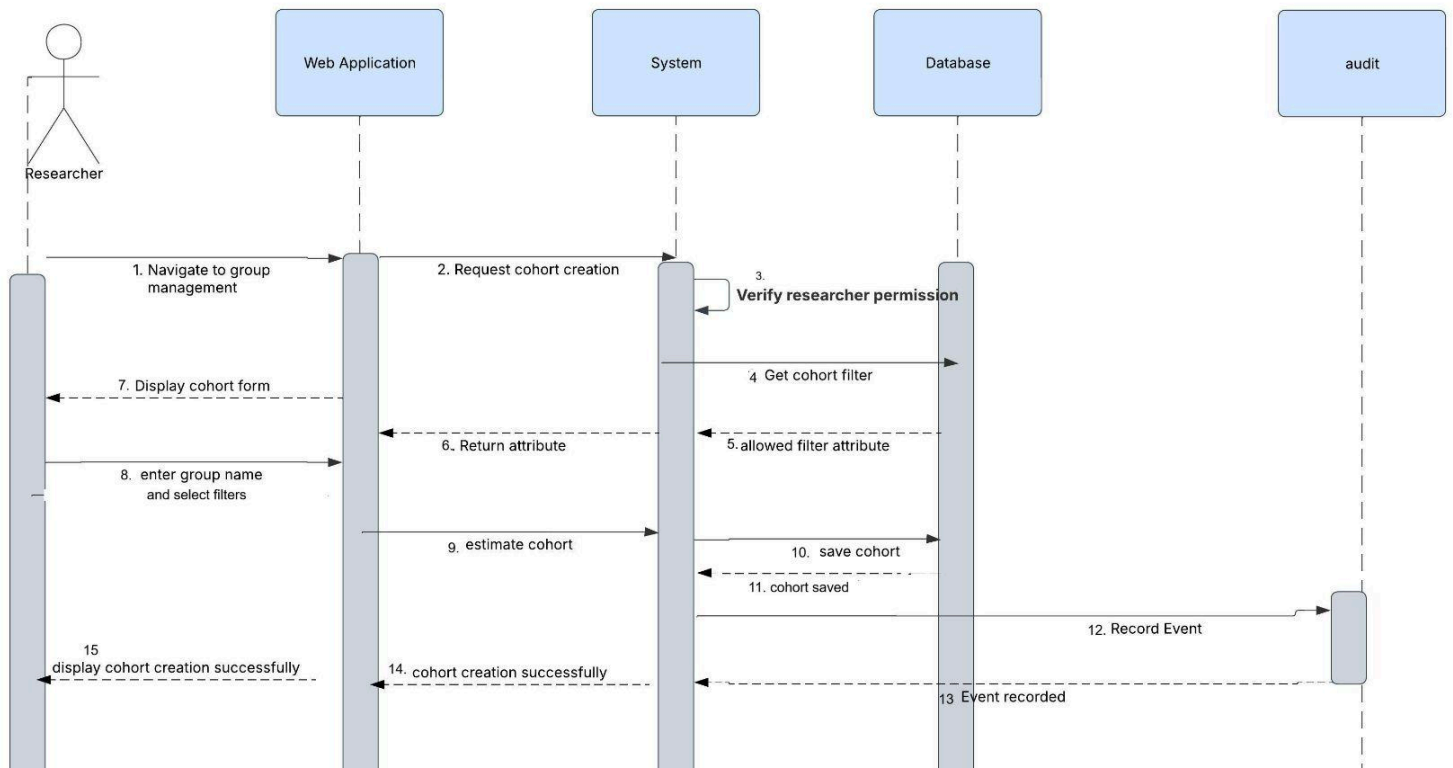
Researcher Adds New Form



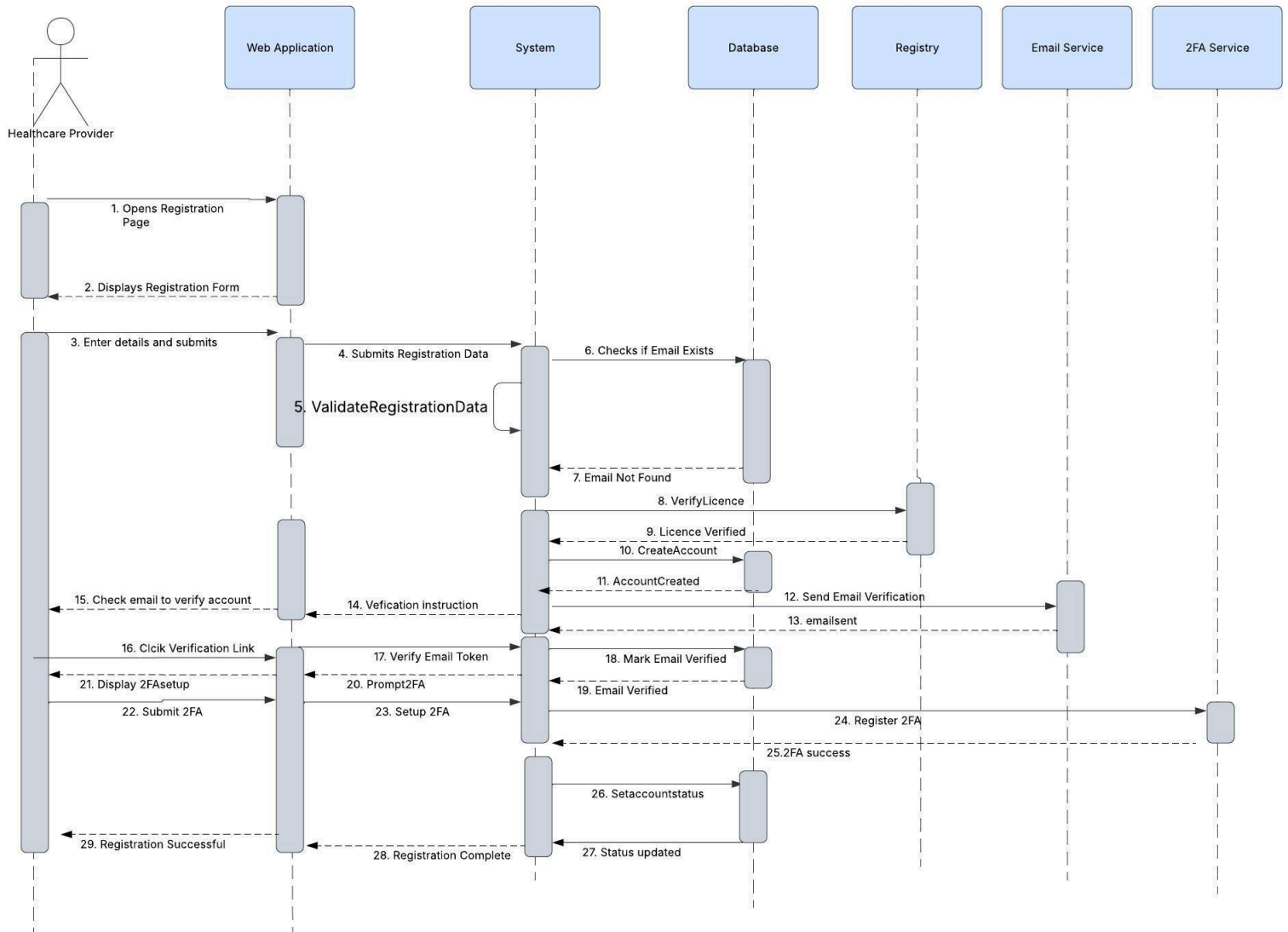
Researcher Updates Existing Form



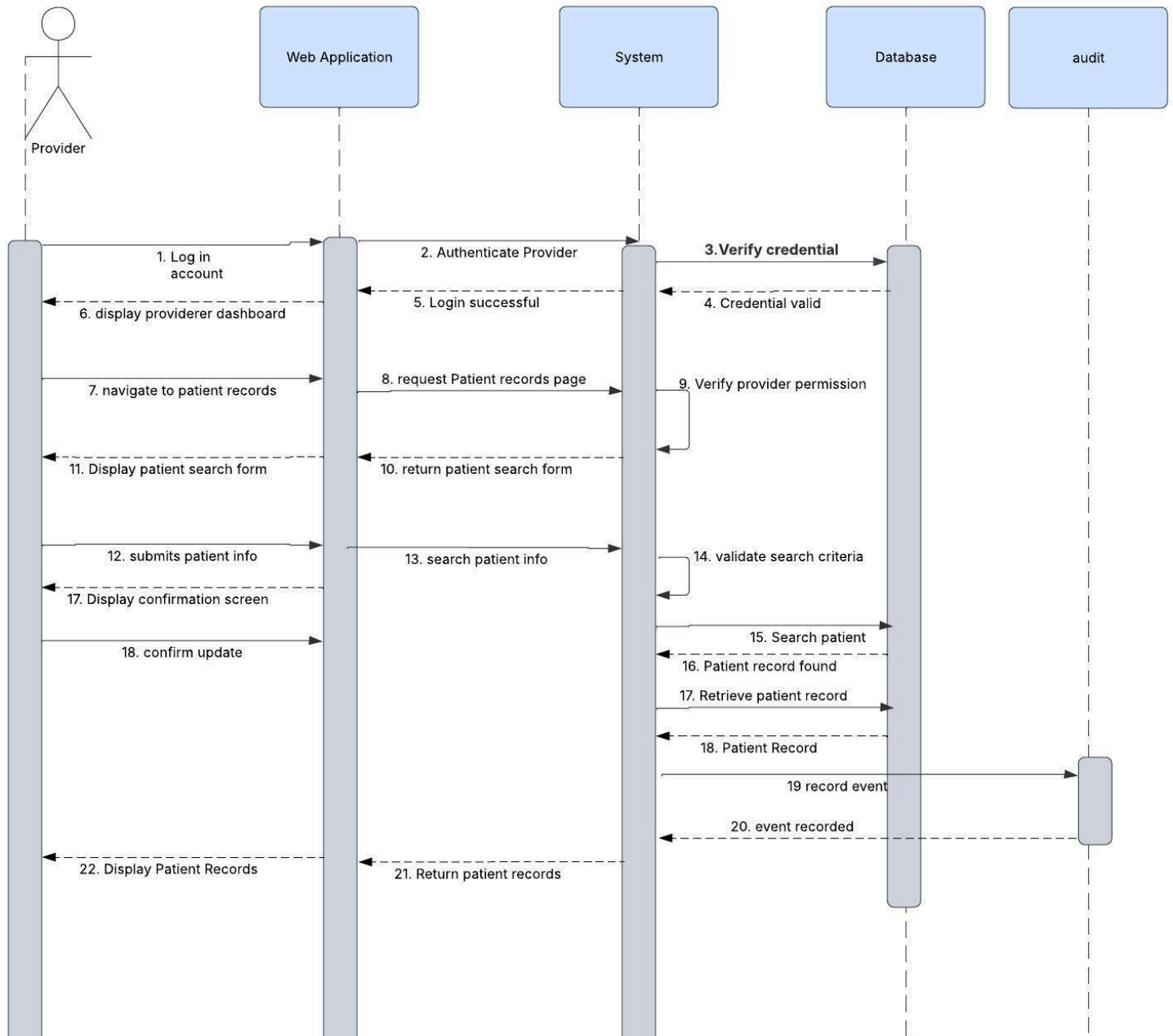
Researcher Creates Cohort



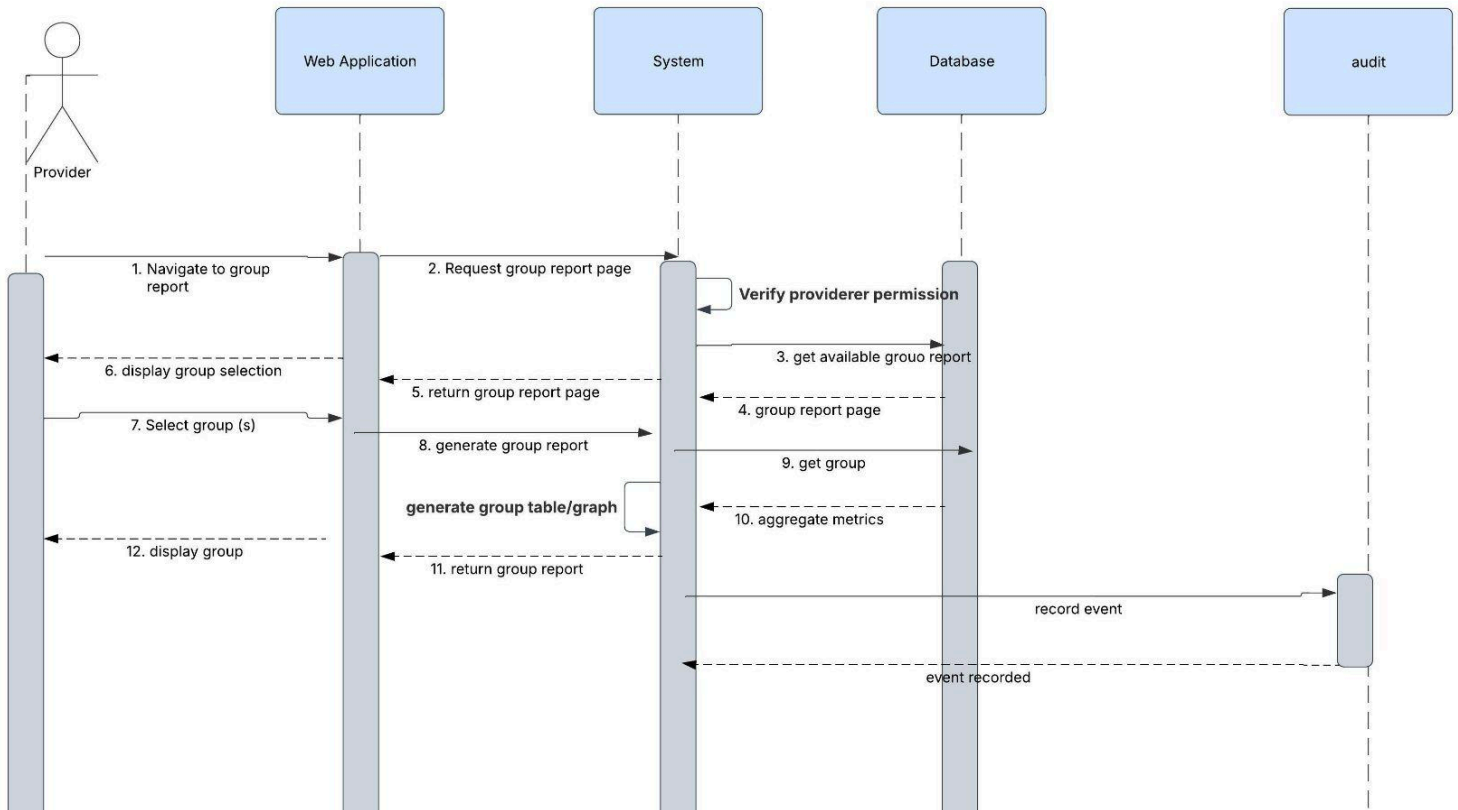
Provider Registers Account



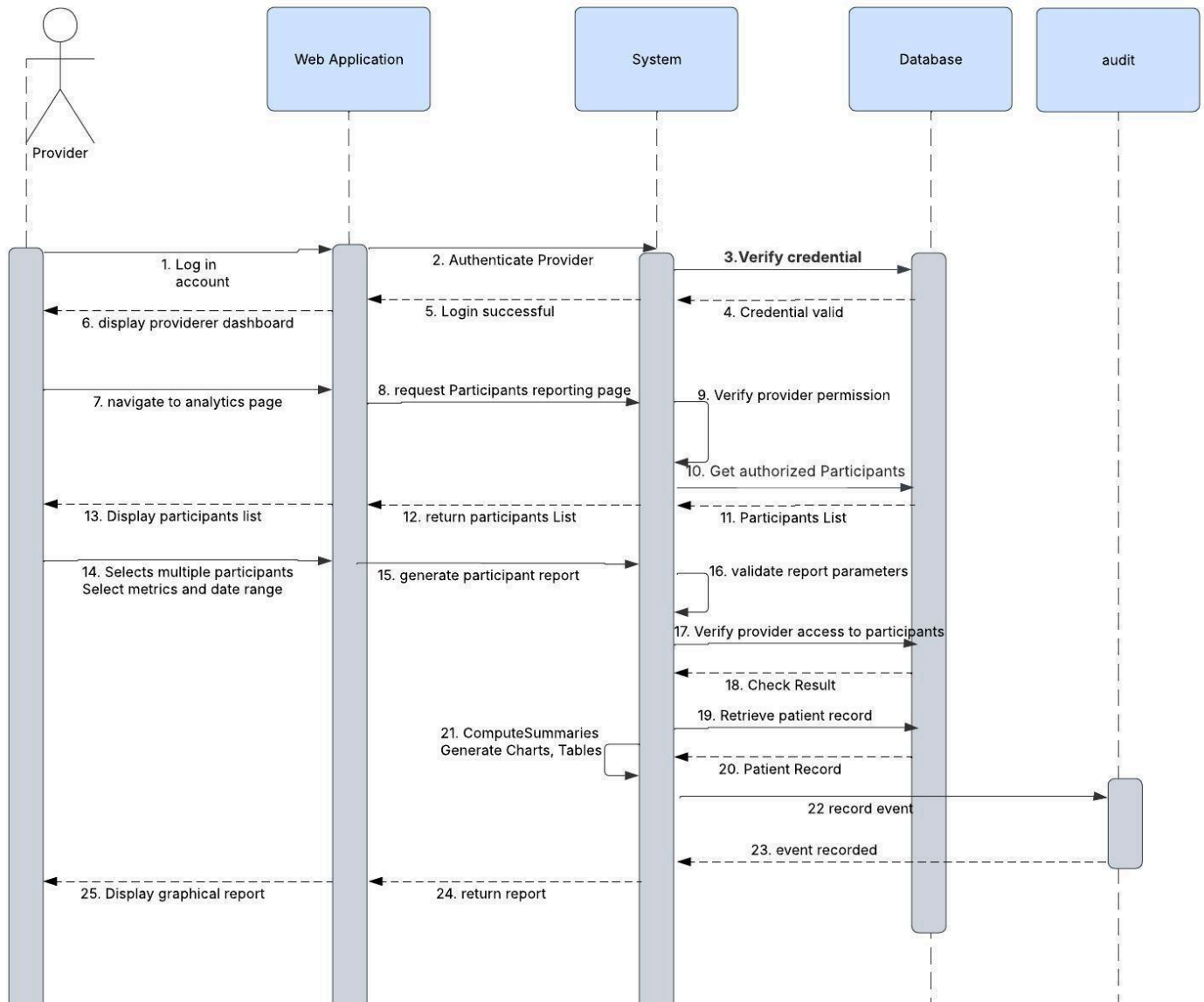
Provider Accesses Patient Data



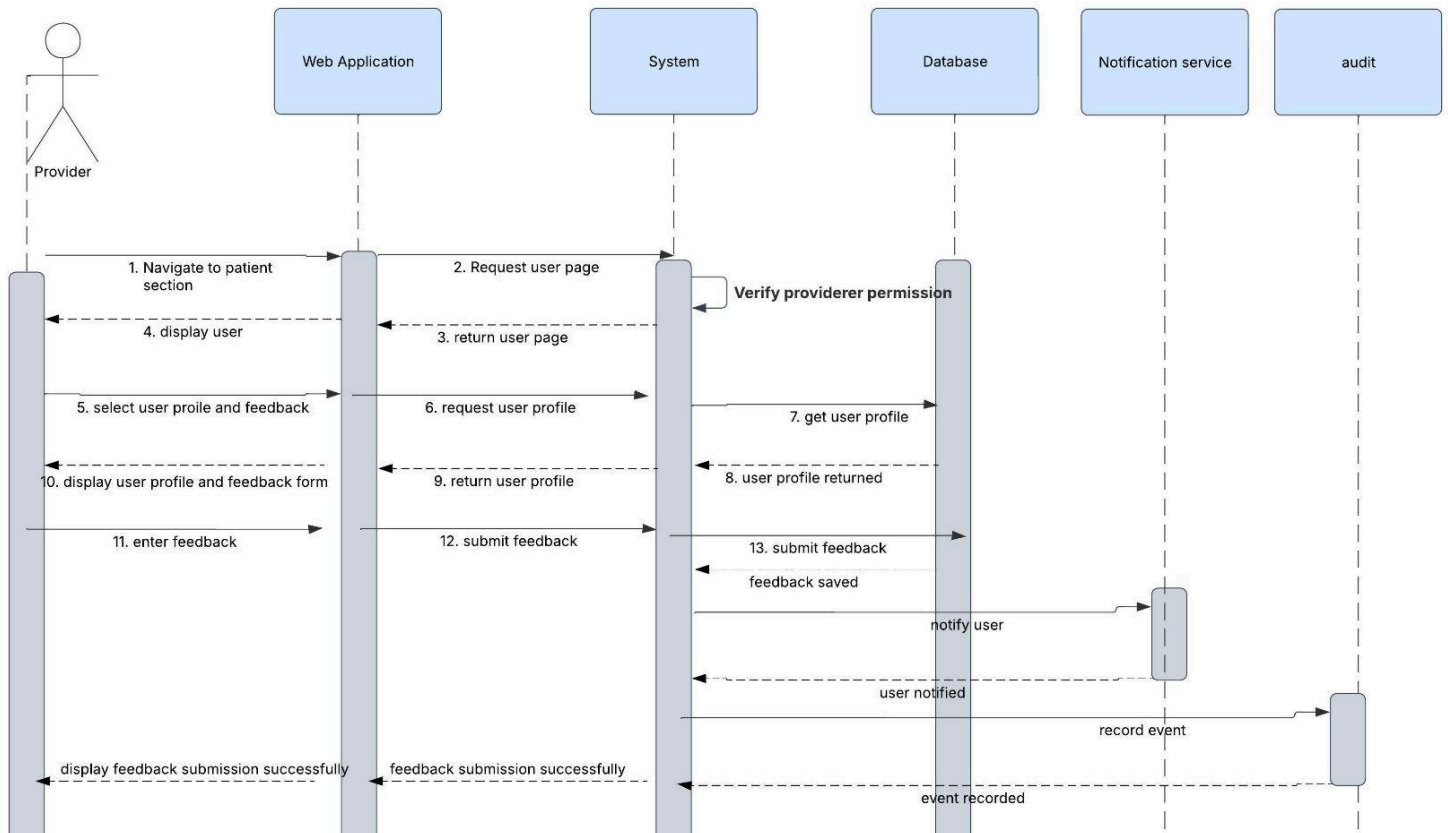
Provider Views Report for Selected Group(s)



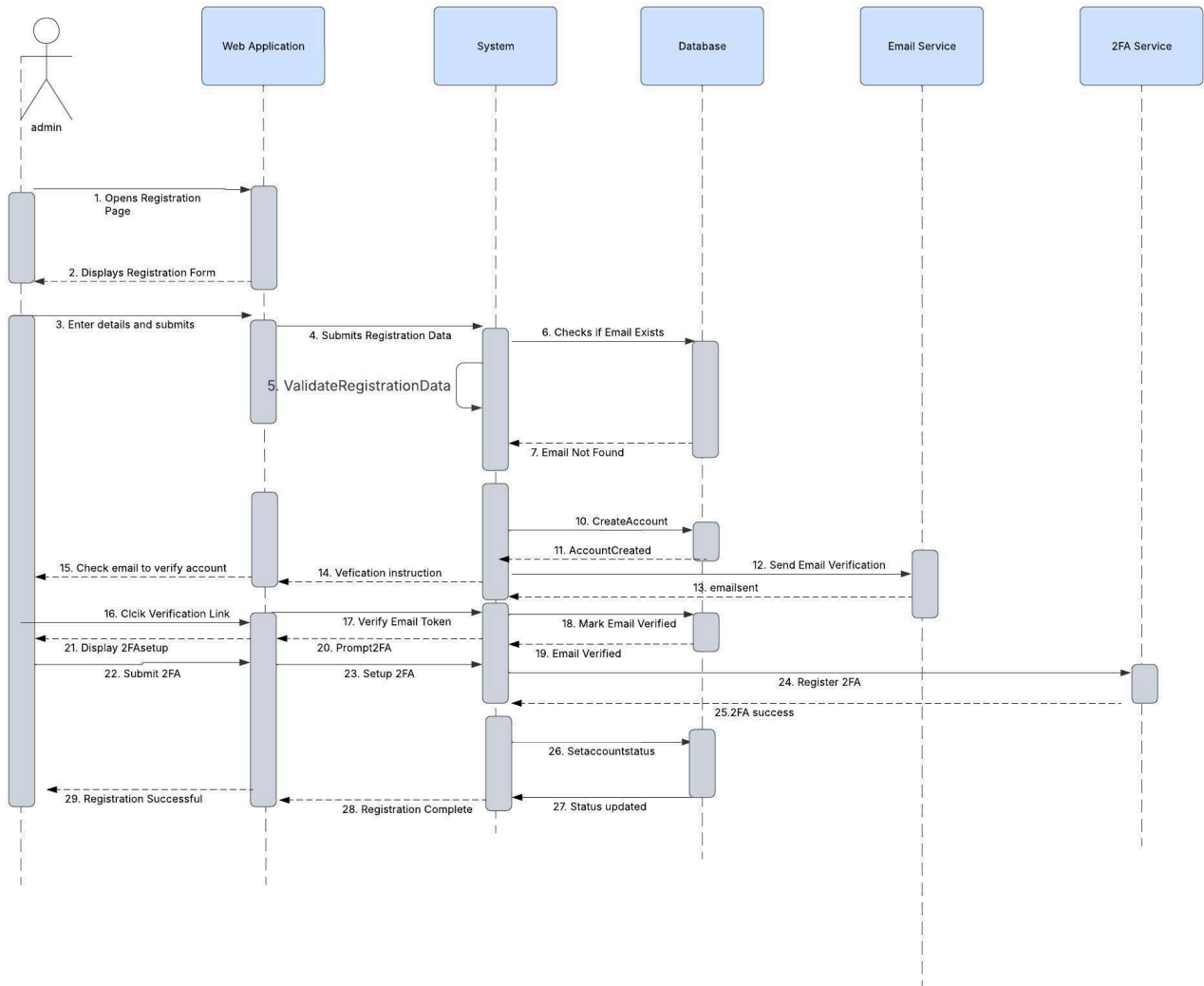
Provider Views Report for Selected Participants



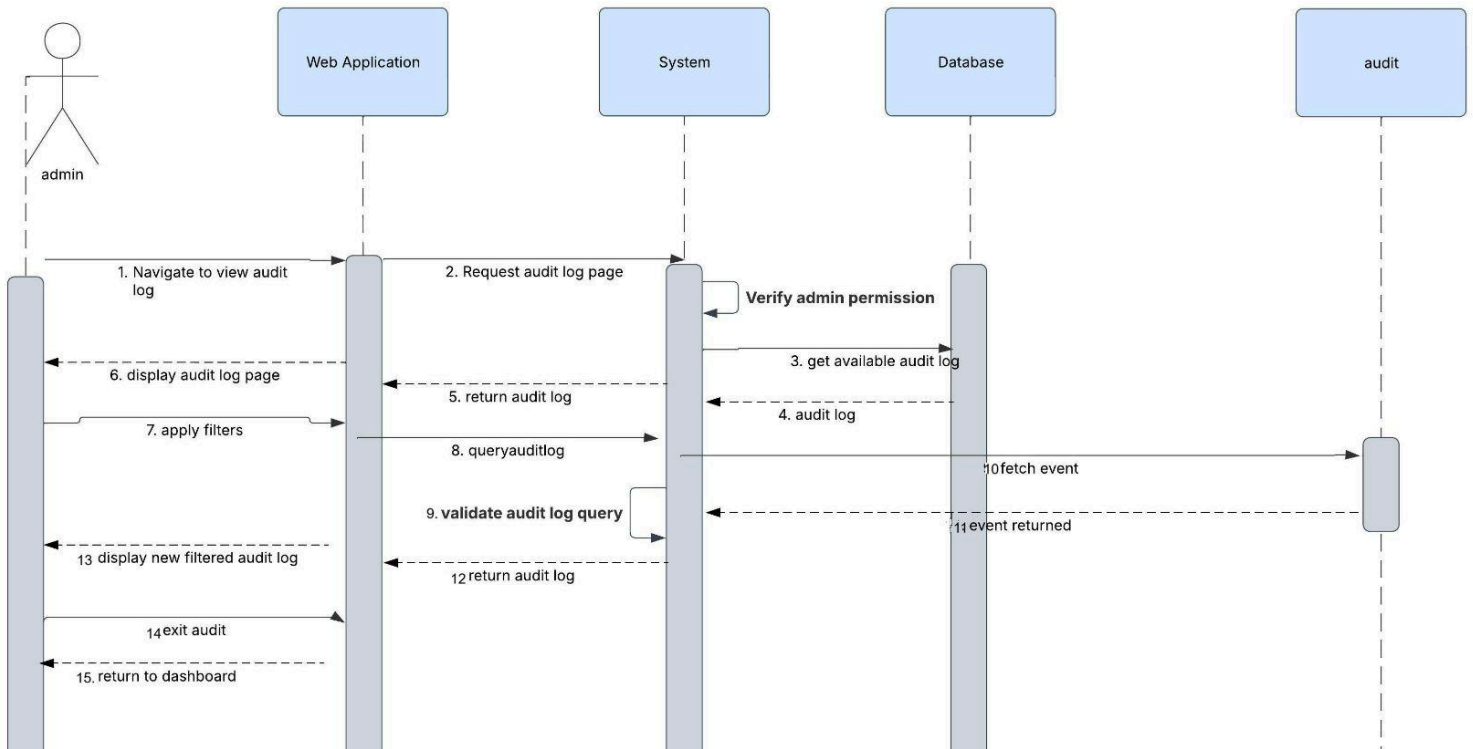
Provider Provides Personalized Feedback



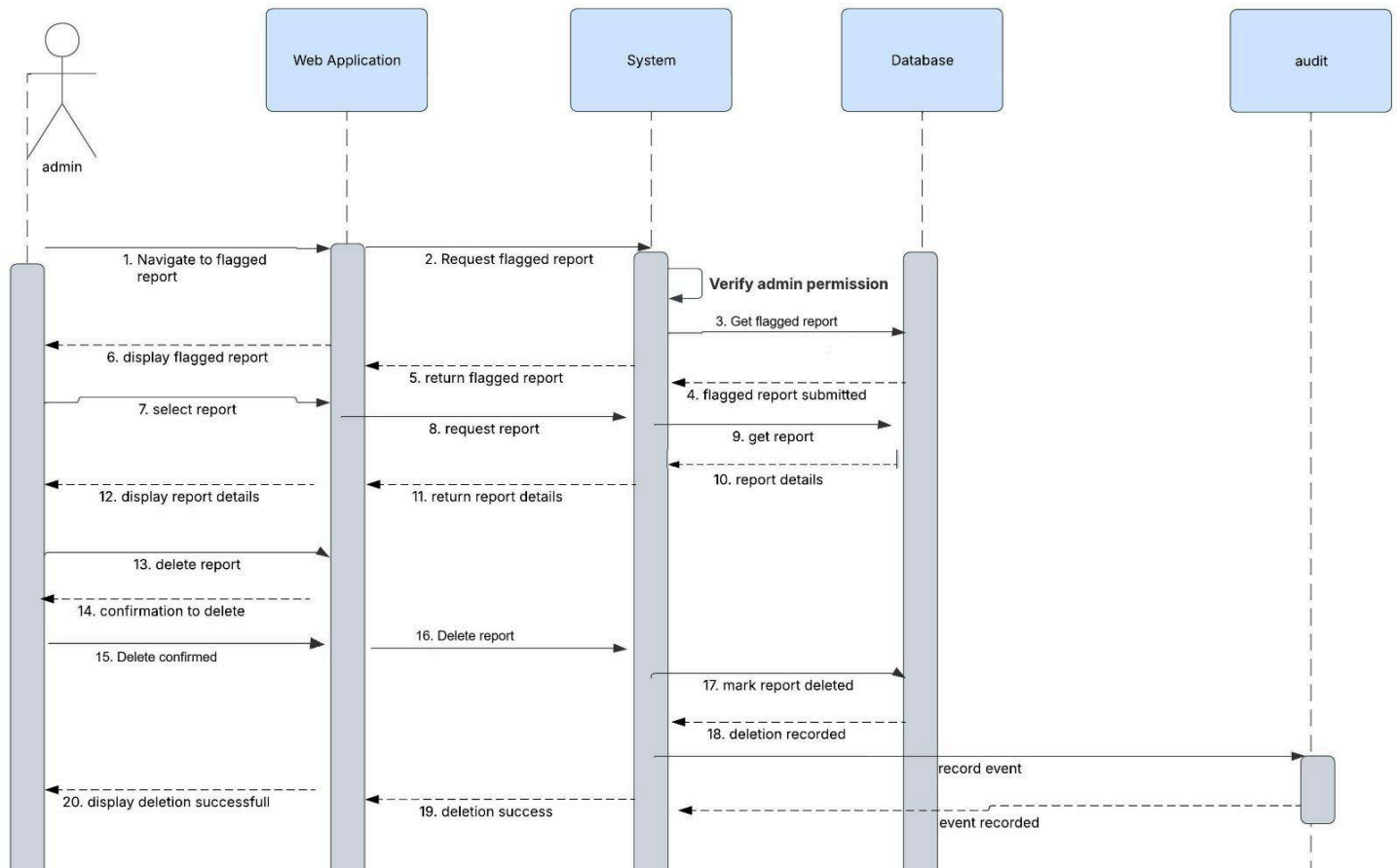
Admin Registers Account



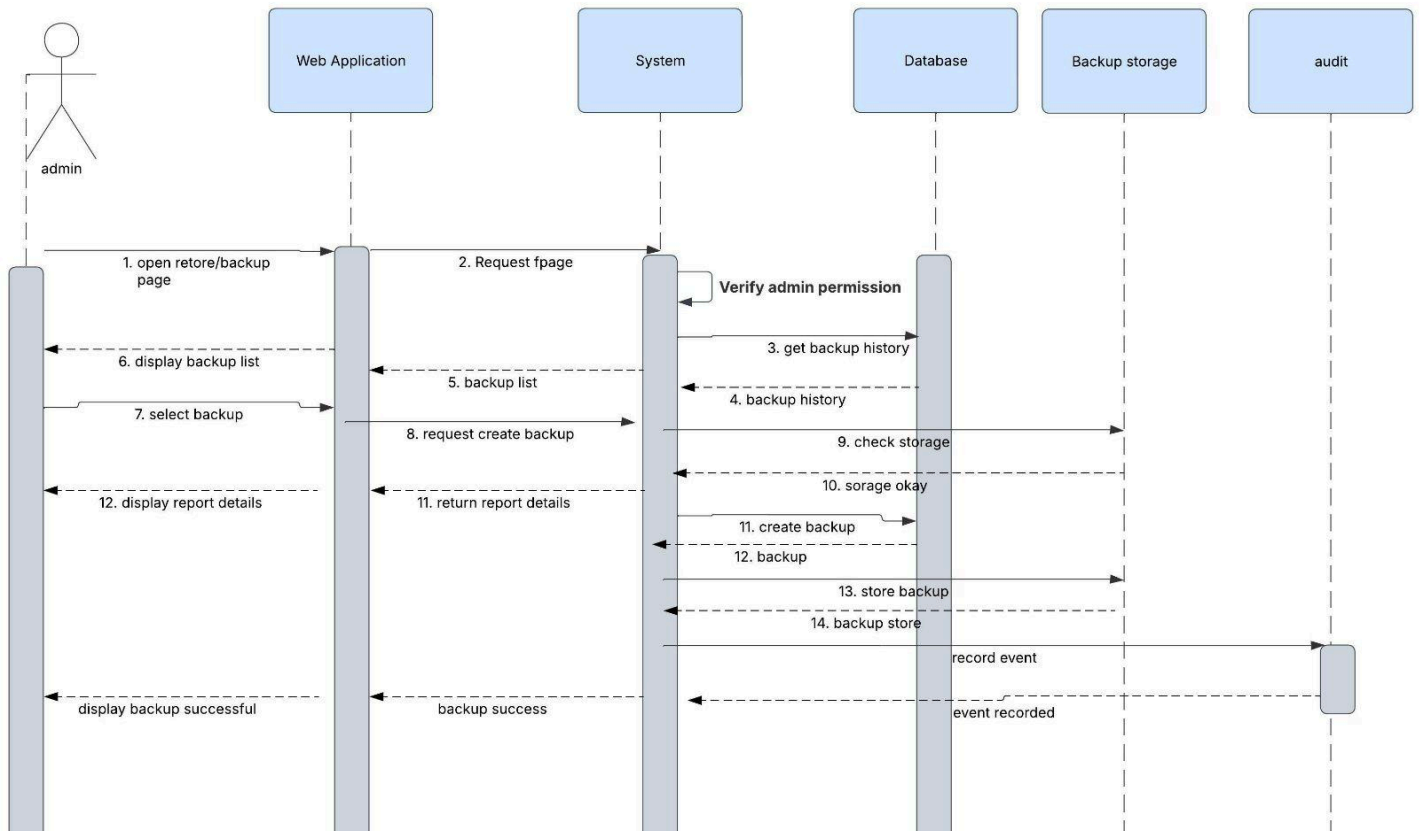
Admin Views Audit Log



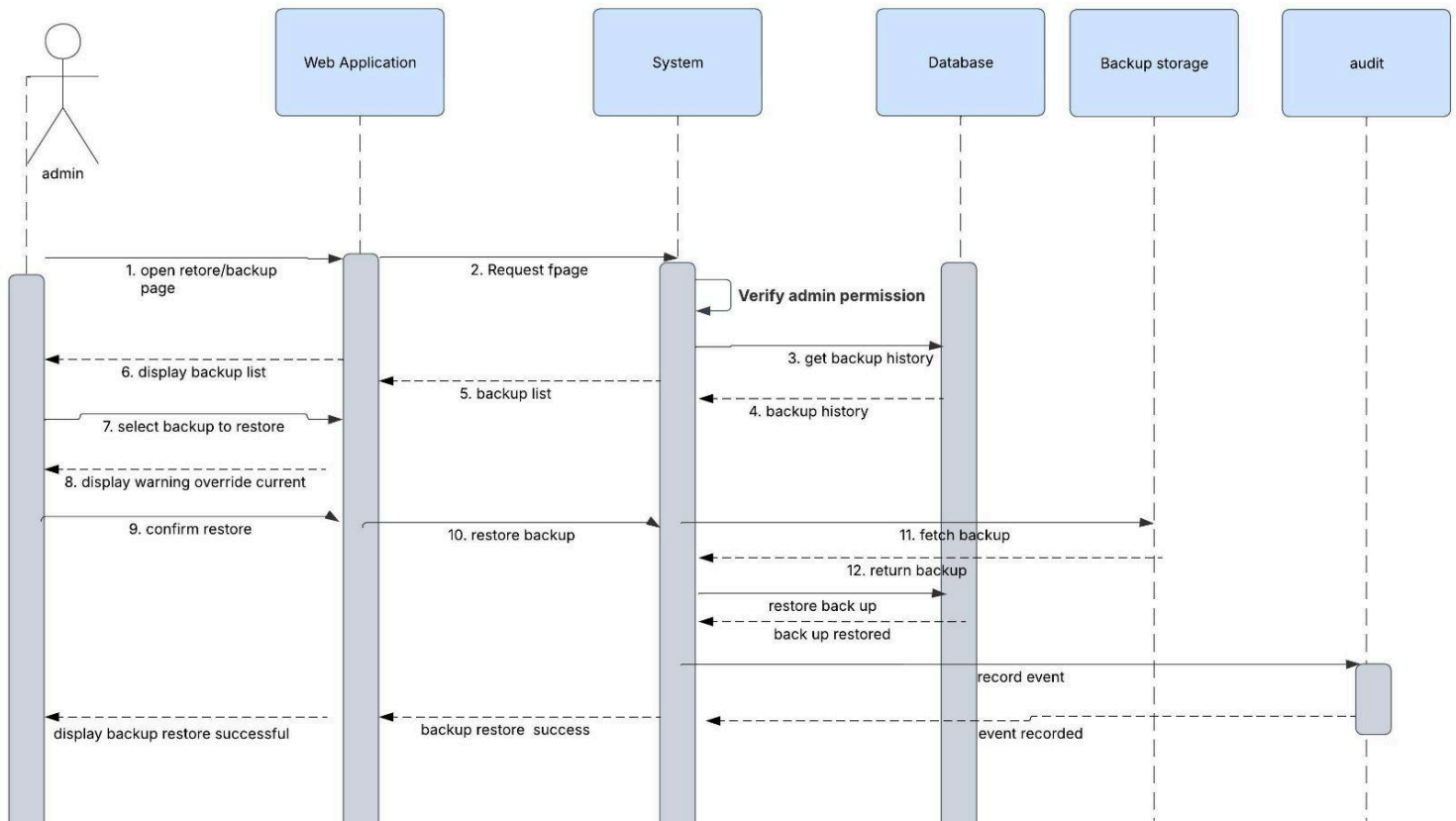
Admin Deletes Problematic User Report



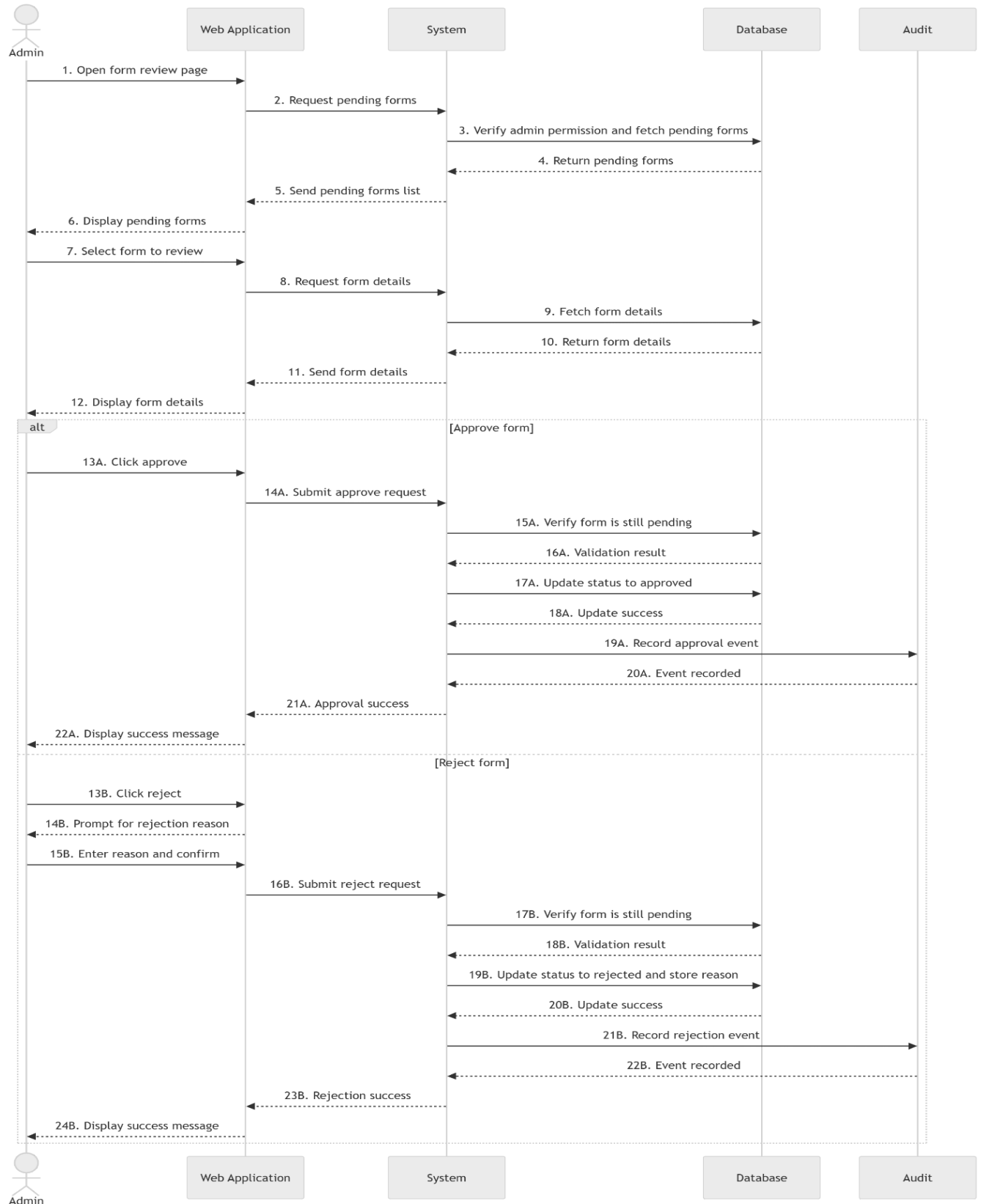
Admin Backs Up Database



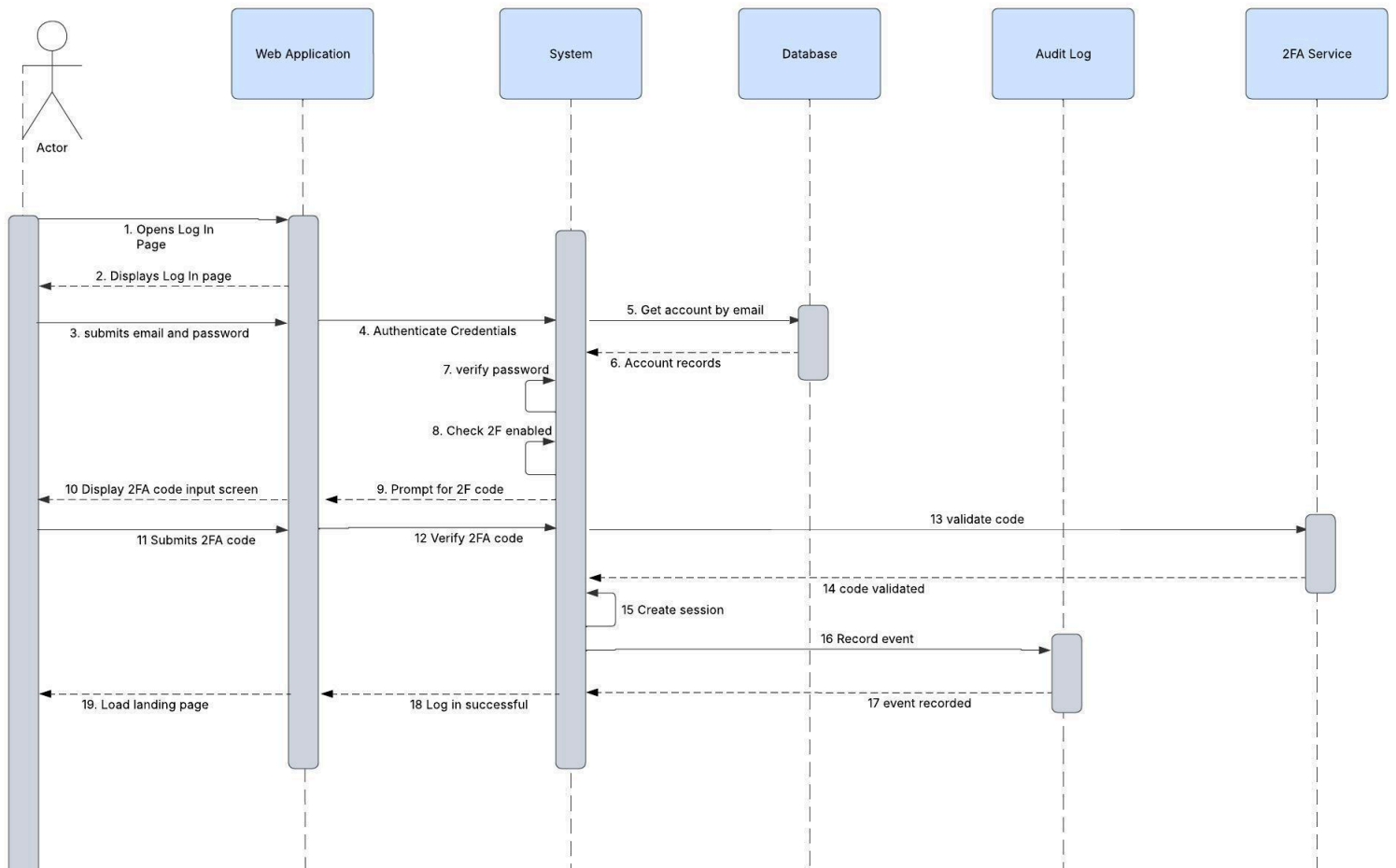
Admin Restores Database



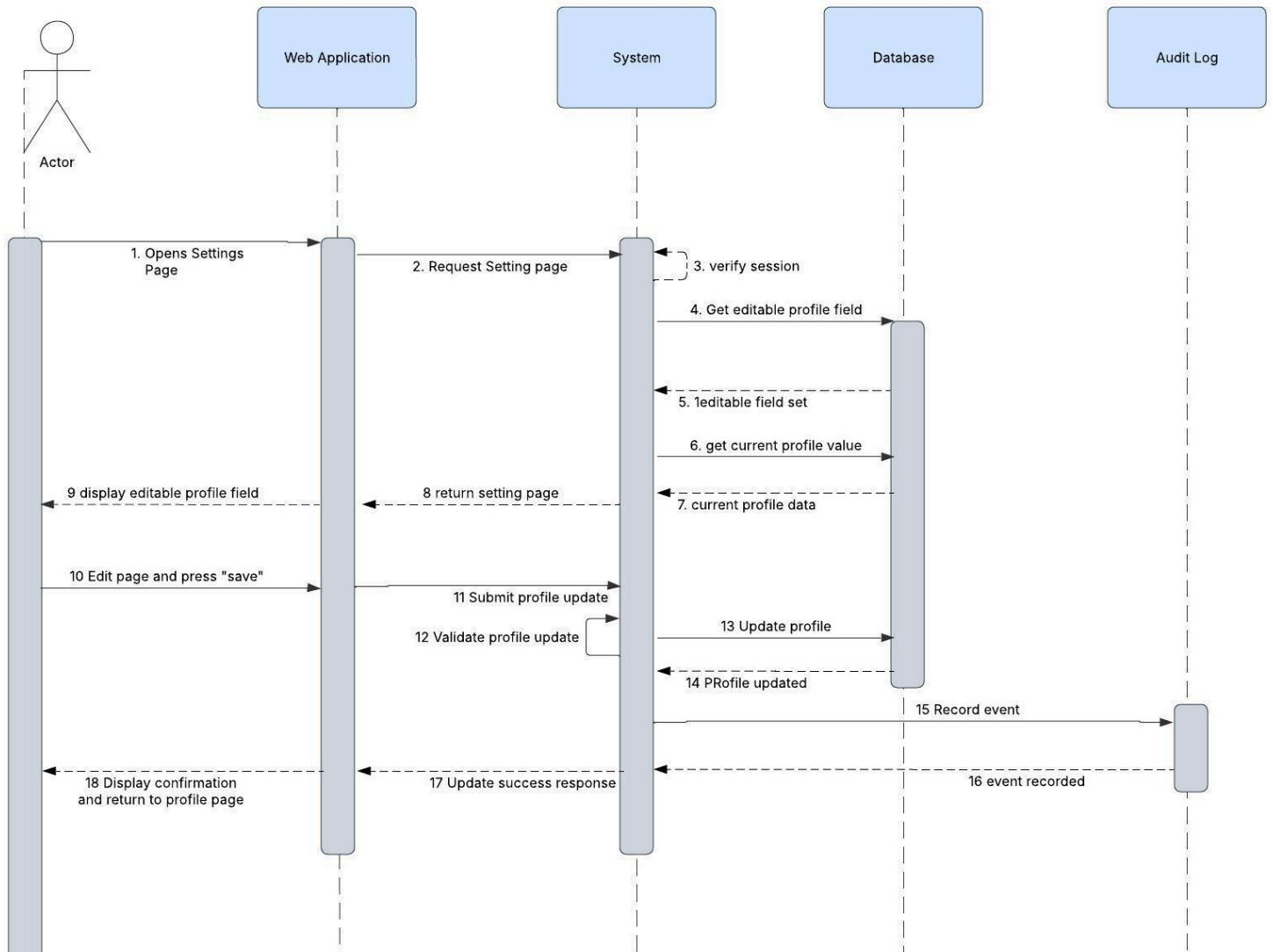
Admin Approve/Reject Forms



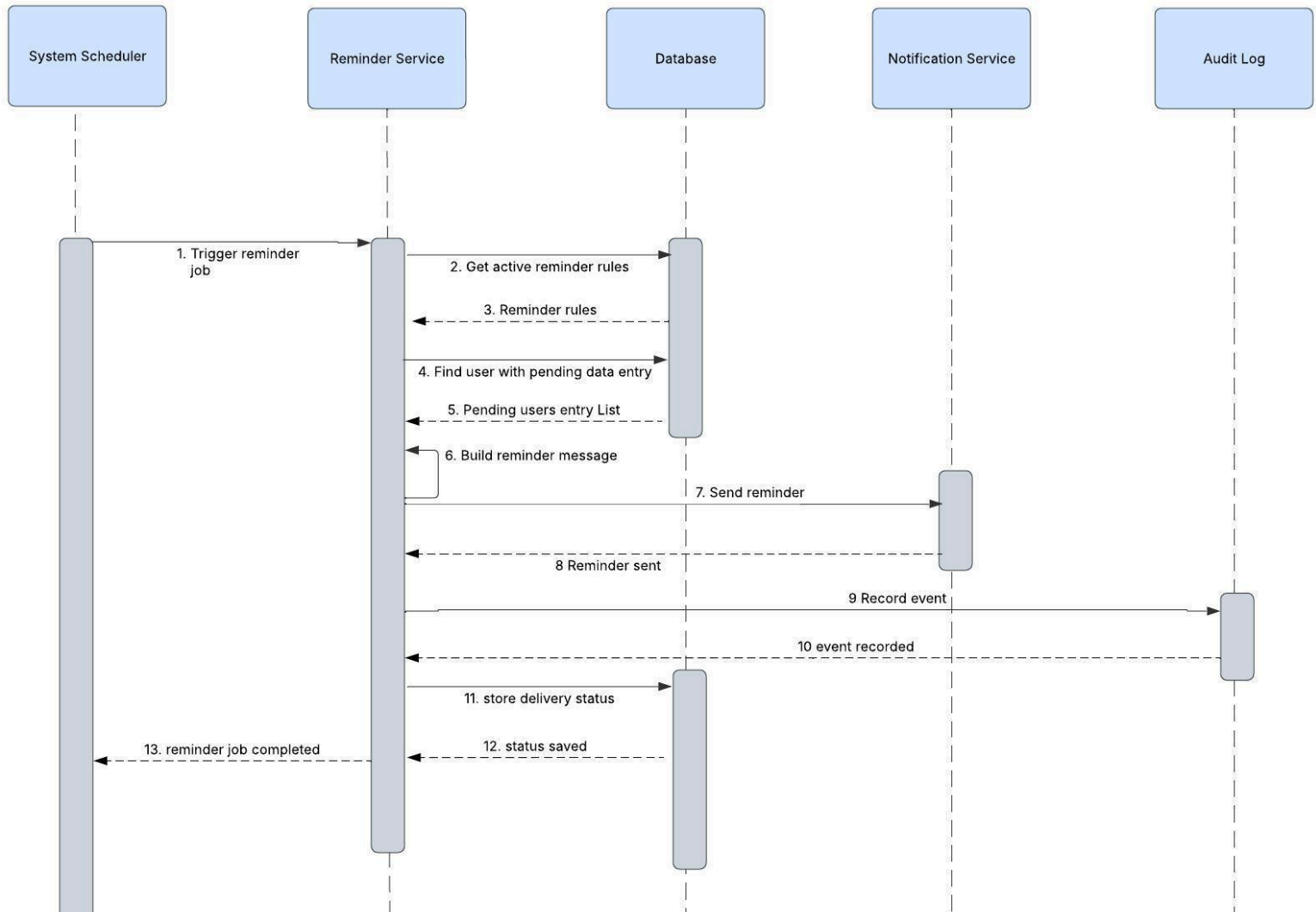
Log In



Update Account Information (All Roles)



Stretch Goal: Reminders for Data Entry



Stretch Goal: System-Generated Suggestions (Automated Insight)

